

COMS 6998: AI-Native Computing
Columbia University
Guest Lecture

Introduction to AI Accelerators



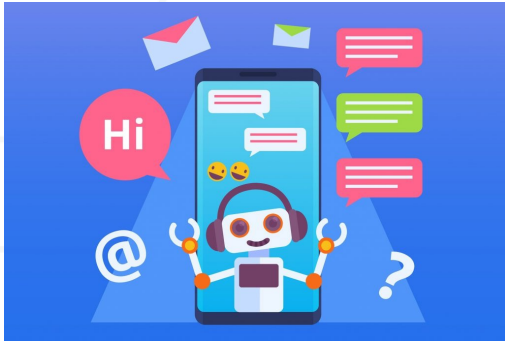
Tushar Krishna
Associate Professor
School of ECE
Georgia Institute of Technology

tushar@ece.gatech.edu

<http://tusharkrishna.ece.gatech.edu>

AI is pervasive today!

Chatbots



Code Generation

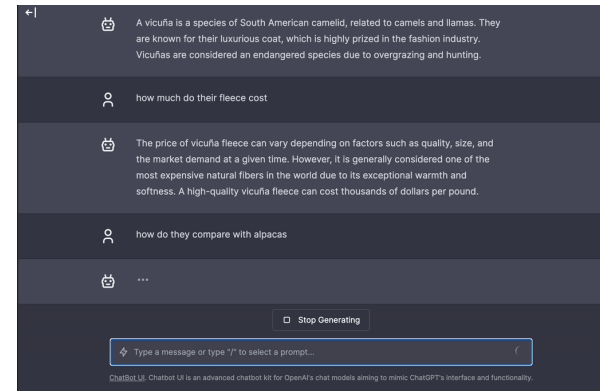
```

1 segmentation.rb
2
3 def segmentation(items, separator)
4   curr = []
5   segments = []
6   items.each do |item|
7     ...
8   end
9
10 segmentation([1,2,4,0,2,5,0,3,0], 0).each do |segment|
11   puts(segment.join(", "))
12 end
13

```

"25% of code at google in last quarter was AI generated"

Text Generation



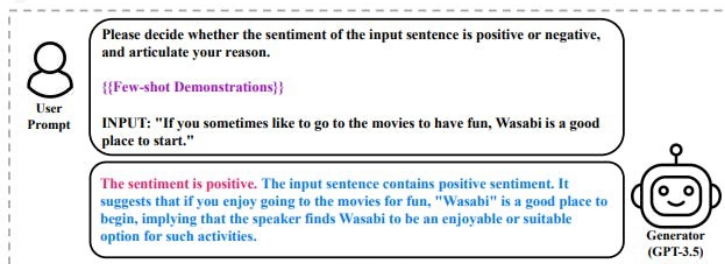
Language Translation

[Instruction]: Translate the following sentences from English to Chinese.
[Input]: Did you see it go?

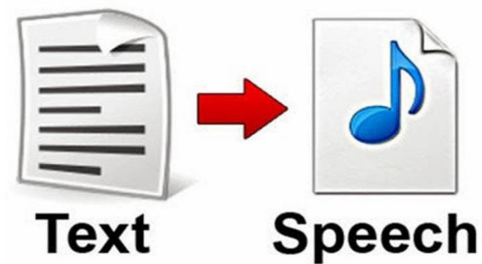
LLM

[Output]: 看清楚了吗?

Sentiment Analysis



Text to Speech

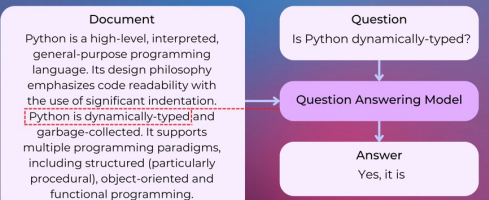


Recommendations



Question Answering

Generative Question Answering



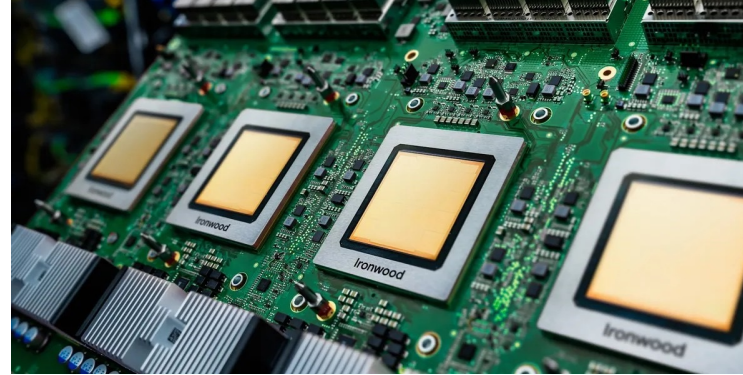
➡ Algorithmic view of AI (Datasets and Models)

Computer Architect's view of AI

“AI Datacenters”
“AI Supercomputers”



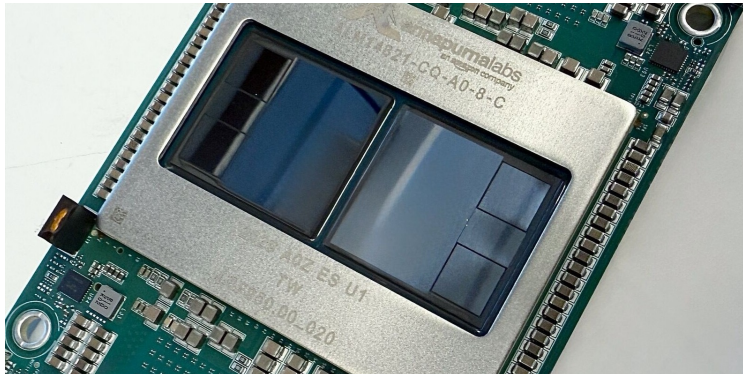
NVIDIA
Vera Rubin NVL72



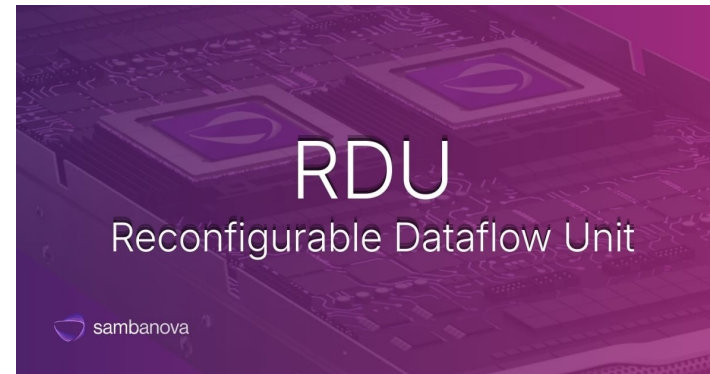
Google
TPU v7 Ironwood



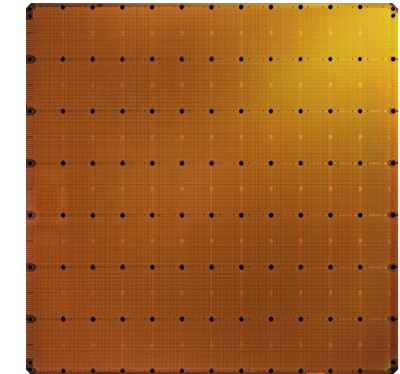
AMD
Helios rack (MI450)



AWS
Trainium3



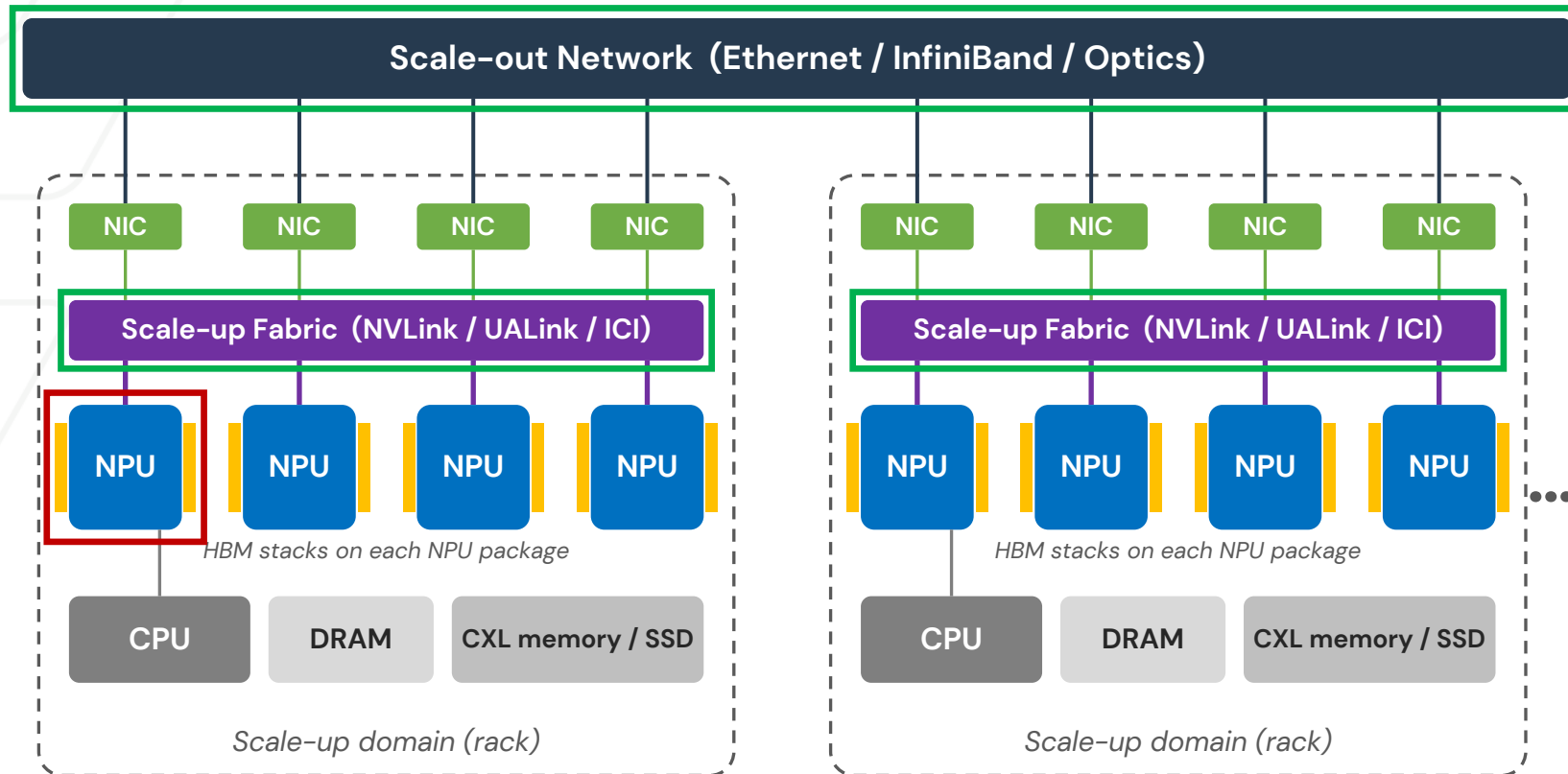
SambaNova
SN50 RDU



Cerebras
WSE-3 Turbo

10000 ft view

Today's class



- ✓ Customized accelerators for AI (aka NPUs)
- ✓ Customized network fabrics to scale the AI task across multiple accelerators

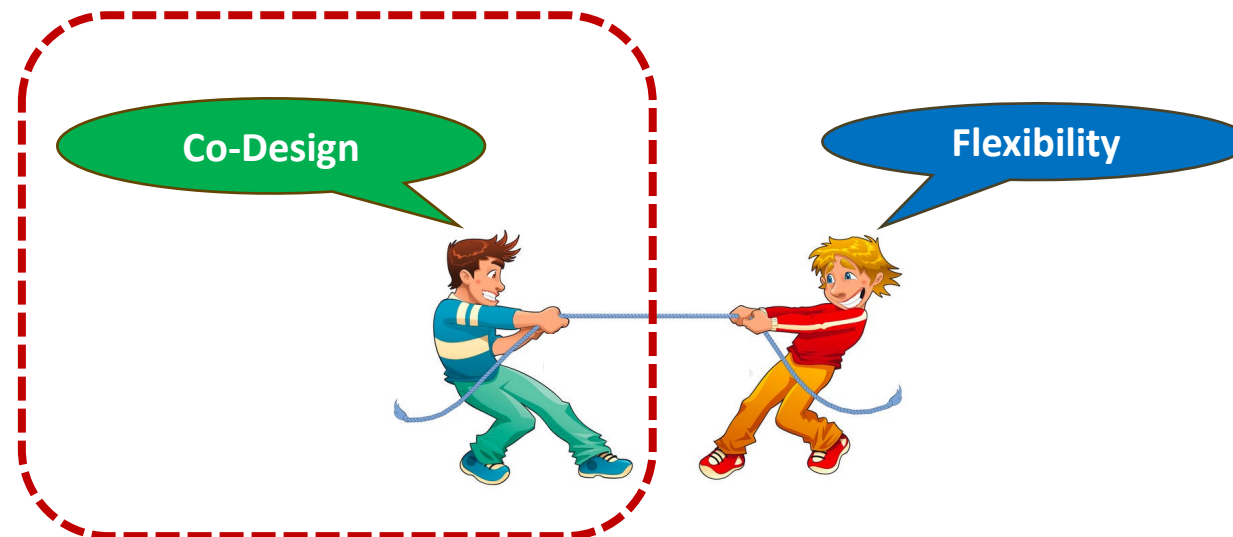
How to design and optimize AI Datacenters?

"AI is not just about algorithms and software; it's about creating the right hardware infrastructure that can enable these algorithms to run efficiently. **Hardware and software must be co-designed** to fully unleash the potential of AI."

— Jensen Huang, CEO of NVIDIA

"The future of computing is going to be about **flexibility** — flexibility to create new architectures, flexibility to scale, flexibility to bring AI into every single thing that we do."

— Jensen Huang, CEO of NVIDIA



What is “Co-Design”?

Co-design: Inter- or multi-disciplinary work,
where changing the interface or contract between areas
allows much better solutions than work in a single area

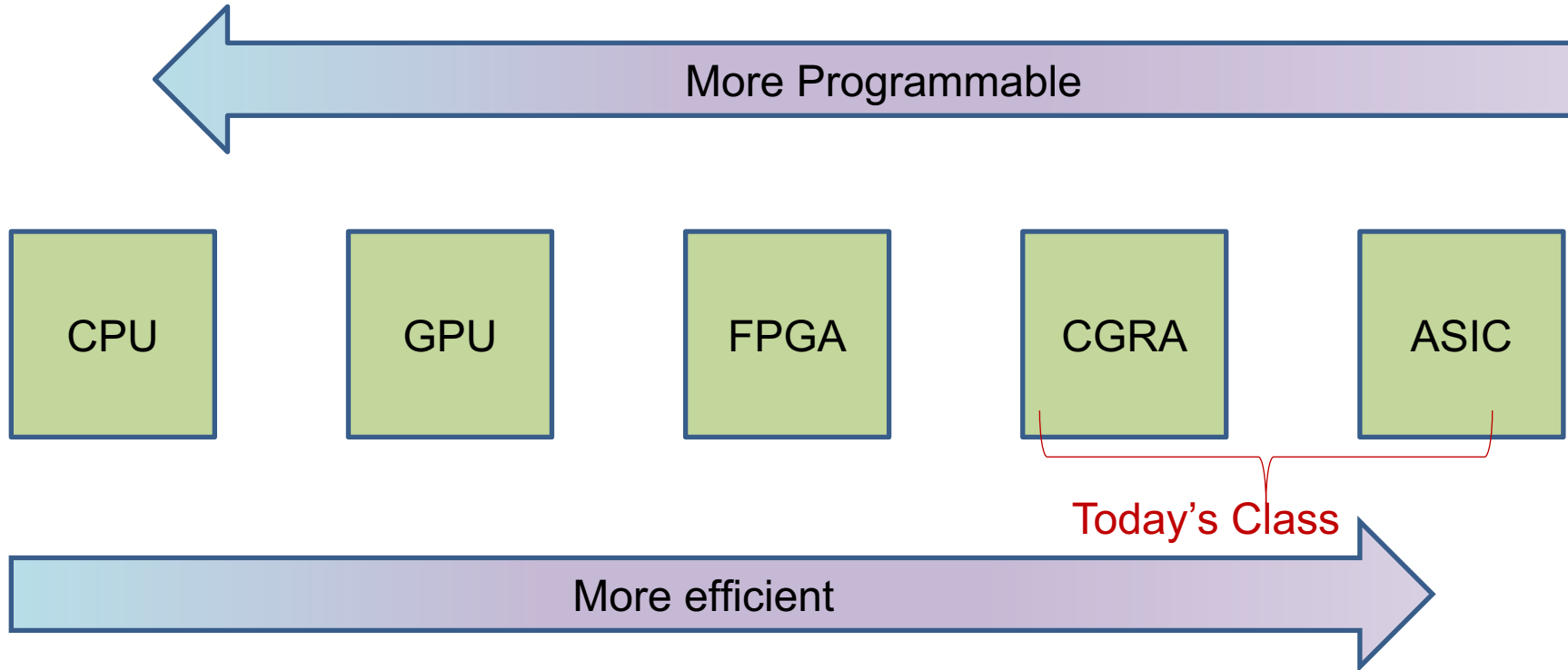


Co-design opens up more possibilities for positive-sum solutions

- Within one field, we often focus on trade-offs, which can be zero-sum
“Performance costs power”
- Working across multiple fields gives additional degrees of freedom
Exploit “don’t-cares” in one or more fields
“Accuracy and performance with low power”

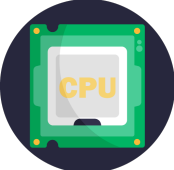
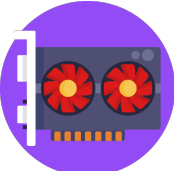
Compute Choices

FPGA => Field programmable gate array
CGRA => Coarse-grained reconfigurable array
ASIC => Application-specific integrated circuit

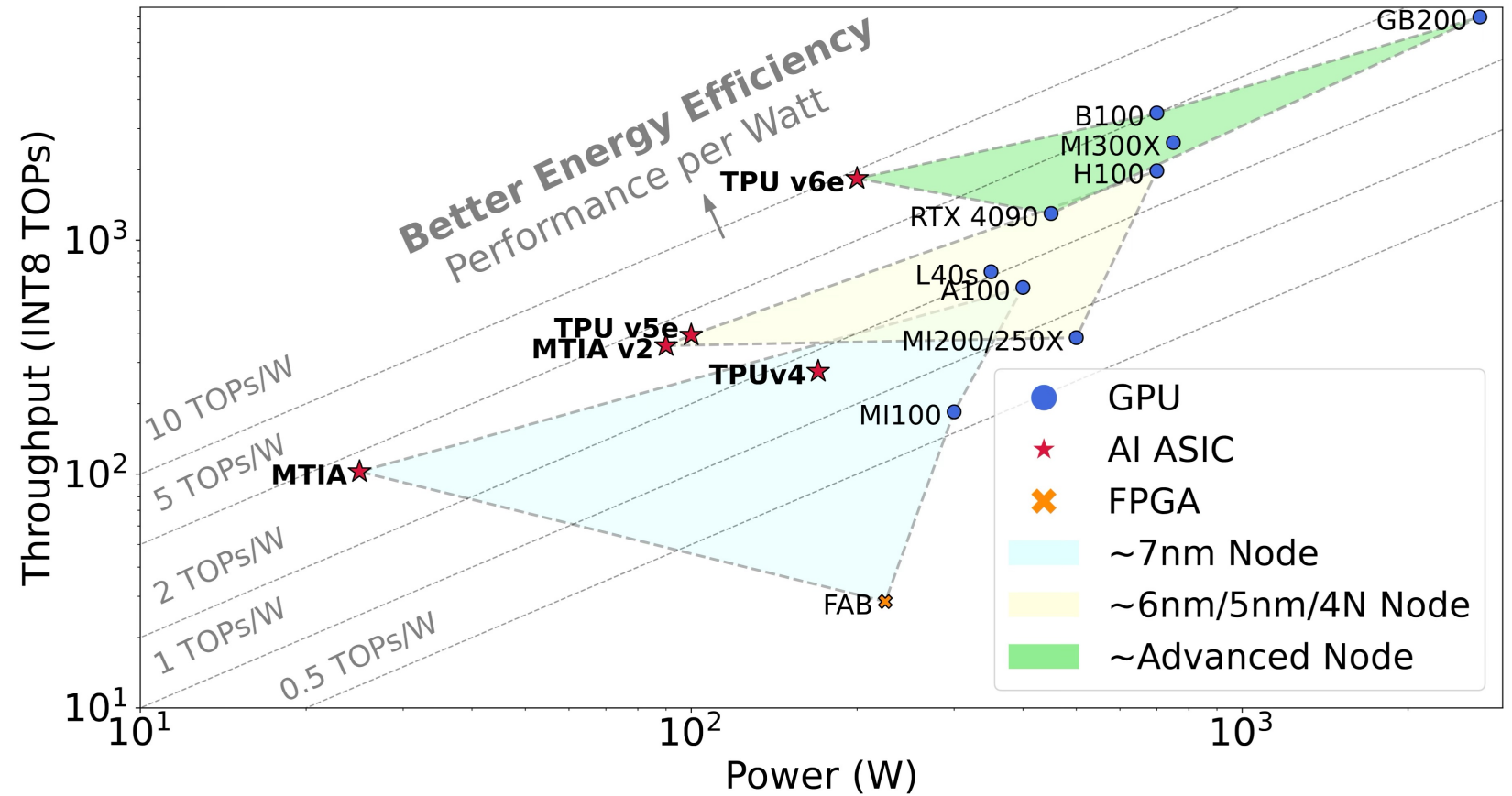


Note: we will use “AI/DNN Accelerator”, “AI ASIC”, “Tensor Processing Unit (TPU)”, “Neural Processing Unit (NPU)” interchangeably

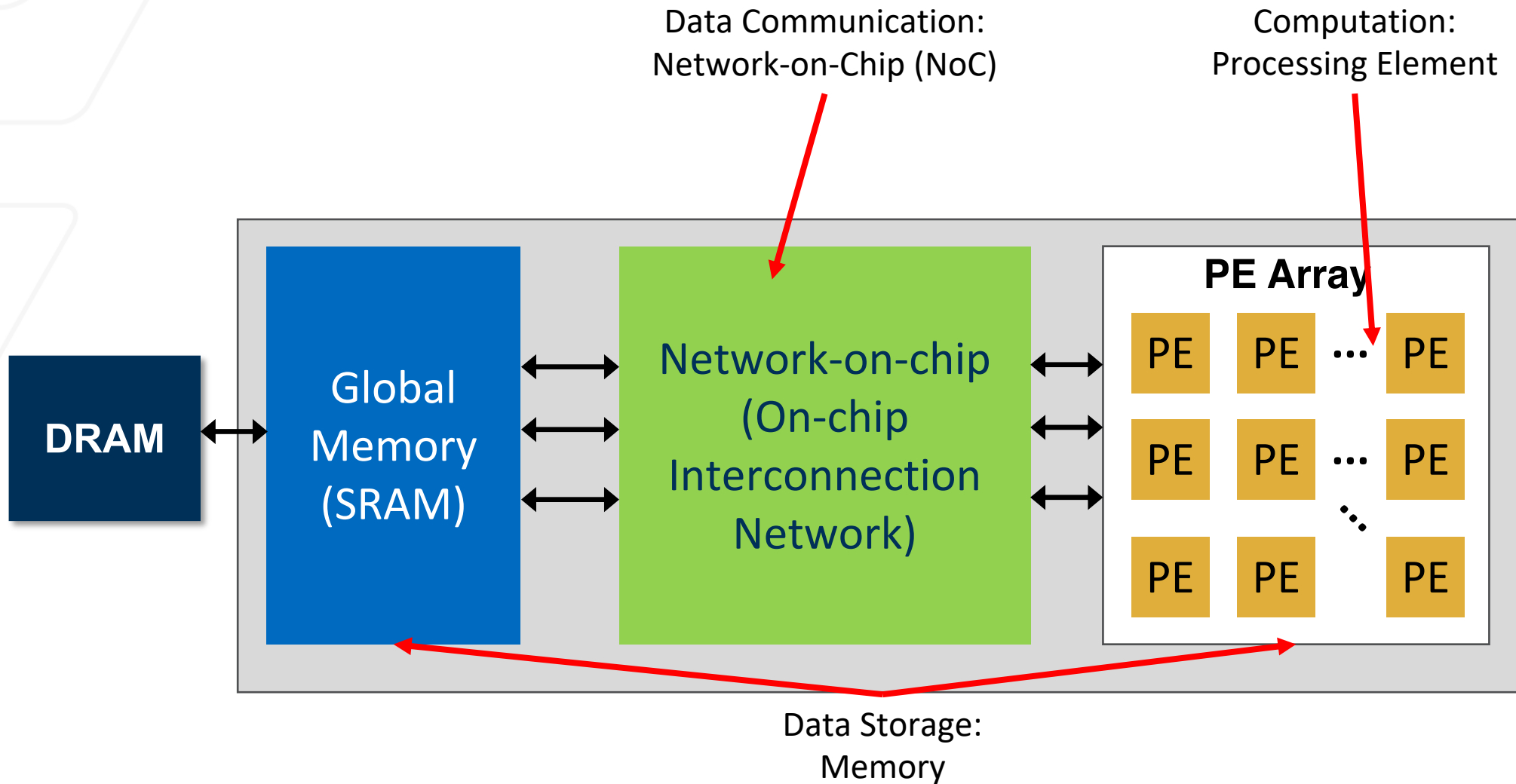
Comparison of HW choices

Challenges HW Options		
	Latency	Energy
 CPU		
 GPU		
Accelerator/ TPU/NPU		

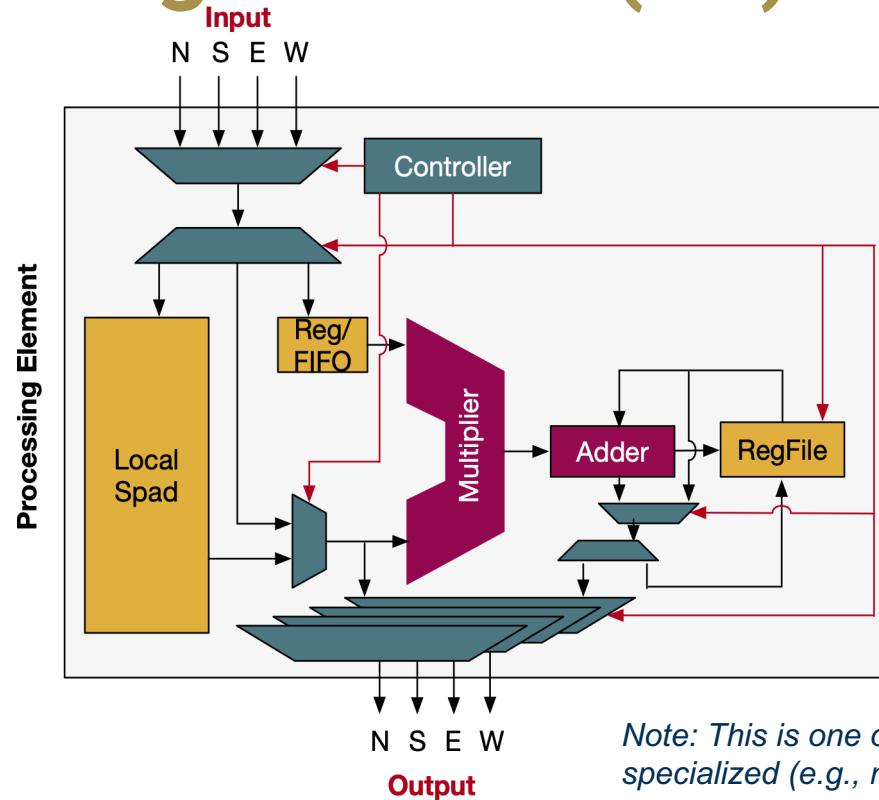
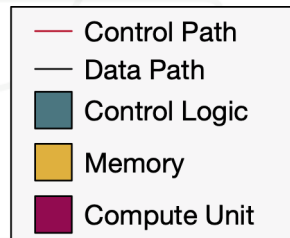
AI Accelerators show better Throughput/Watt



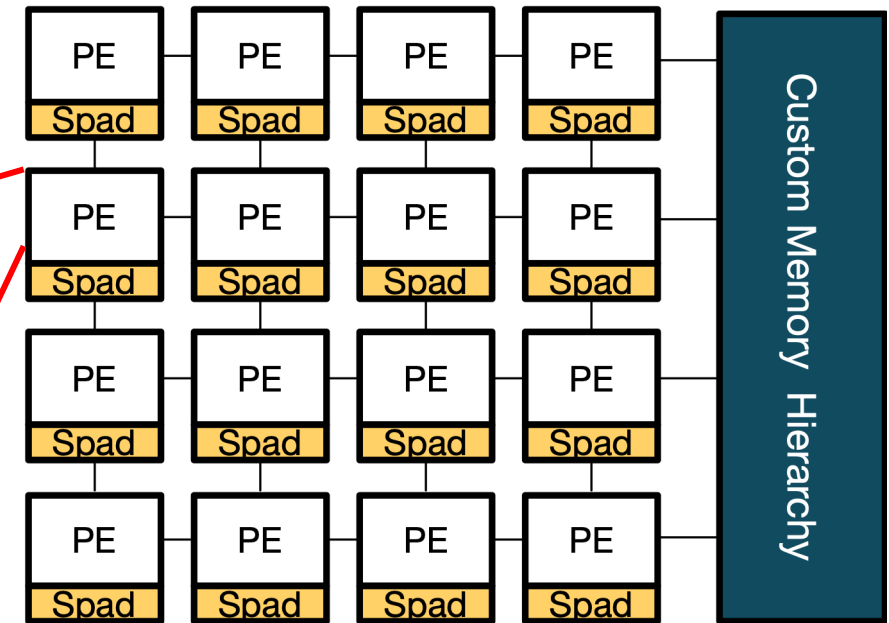
Structure of an AI Accelerator



Processing Element (PE)



Note: This is one of the possible designs. PEs are often further specialized (e.g., remove output ports to N and W)



Features

- Local memories for data reuse
- Can forward an output or operand (input or weight) to another PE
- Only contains an INT8 multiplier and an INT8 adder
- Minimal control logic; usually not instruction-controlled (control register-controlled)

Reduce data movement

Minimal HW

Difference from CPU Architecture

- Registers distributed across PEs
 - One per operand per PE
 - Can be more than one as well
- Operands “forwarded” from one register to the other
 - More energy-efficient than reading large register file
- Stage data through array in deterministic manner
 - No need for hazard checks etc
- Use programmable scratchpads (not caches)
 - Data reuse pattern completely deterministic (after “mapping”)

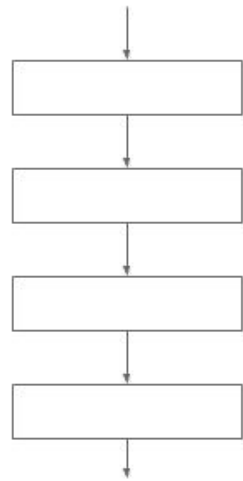
Outline

- Systolic Array
- Dataflow
- Flexible AI Accelerators
- Takeaways

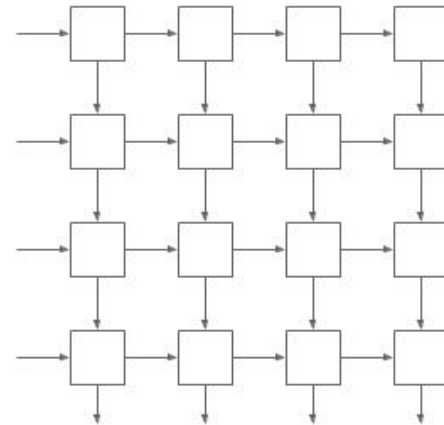
Outline

- **Systolic Array**
- Dataflow
- Flexible AI Accelerators
- Takeaways

Systolic Arrays Intuition: 2D Pipelines



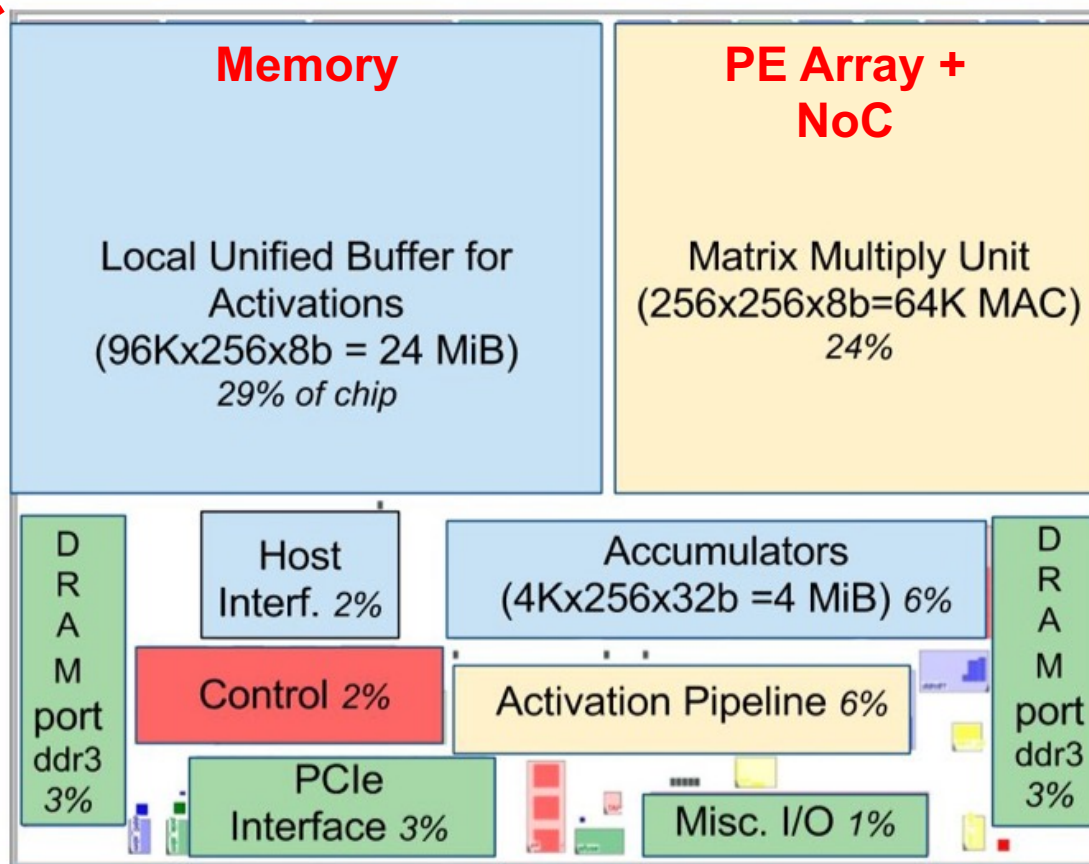
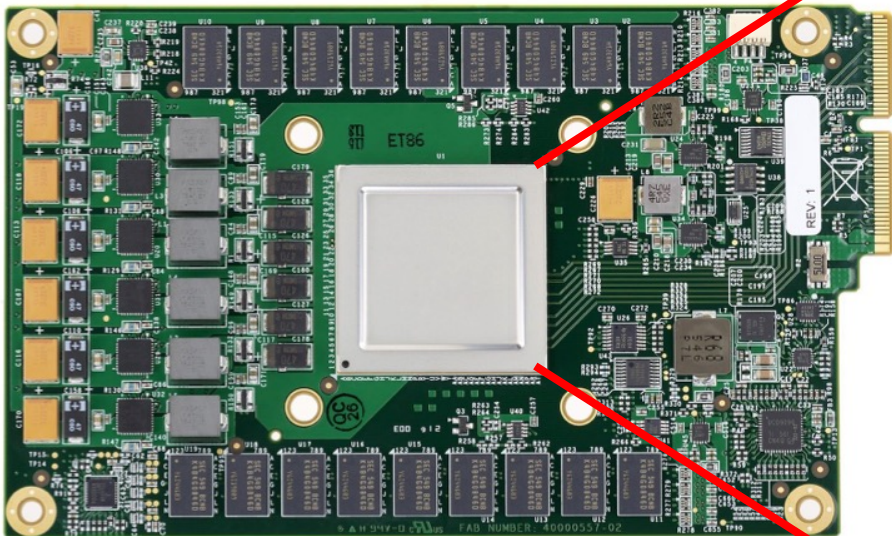
1D Pipeline



2D Pipeline

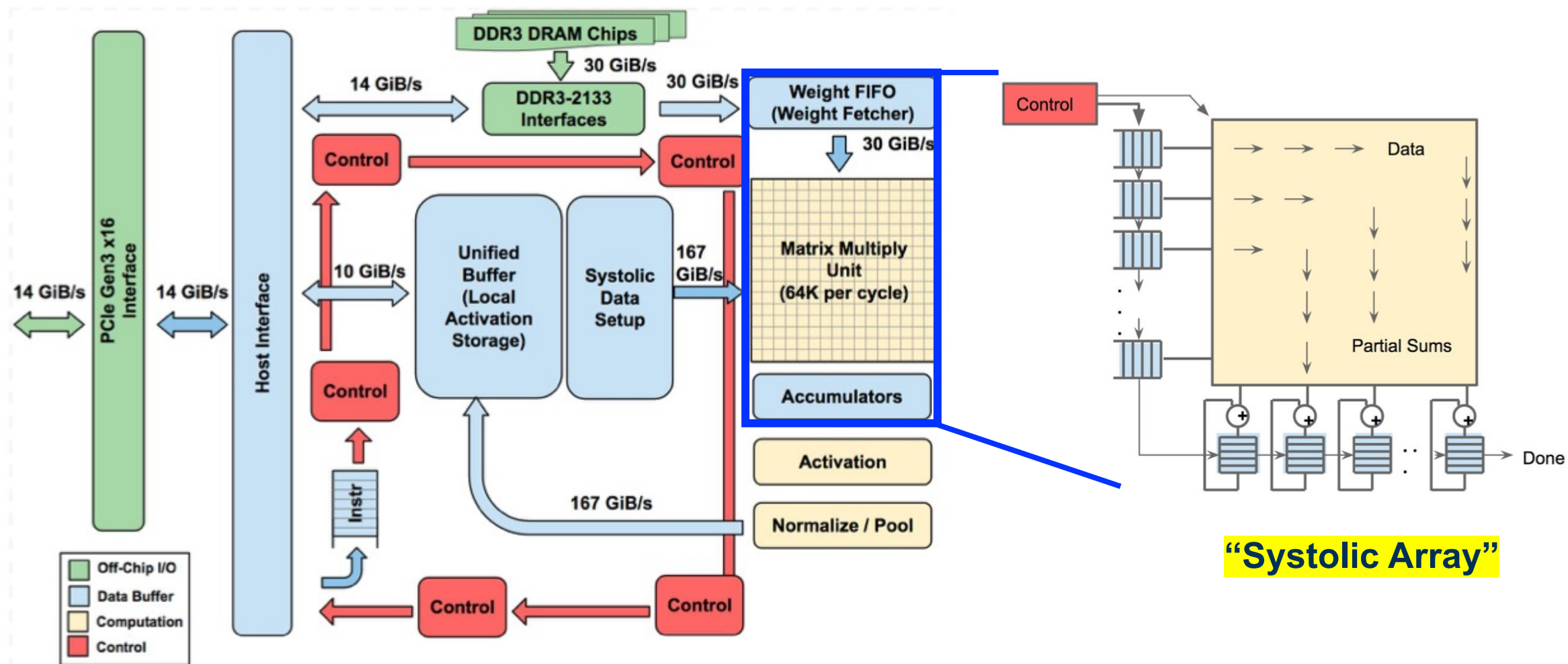
Google TPUv1 (2016)

Chip Layout:



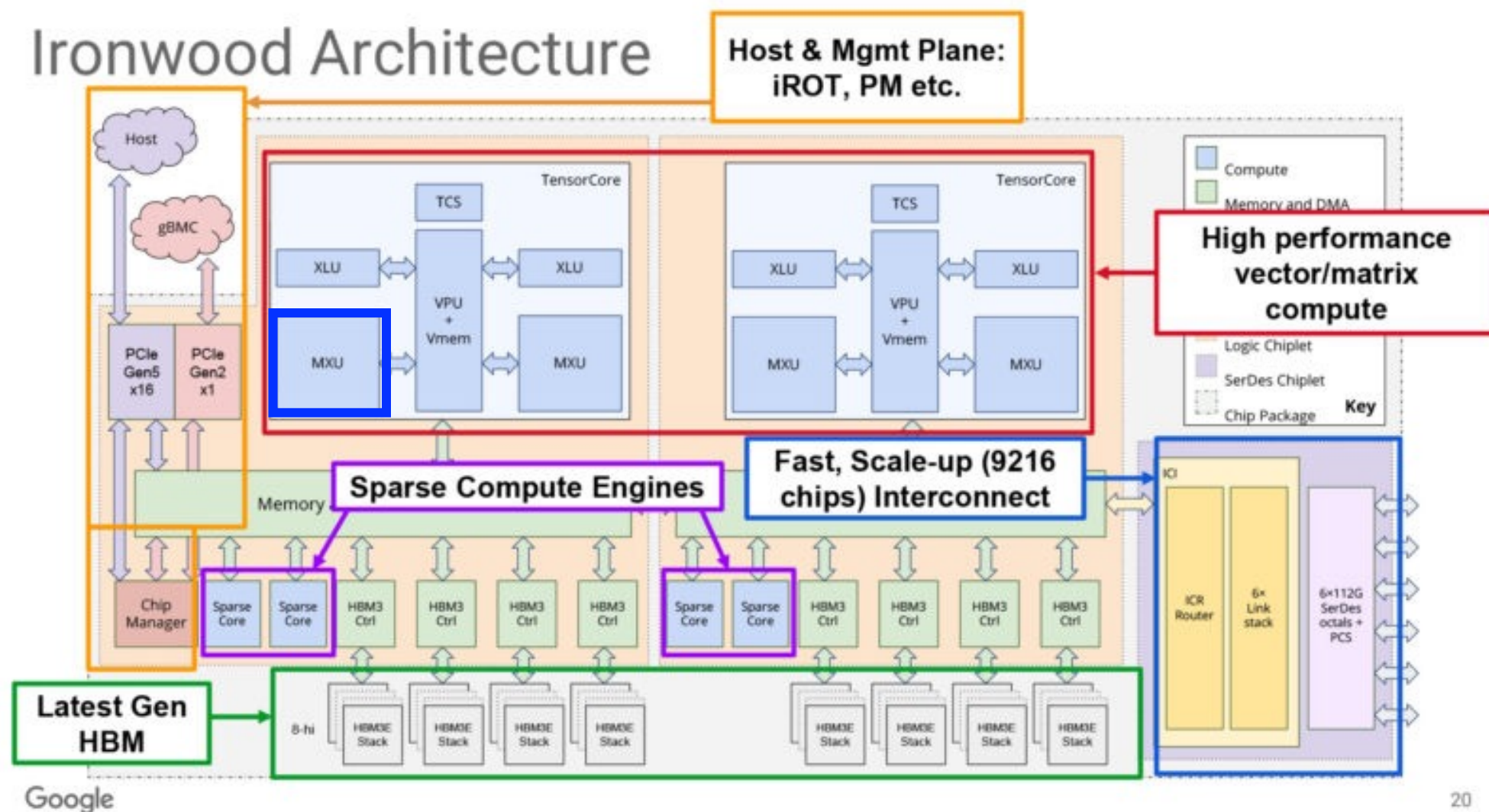
- N. P. Jouppi et al., "In-Datcenter Performance Analysis of a Tensor Processing Unit" ISCA, 2017.

Google TPUv1 (2016)



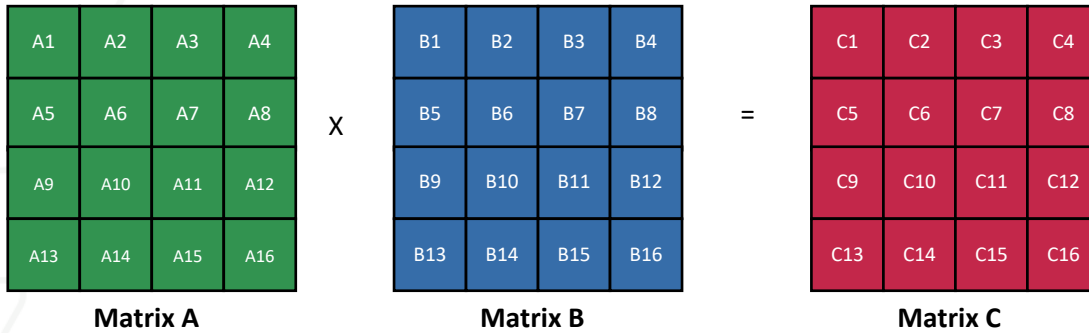
- N. P. Jouppi et al., “In-Datcenter Performance Analysis of a Tensor Processing Unit,” ISCA, 2017.

Google TPUv7 aka *Ironwood* (2025)



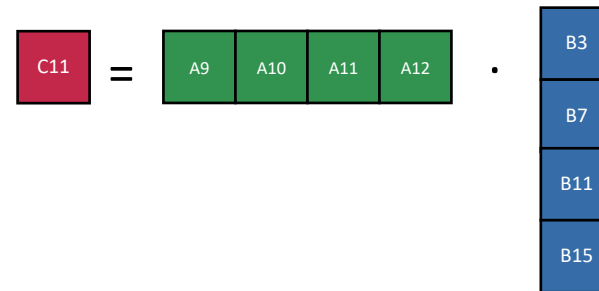
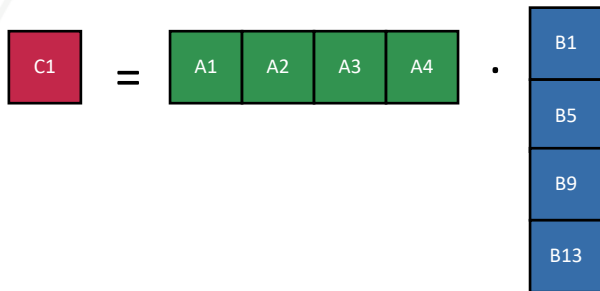
20

What to Accelerate?



General Matrix Multiplication or GEMM

Example Computation operations

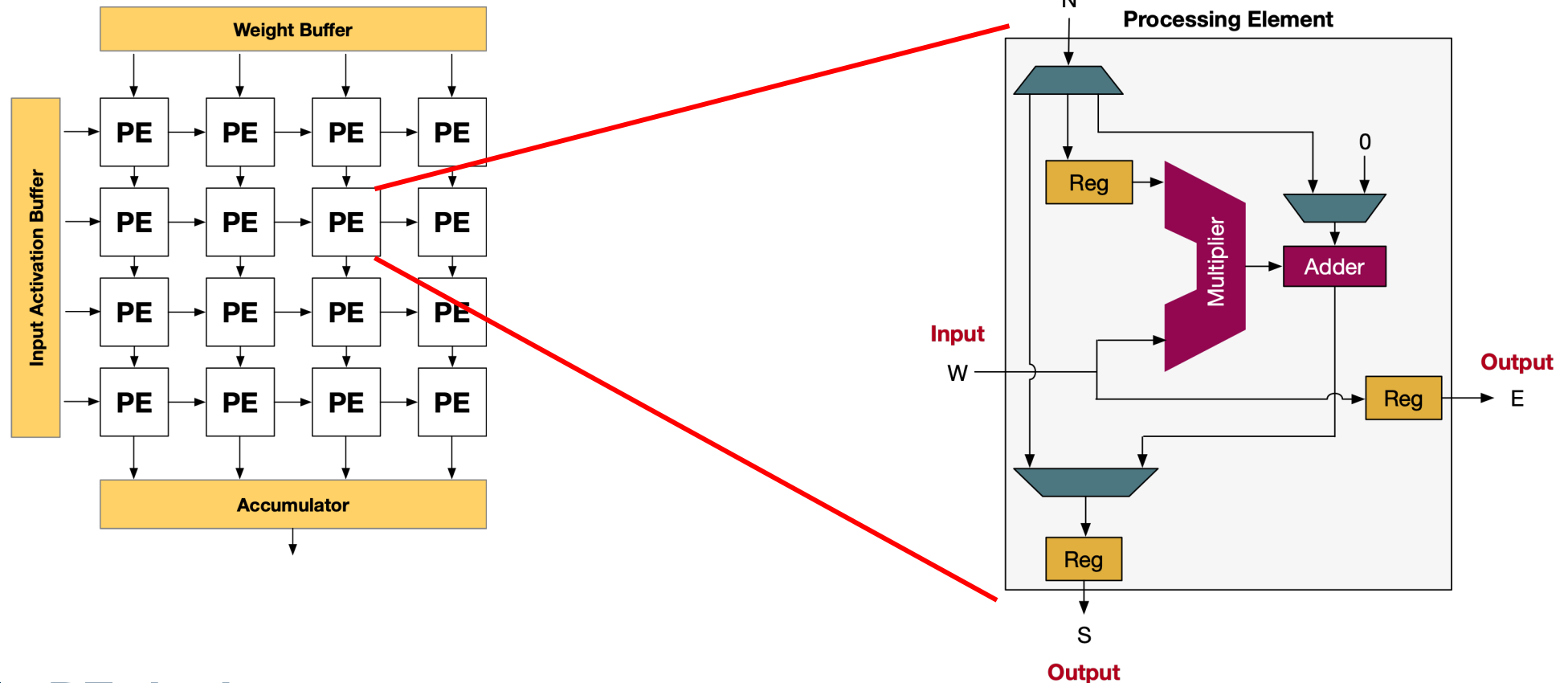


$$C1 = A1 \cdot B1 + A2 \cdot B5 + A3 \cdot B9 + A4 \cdot B13$$

$$C11 = A9 \cdot B3 + A10 \cdot B7 + A11 \cdot B11 + A12 \cdot B15$$

But .. Beware of Amdahl's law

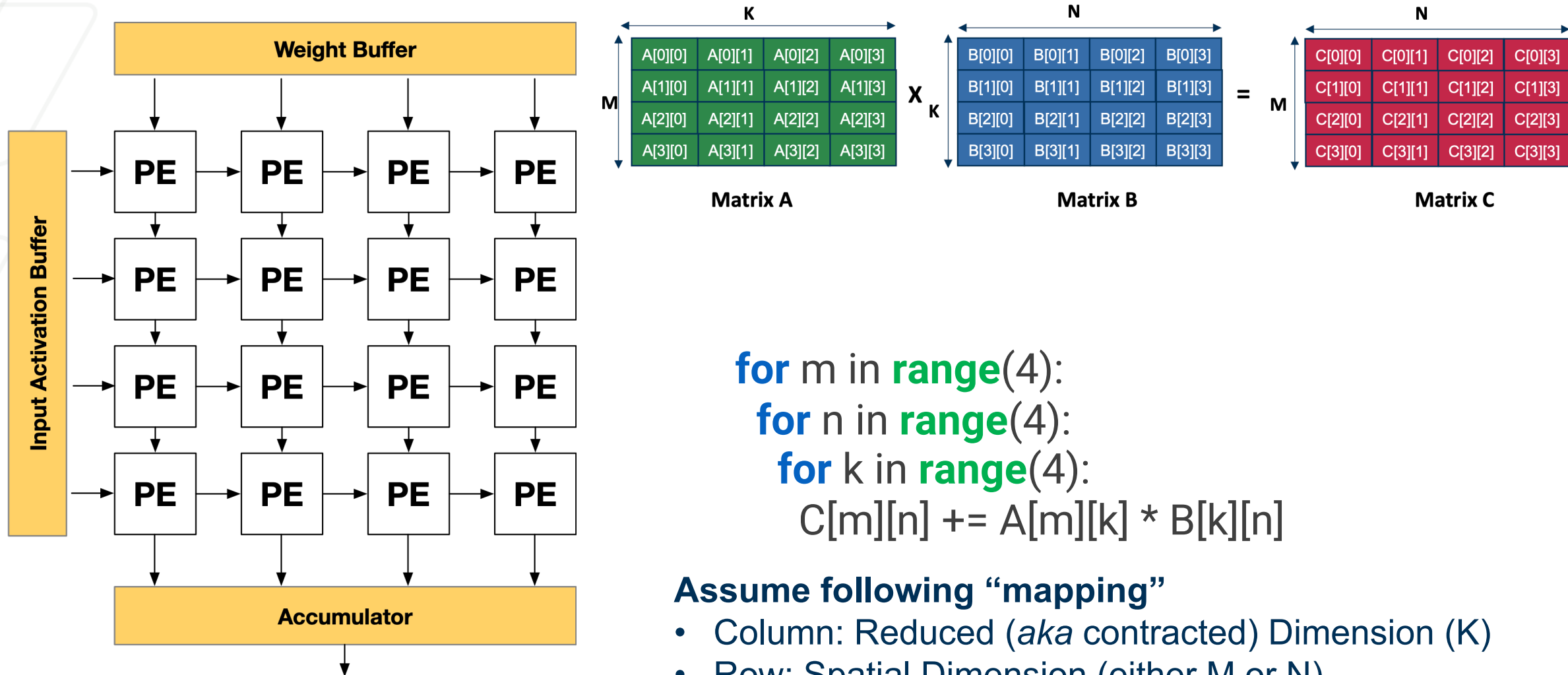
PEs in Systolic Arrays (TPUv1 style)



■ Simple PE design

- No local scratchpad; several one-slot registers
- Enables to increase the number of PEs (small area for each PE -> more PEs within the same area)

Matrix Multiplication on a Systolic Array



```

for m in range(4):
    for n in range(4):
        for k in range(4):
            C[m][n] += A[m][k] * B[k][n]
  
```

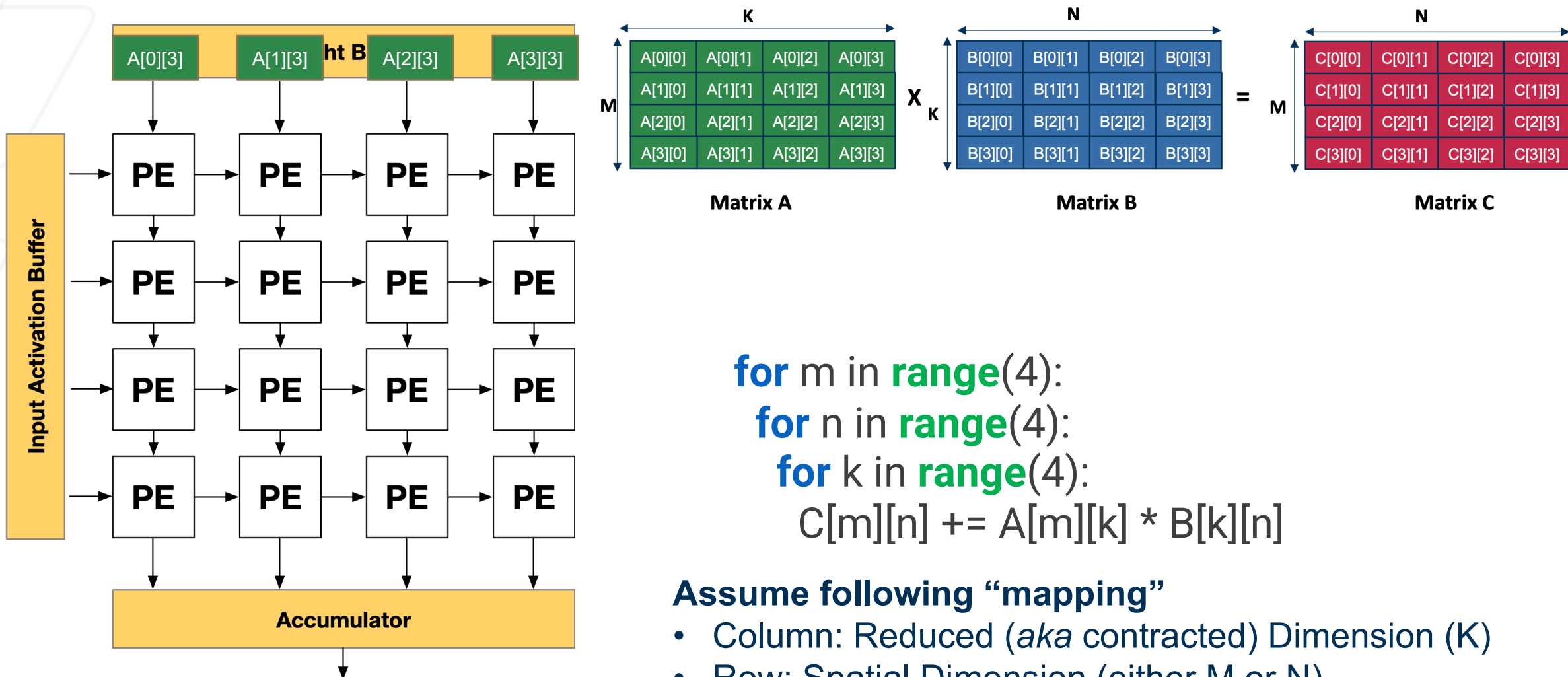
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array

Phase 1:
Load Weight Tensor



```
for m in range(4):
    for n in range(4):
        for k in range(4):
            C[m][n] += A[m][k] * B[k][n]
```

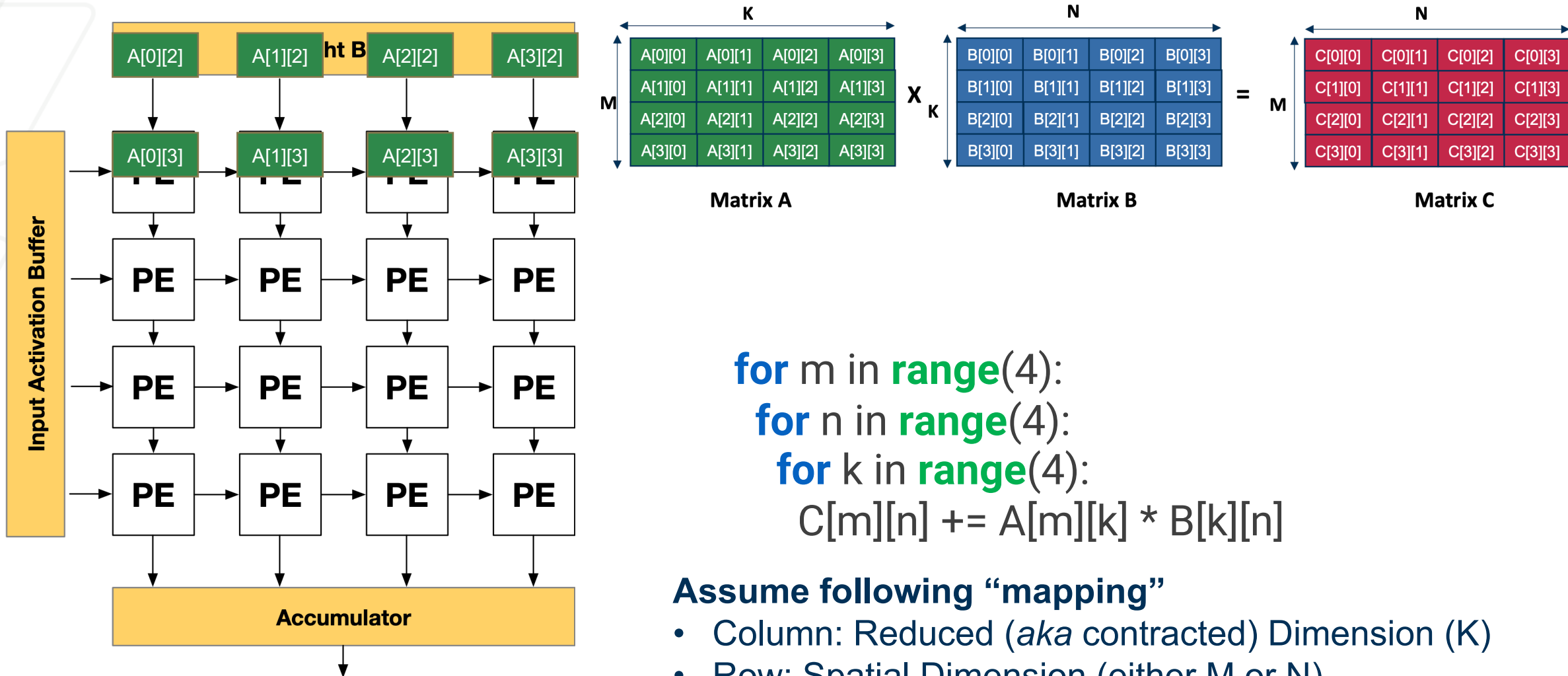
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array

Phase 1:
Load Weight Tensor



```
for m in range(4):
    for n in range(4):
        for k in range(4):
            C[m][n] += A[m][k] * B[k][n]
```

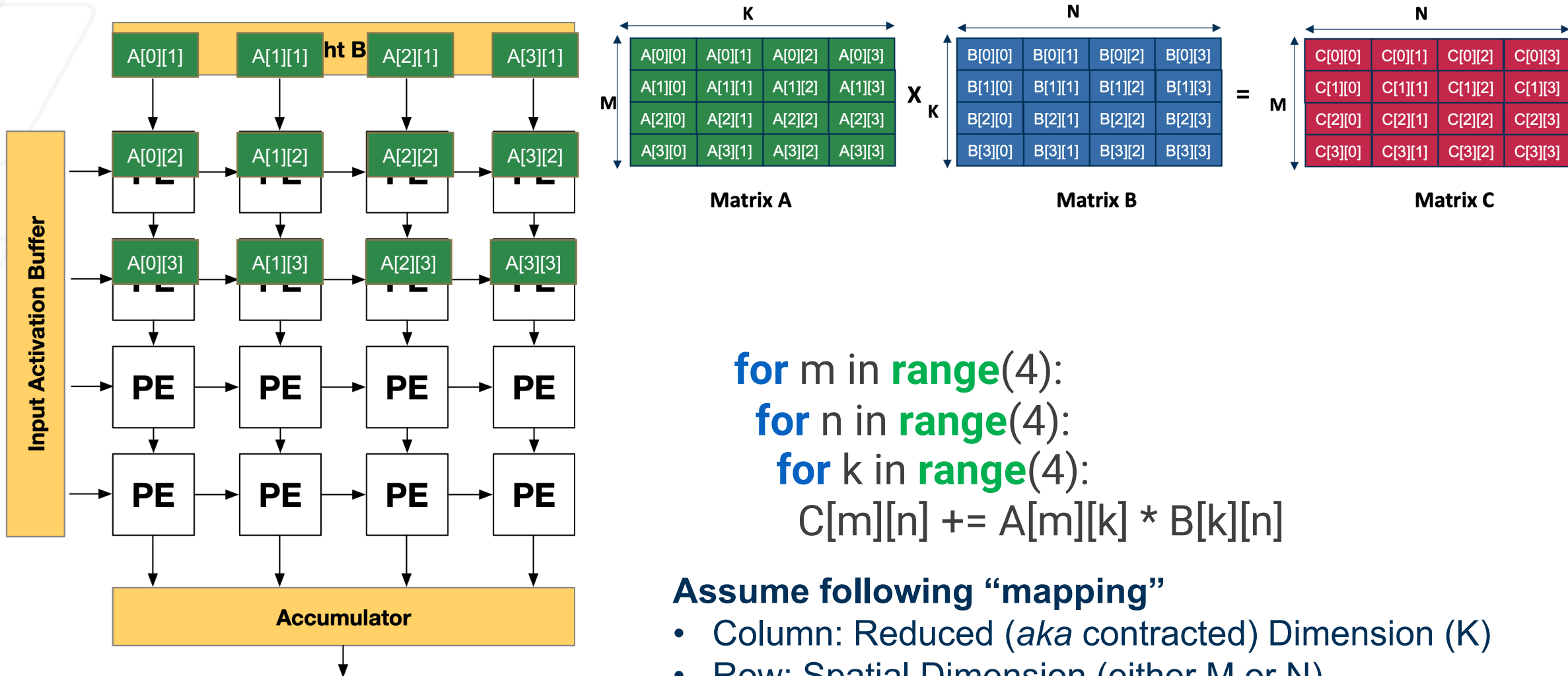
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array

Phase 1:
Load Weight Tensor



```
for m in range(4):
    for n in range(4):
        for k in range(4):
            C[m][n] += A[m][k] * B[k][n]
```

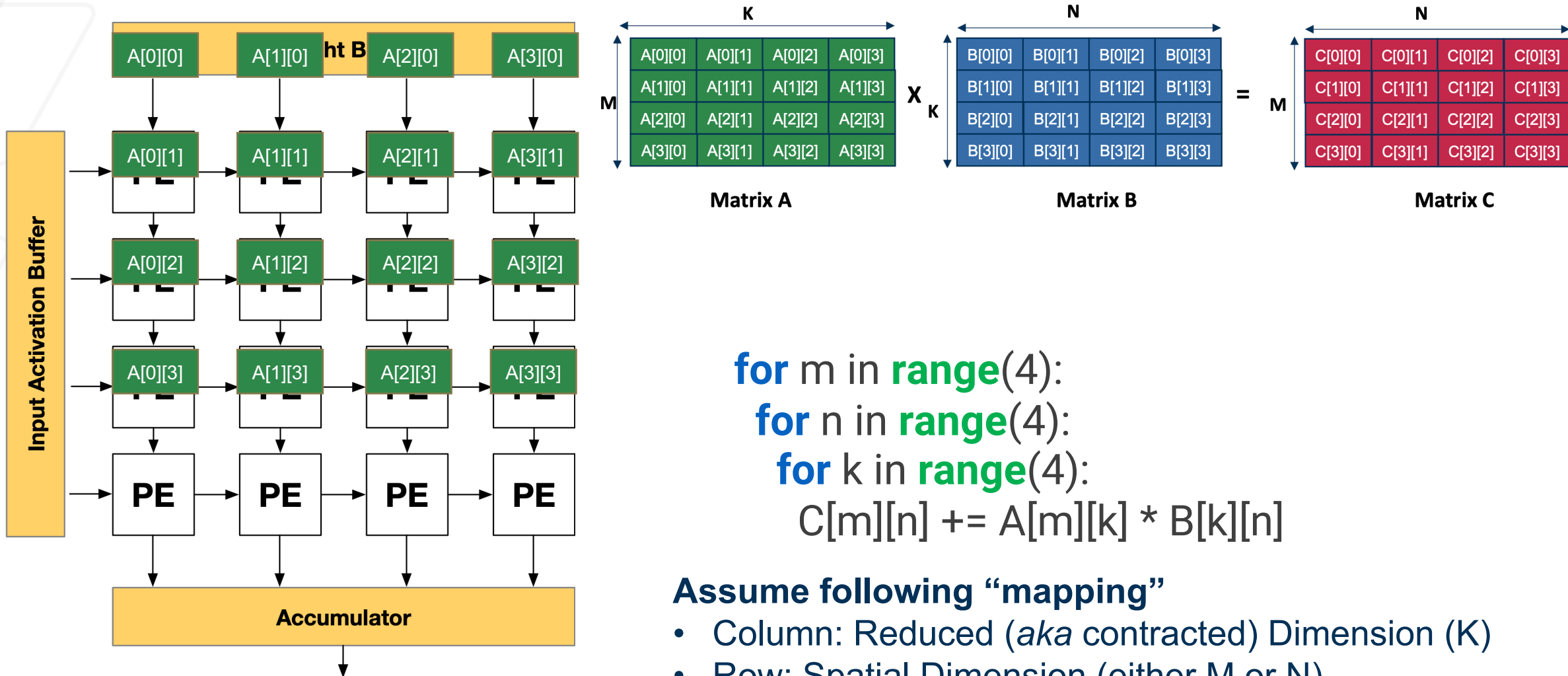
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

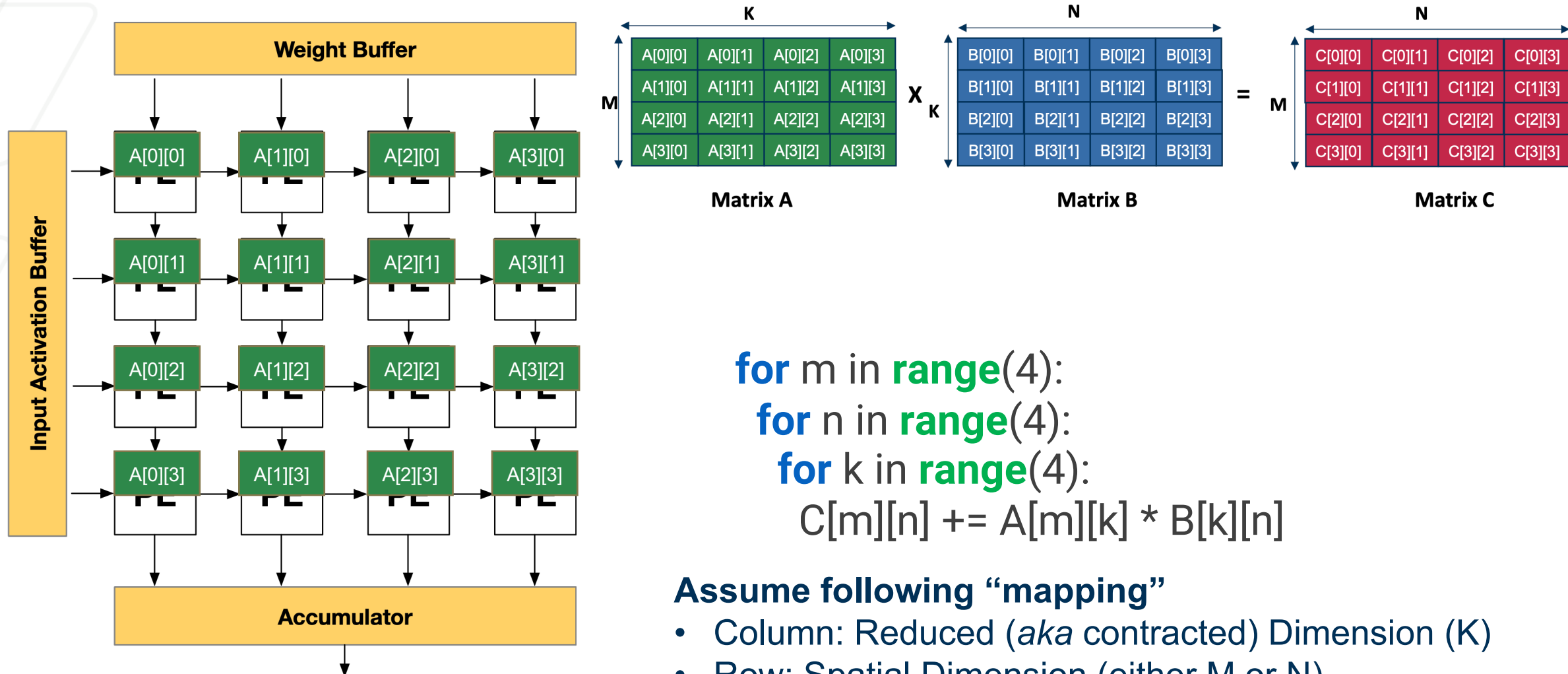
Matrix Multiplication on a Systolic Array

Phase 1:
Load Weight Tensor



Matrix Multiplication on a Systolic Array

Phase 1:
Load Weight Tensor



```
for m in range(4):
    for n in range(4):
        for k in range(4):
            C[m][n] += A[m][k] * B[k][n]
```

Assume following “mapping”

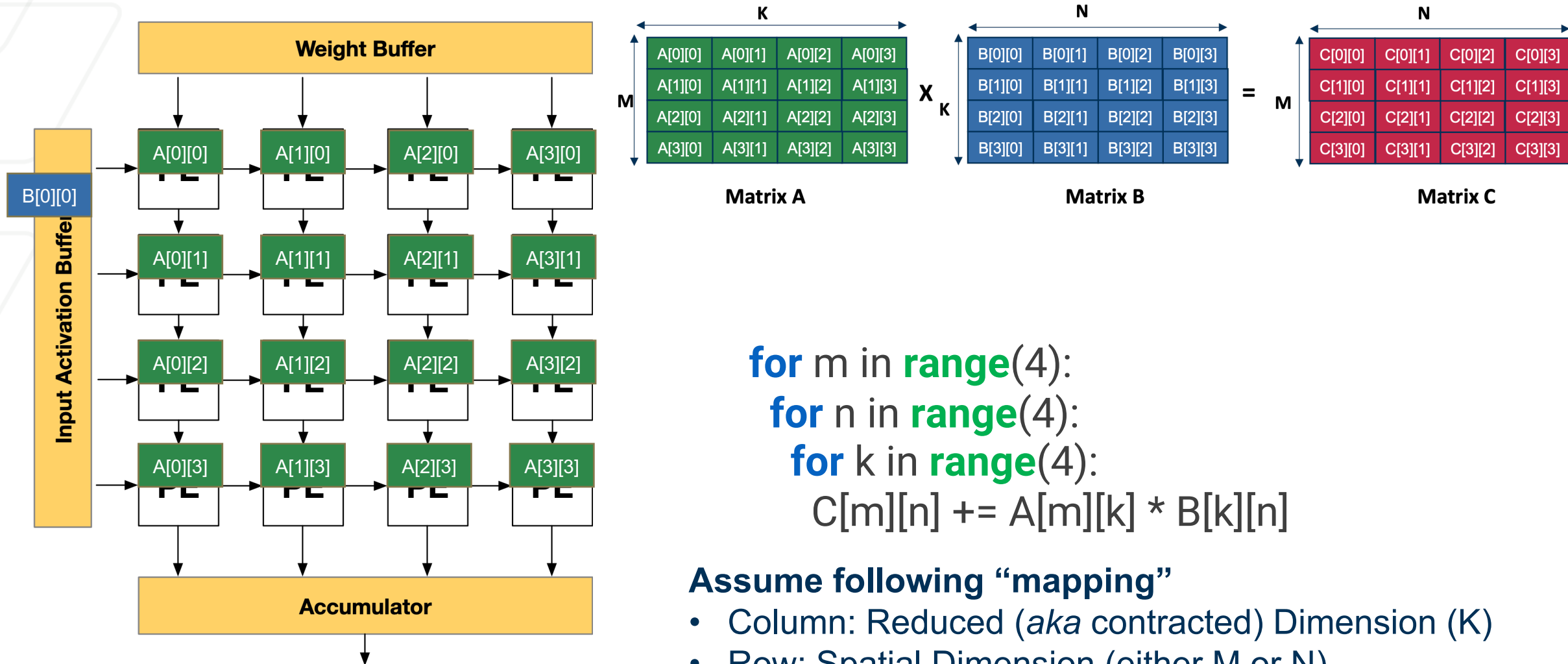
- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix A will now stay resident (i.e., “stationary”) in the PEs

Matrix Multiplication on a Systolic Array

Phase 2:
Stream Activation Tensor



```
for m in range(4):
    for n in range(4):
        for k in range(4):
            C[m][n] += A[m][k] * B[k][n]
```

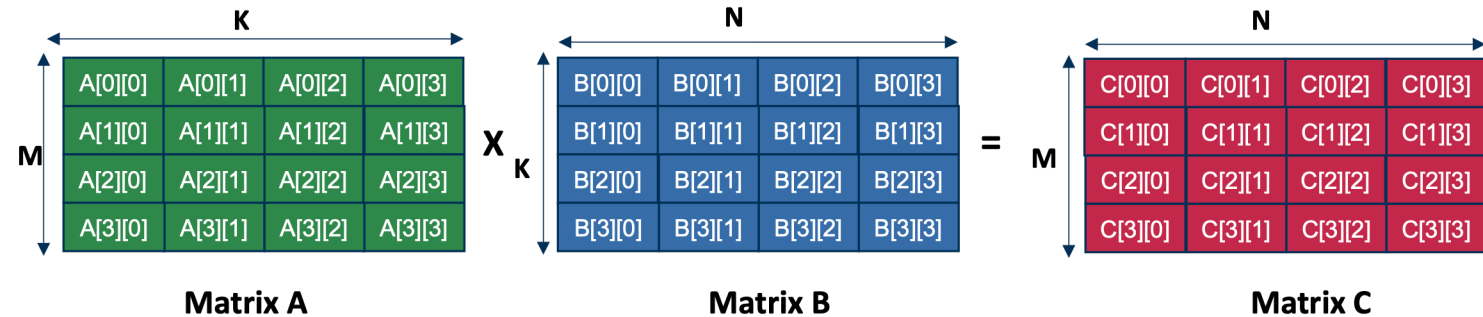
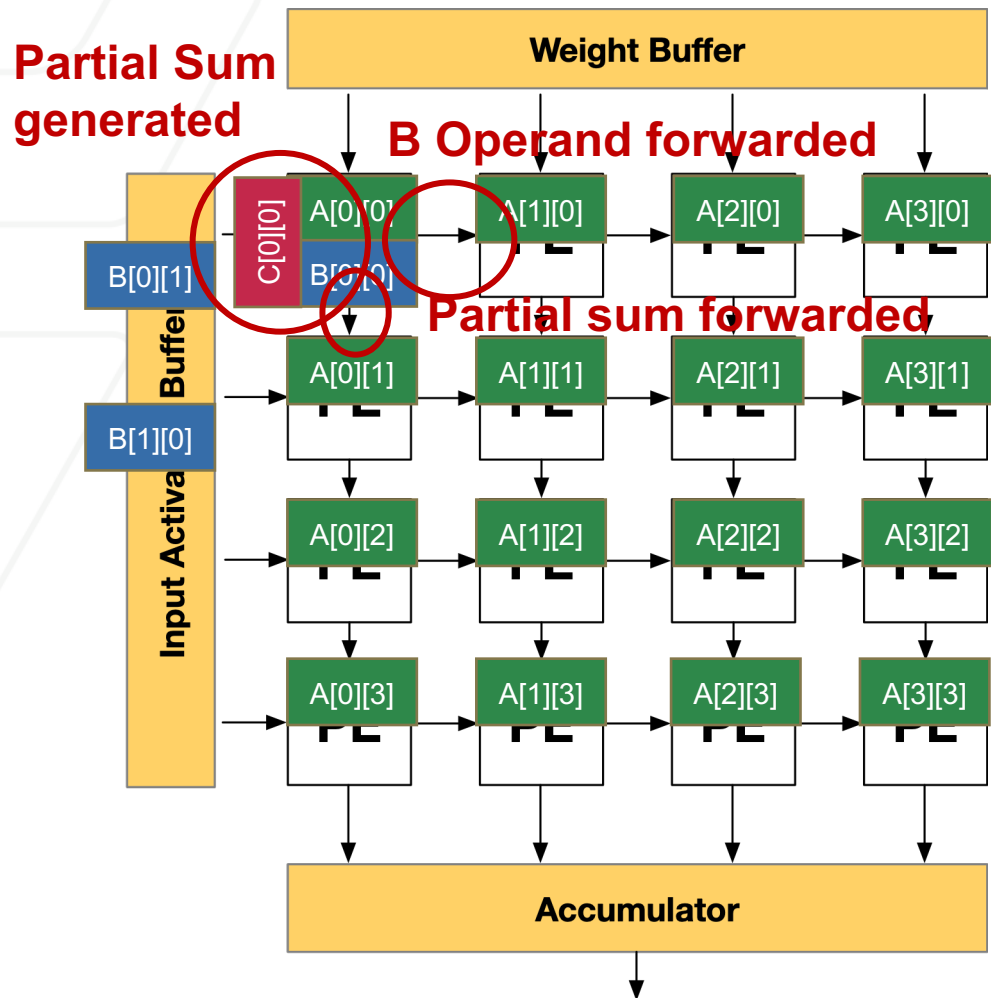
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array

Phase 2:
Stream Activation Tensor



```

for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
  
```

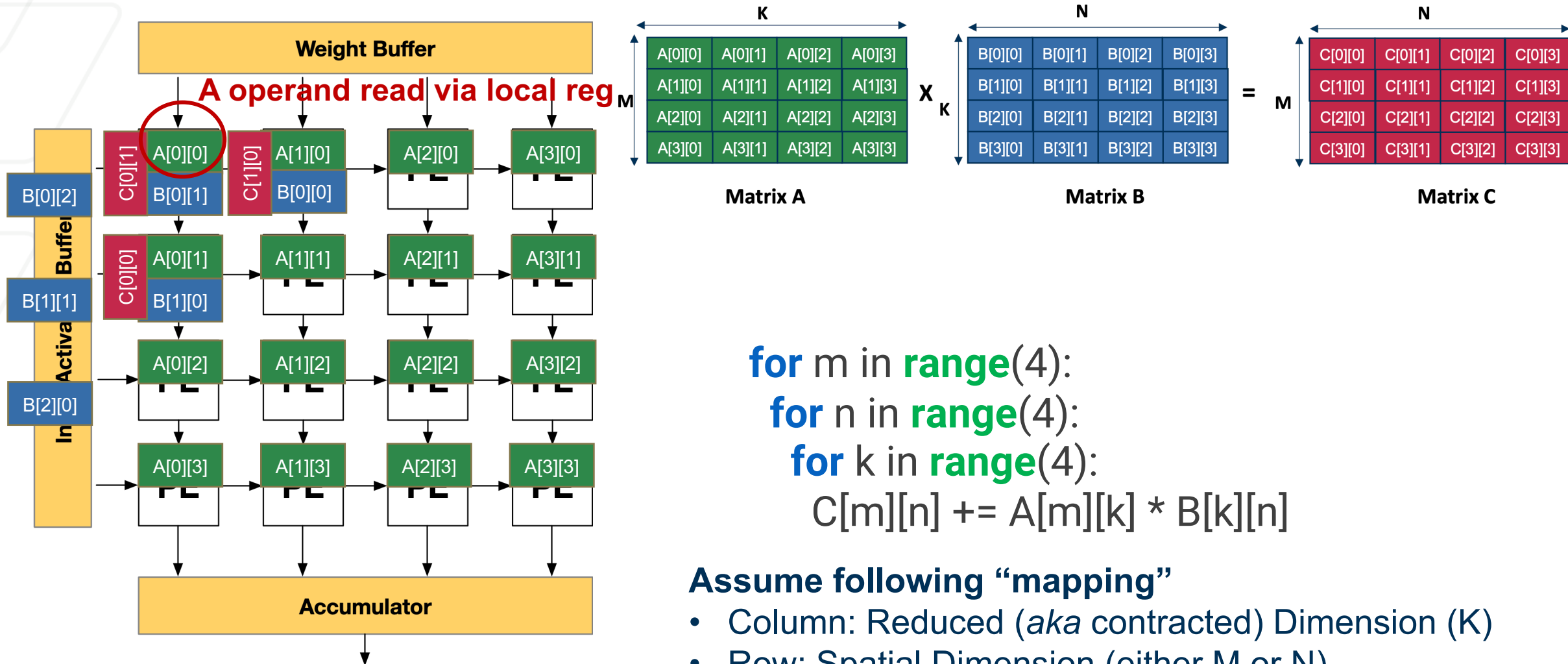
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array

Phase 2:
Stream Activation Tensor



```
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

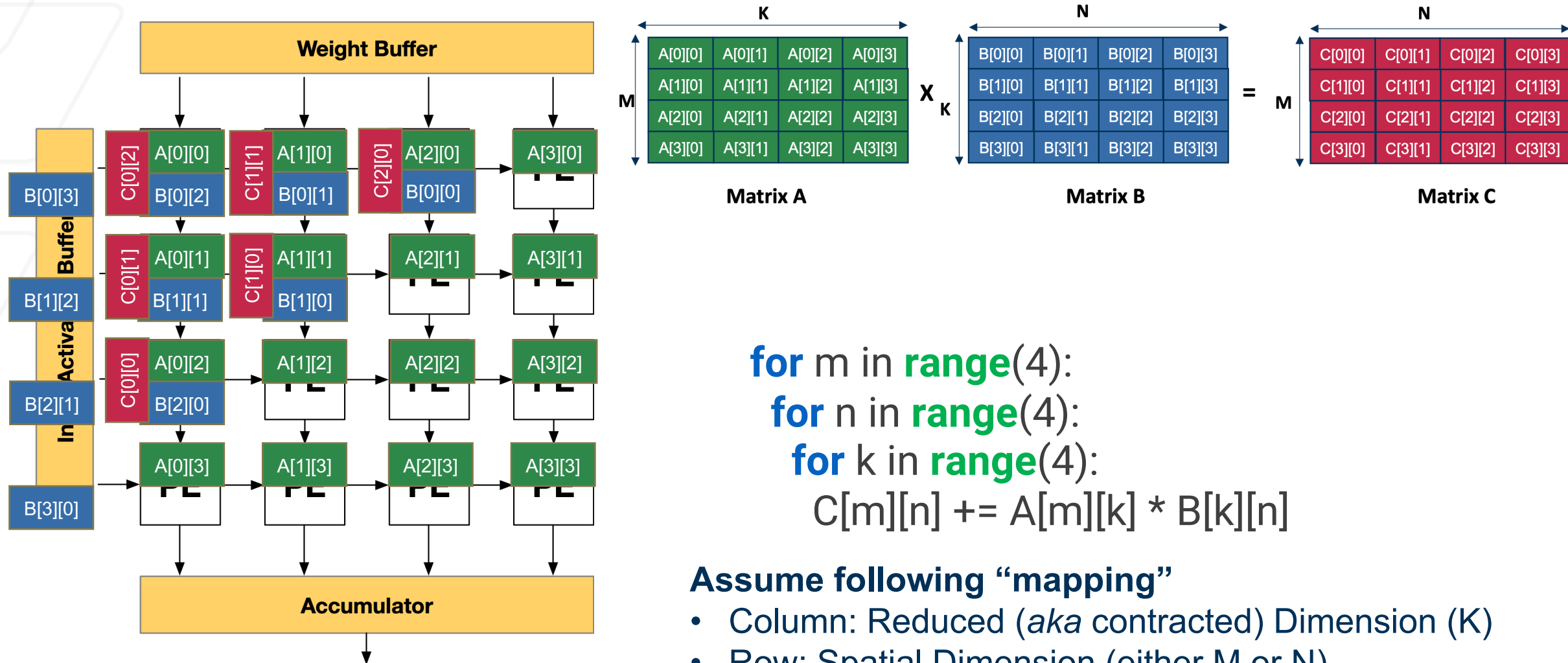
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array

Phase 2:
Stream Activation Tensor



```
for m in range(4):
    for n in range(4):
        for k in range(4):
            C[m][n] += A[m][k] * B[k][n]
```

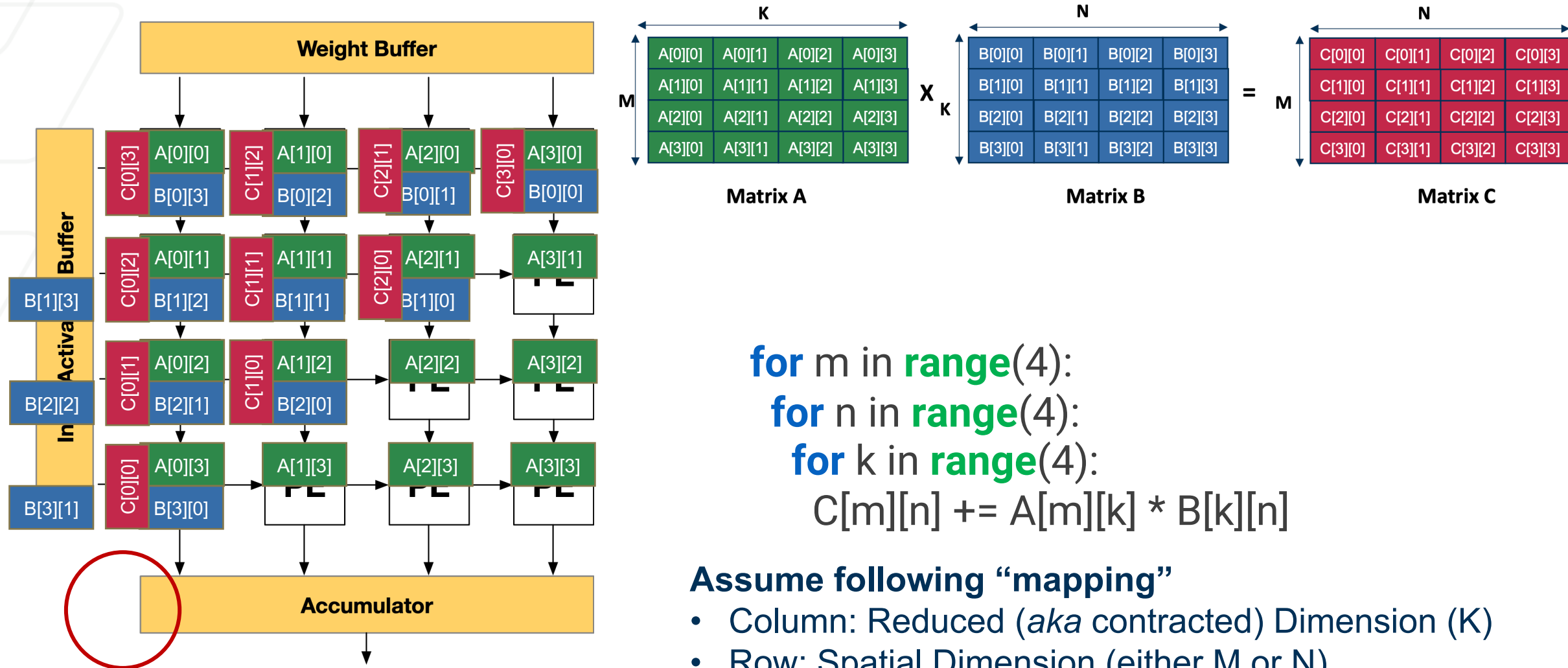
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array

Phase 2:
Stream Activation Tensor



```
for m in range(4):
    for n in range(4):
        for k in range(4):
            C[m][n] += A[m][k] * B[k][n]
```

Assume following “mapping”

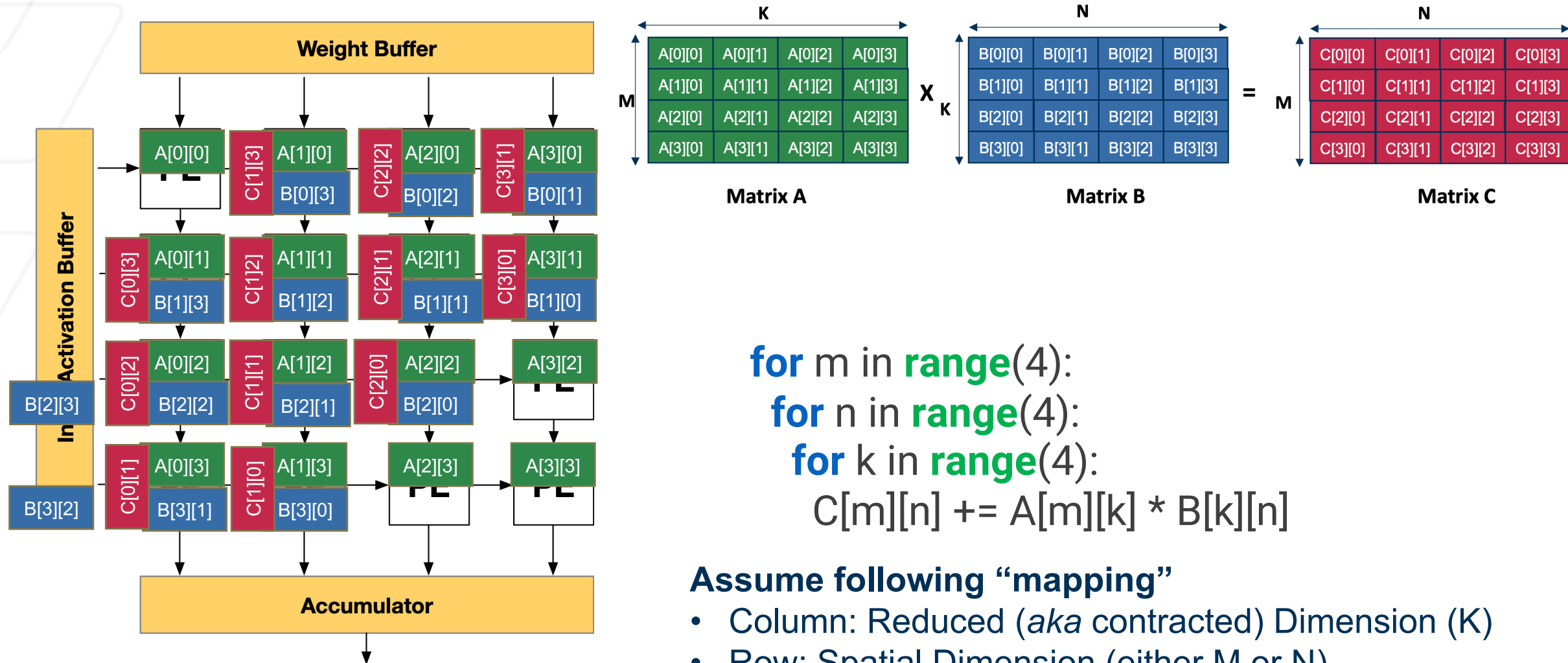
- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

First output generated

Matrix Multiplication on a Systolic Array

Phase 2:
Stream Activation Tensor



```
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

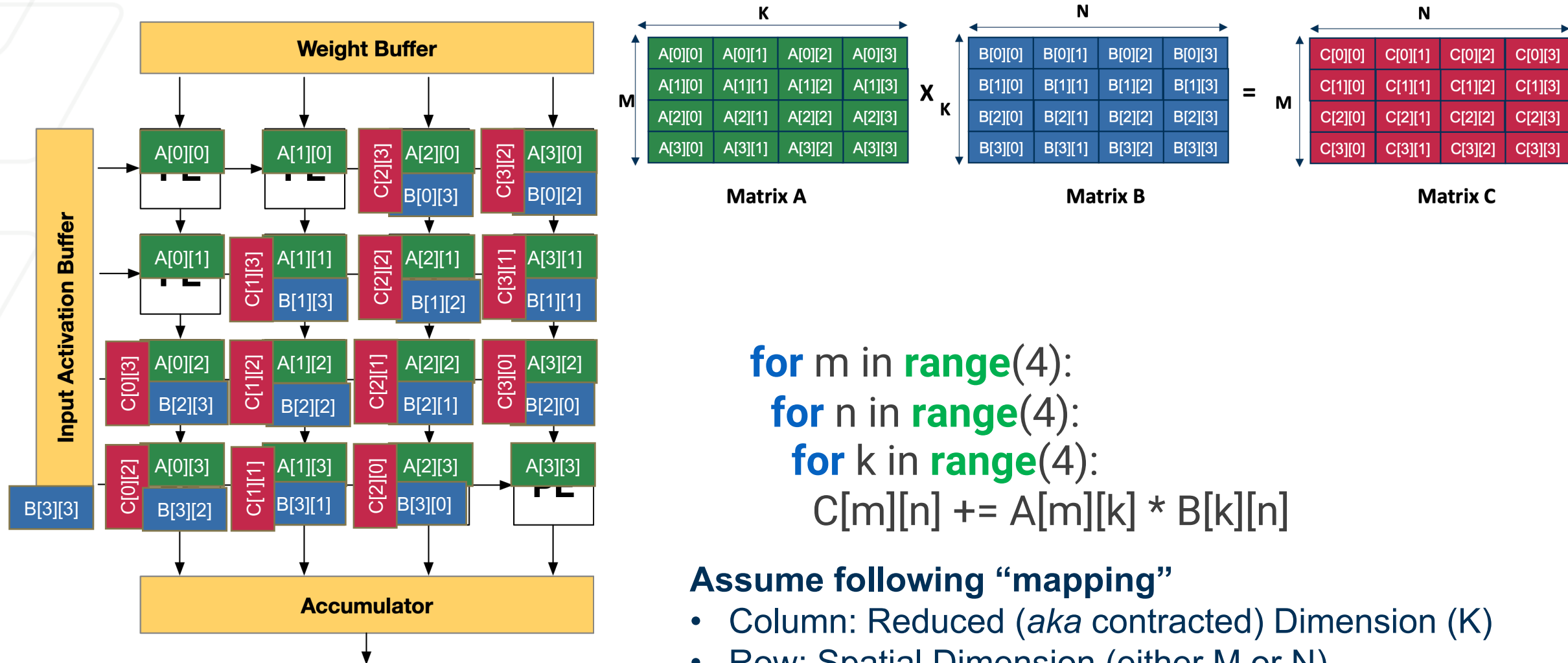
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array

Phase 2:
Stream Activation Tensor



```
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

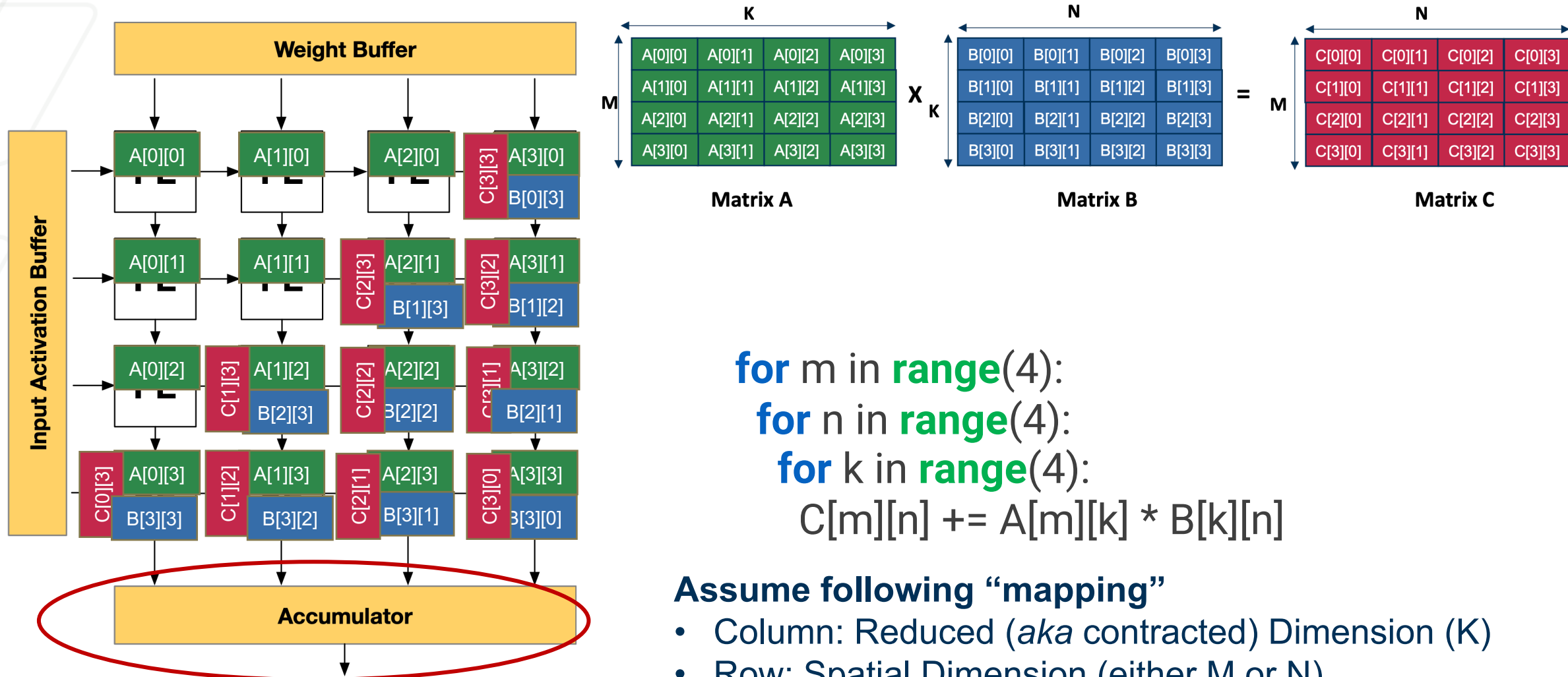
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array

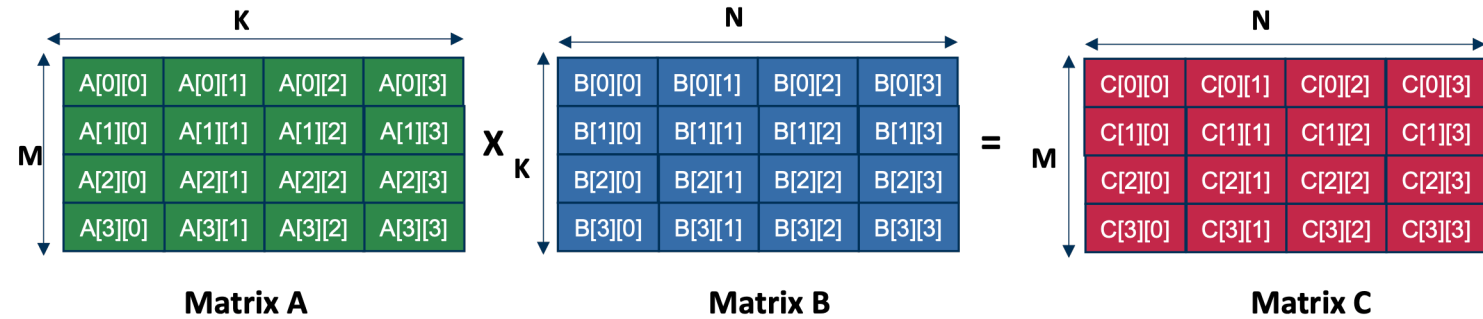
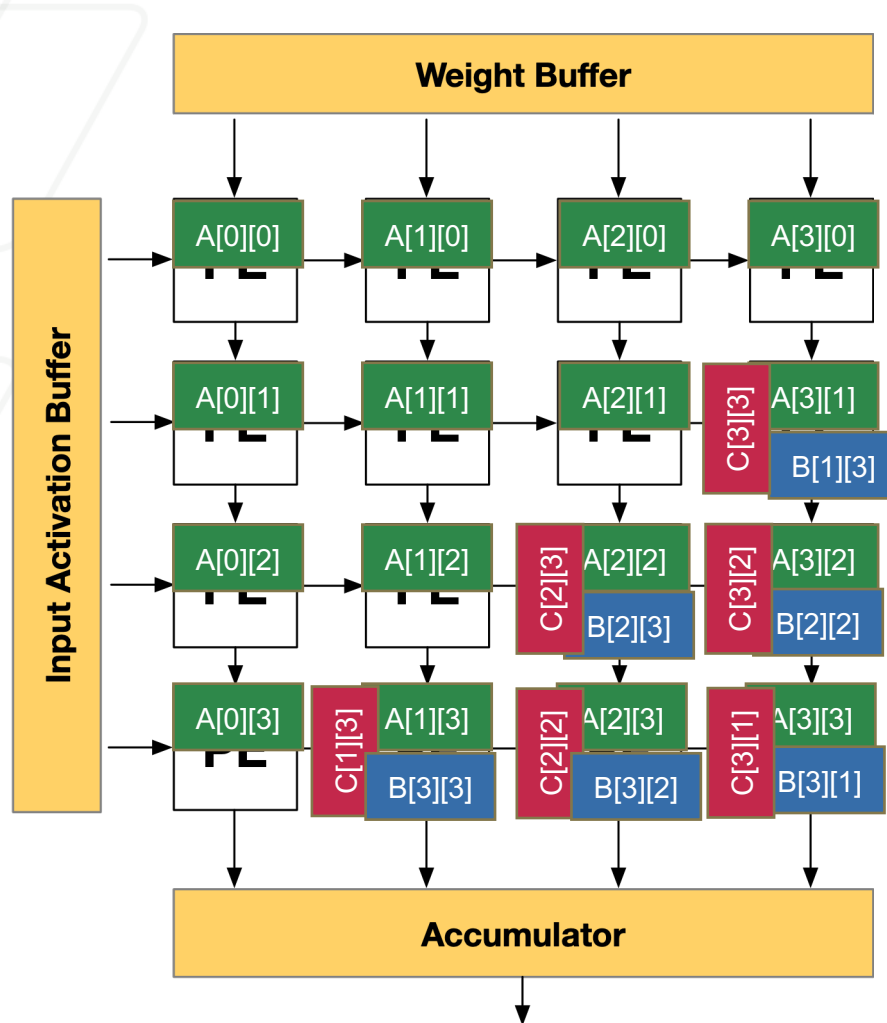
Phase 2:
Stream Activation Tensor



Four outputs every cycle in steady state

Matrix Multiplication on a Systolic Array

Phase 2:
Stream Activation Tensor



```
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

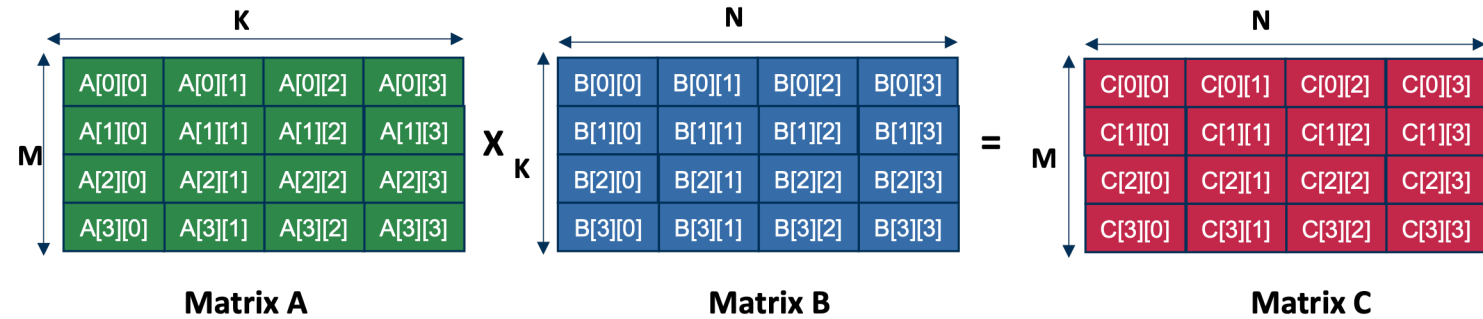
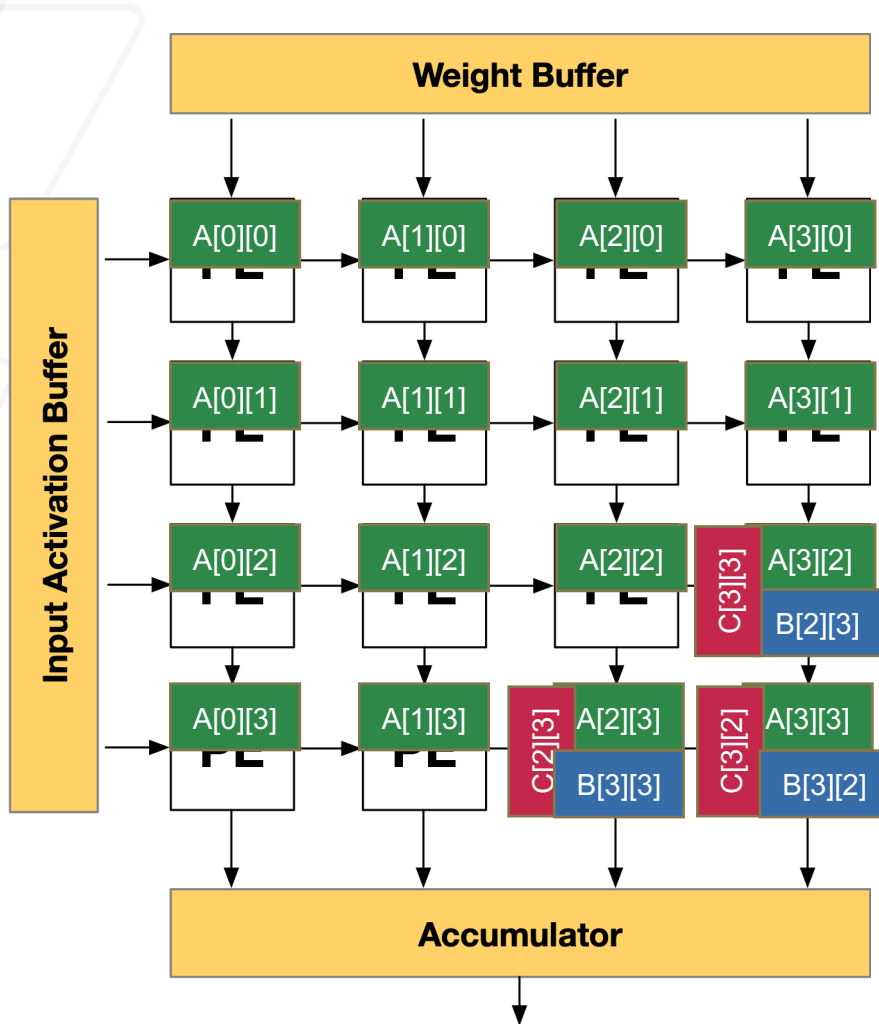
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array

Phase 2:
Stream Activation Tensor



```
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

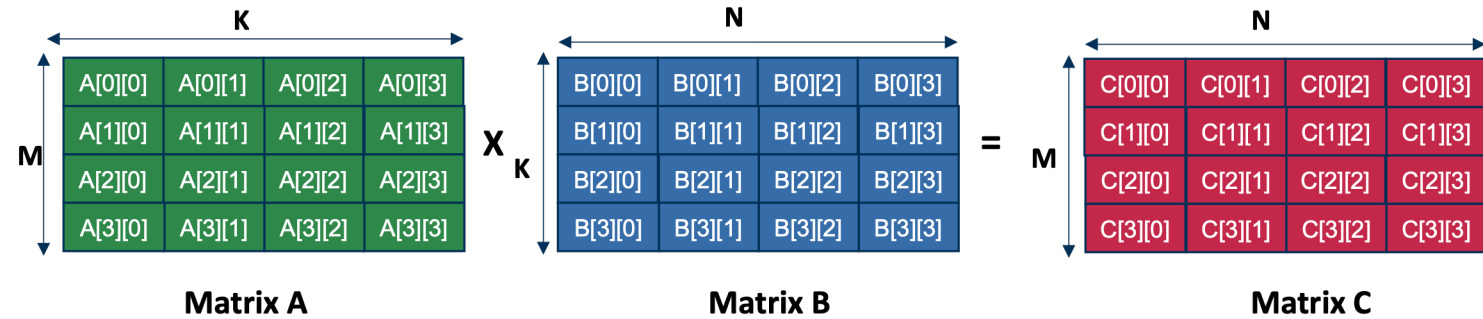
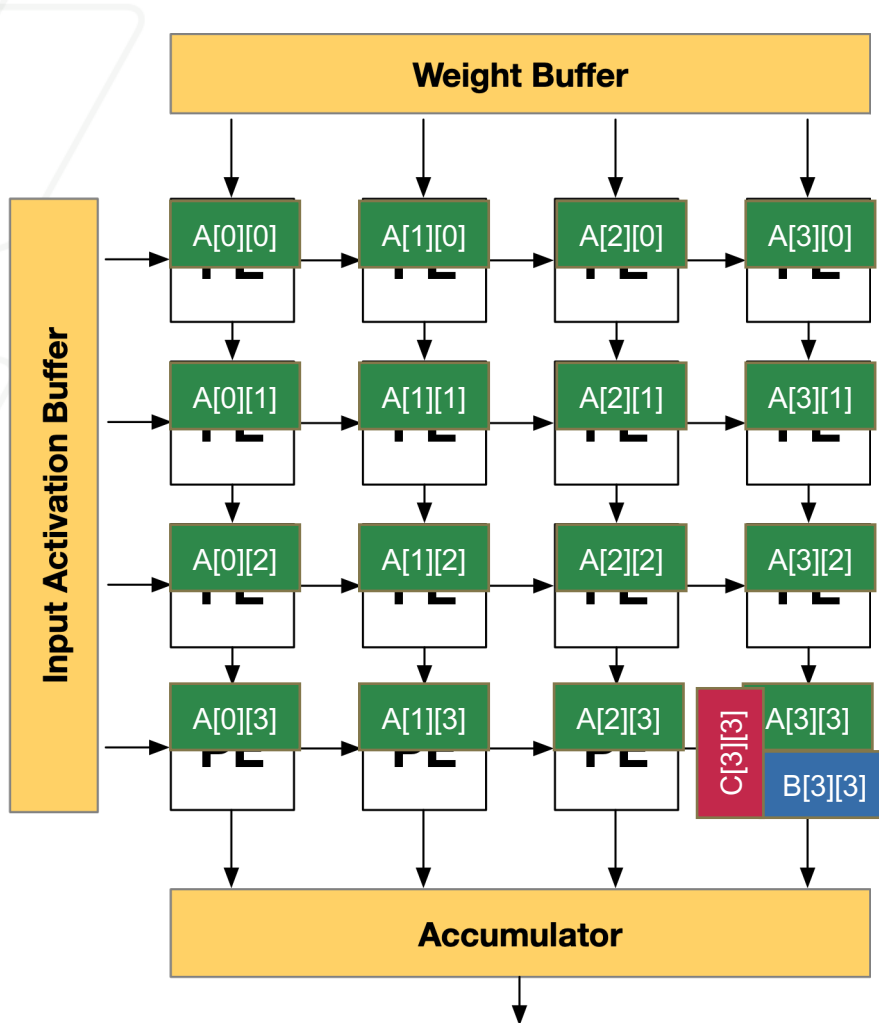
Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array

Phase 2:
Stream Activation Tensor



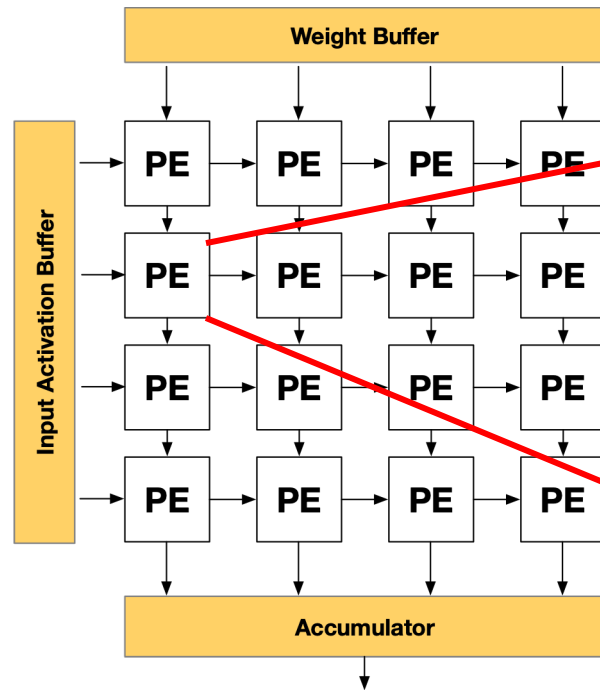
```
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

Assume following “mapping”

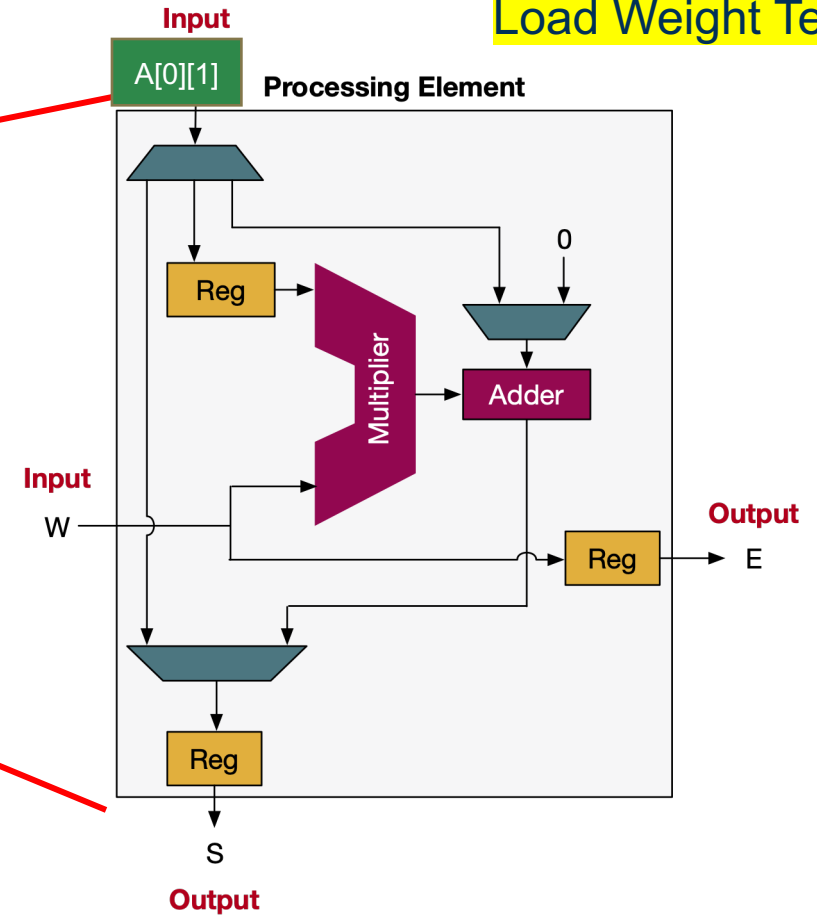
- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

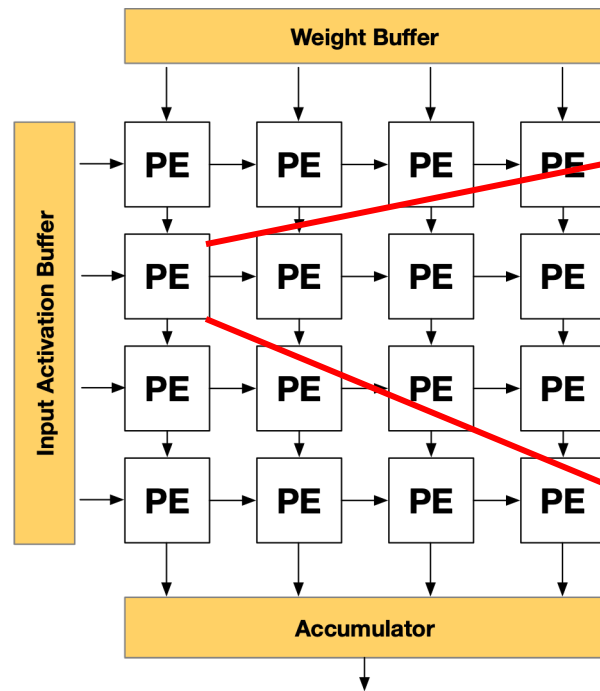
Zooming into PE



Phase 1:
Load Weight Tensor

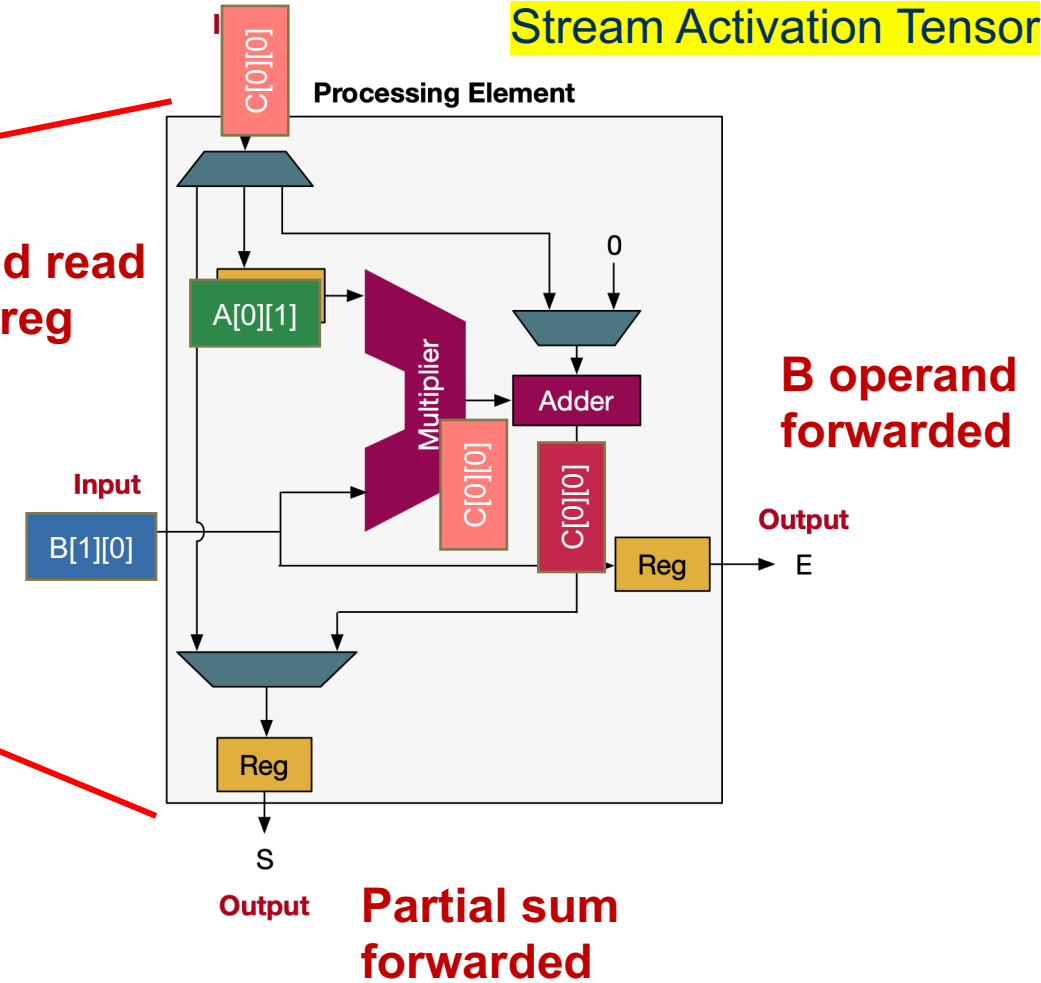


Zooming into PE

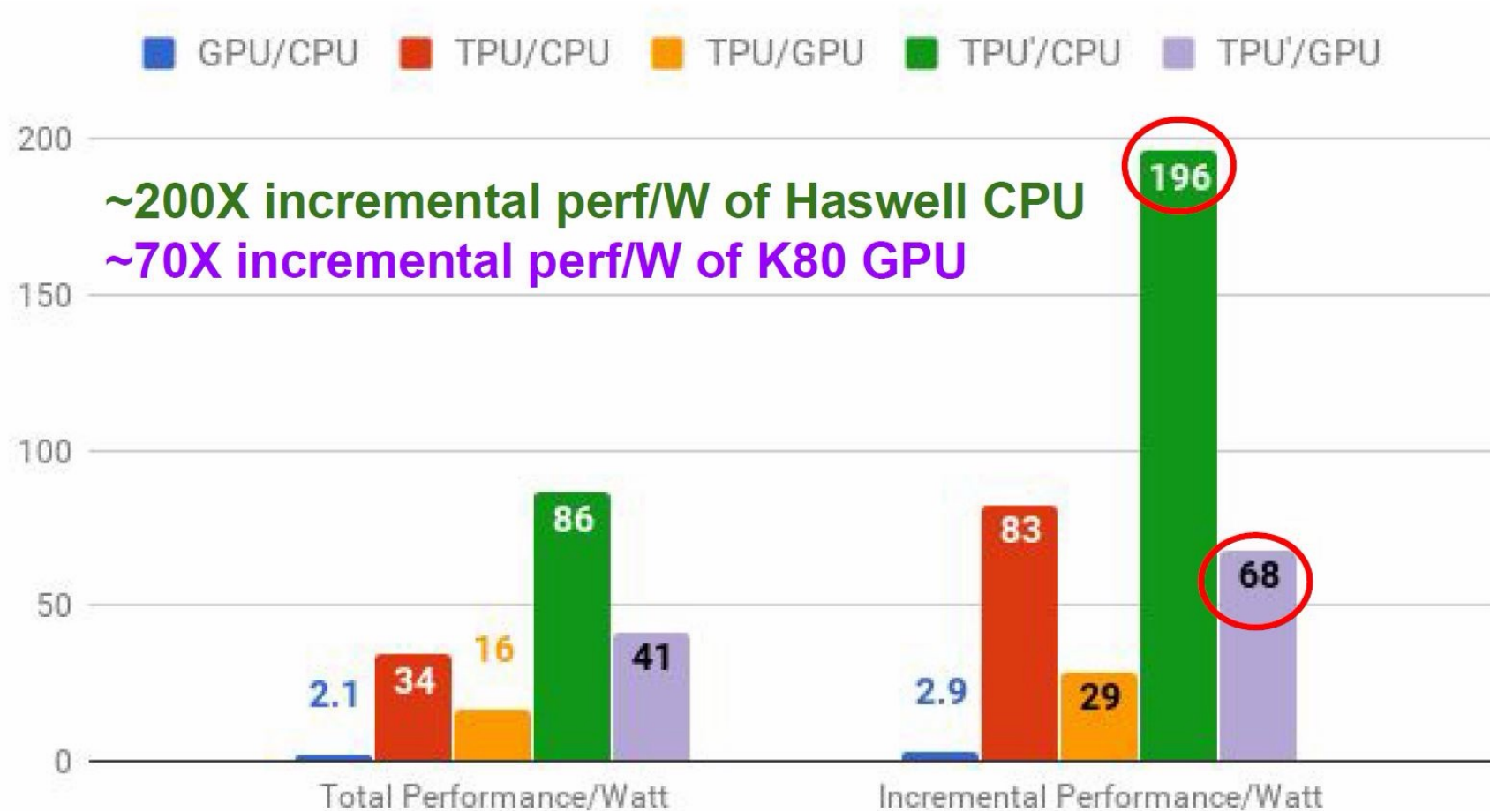


A operand read
via local reg

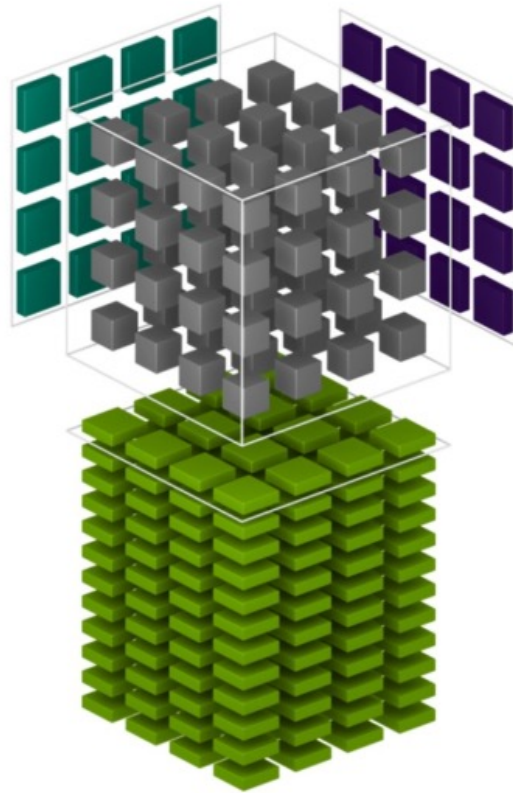
Phase 2: Stream Activation Tensor



Performance/Watt of TPUv1



NVIDIA Response: Tensor Cores (2017)



$$D = \begin{matrix} \text{FP16 or FP32} & \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} & \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} & \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix} \\ & \text{FP16} & \text{FP16} & \text{FP16 or FP32} \end{matrix}$$

Google TPU – large tensor engine vs NVIDIA: multiple tiny tensor engines.

Trade-off?

Outline

- Systolic Array
- **Dataflow**
- Flexible AI Accelerators
- Takeaways

Compute & Memory Behavior of Matrix Multiplication

A1	A2	A3	A4
A5	A6	A7	A8
A9	A10	A11	A12
A13	A14	A15	A16

Matrix A

X

B1	B2	B3	B4
B5	B6	B7	B8
B9	B10	B11	B12
B13	B14	B15	B16

Matrix B

=

C1	C2	C3	C4
C5	C6	C7	C8
C9	C10	C11	C12
C13	C14	C15	C16

Matrix C

Assumptions:

- All input tensors read *once* from **off-chip Main Memory** (DRAM/HBM) and output tensors written to Main Memory
- Infinite **on-chip memory** (RegFile/SRAM scratchpads) to cache them and cache any intermediate results

Example Computation operations

$$C1 = \begin{bmatrix} A1 & A2 & A3 & A4 \end{bmatrix} \cdot \begin{bmatrix} B1 \\ B5 \\ B9 \\ B13 \end{bmatrix}$$

$$C11 = \begin{bmatrix} A9 & A10 & A11 & A12 \end{bmatrix} \cdot \begin{bmatrix} B3 \\ B7 \\ B11 \\ B15 \end{bmatrix}$$

$$C1 = A1*B1 + A2*B5 + A3*B9 + A4*B13$$

$$C5 = A5*B1 + A6*B5 + A7*B9 + A8*B13$$

$$C11 = A9*B3 + A10*B7 + A11*B11 + A12*B15$$

How many multiply and add computations are needed for computing each output?

What is the amount of data that needs to be read from Main Memory?

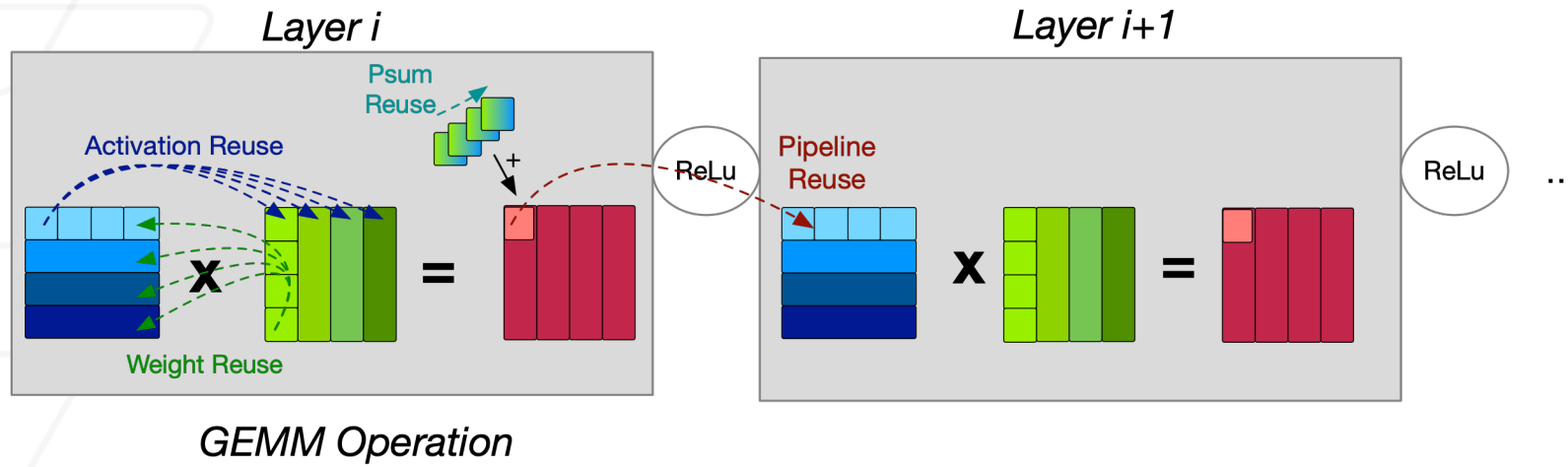
What is the amount of data that needs to be written to Main Memory?

Can any data be “reused” via on-chip memory?

Arithmetic Intensity = Number of Computations / Total Memory Accesses (Rd + Wr)

Is a proxy for max possible data reuse

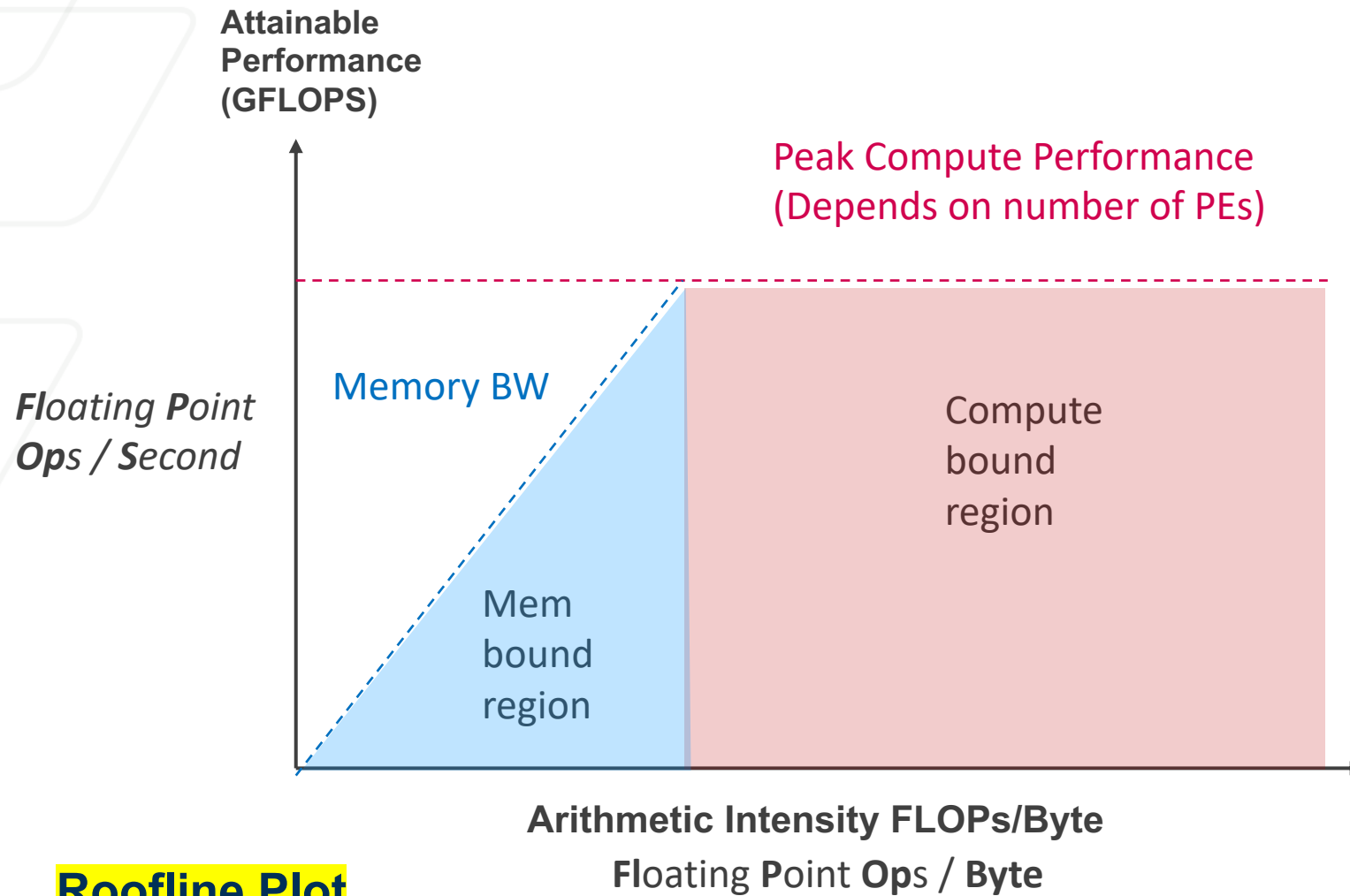
What is Data Reuse?



**Algorithmic Reuse
(aka Arithmetic Intensity)**

Number of computations per
byte of data fetched from
memory

Why does data reuse help?

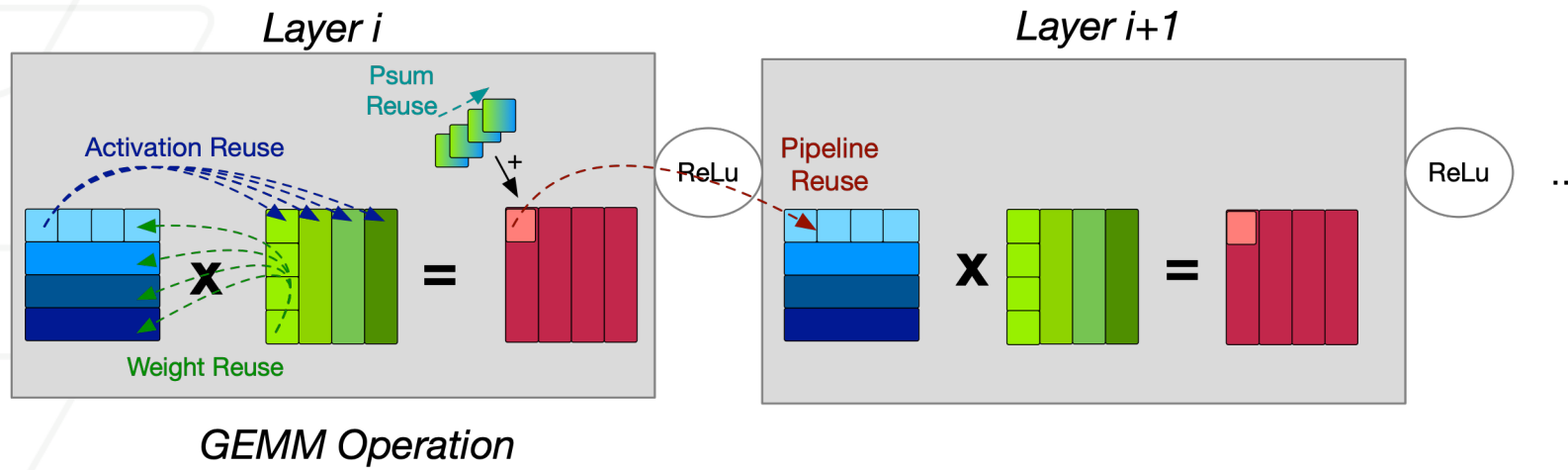


Roofline Plot

Compute Bound => Throughput bound by number of compute units

Memory Bound => Throughput bound by Memory BW

Preview: how to exploit reuse



Algorithmic Reuse (aka Arithmetic Intensity)

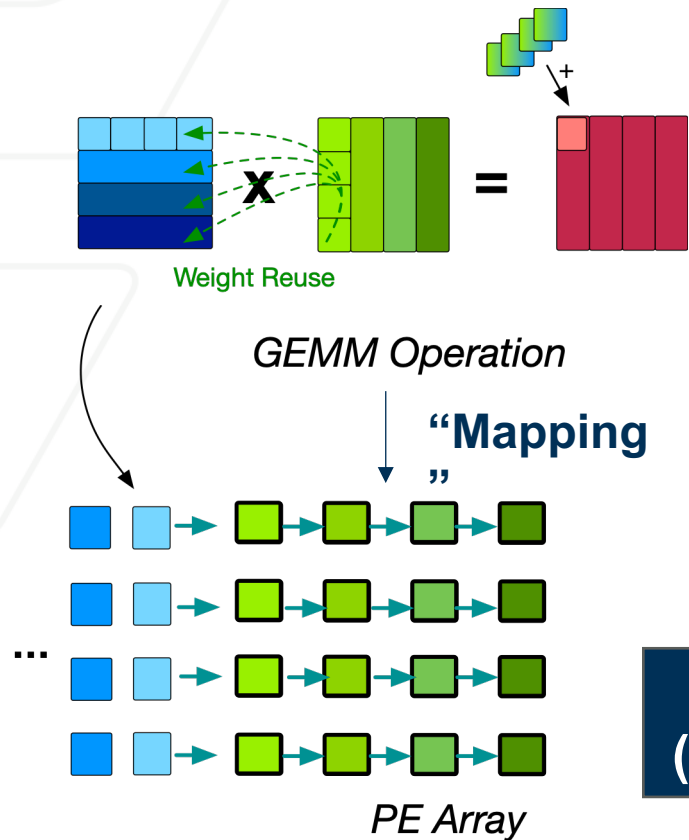
Number of computations per
byte of data fetched from
memory

*How to leverage this
in hardware?*

“Dataflow”

fine-grained compute and memory
scheduling over **space** and **time**

Preview: how to exploit reuse



e.g., Google TPU

Algorithmic Reuse (aka Arithmetic Intensity)

Number of computations per
byte of data fetched from
memory

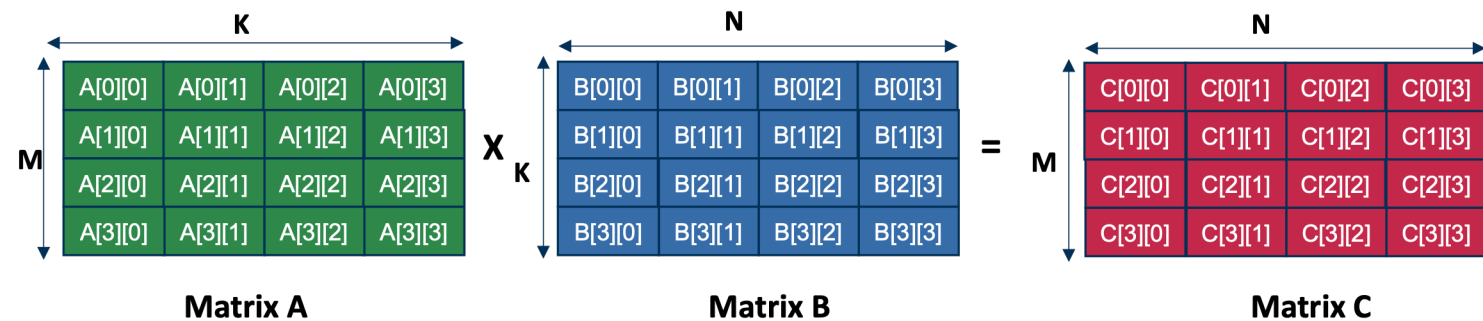
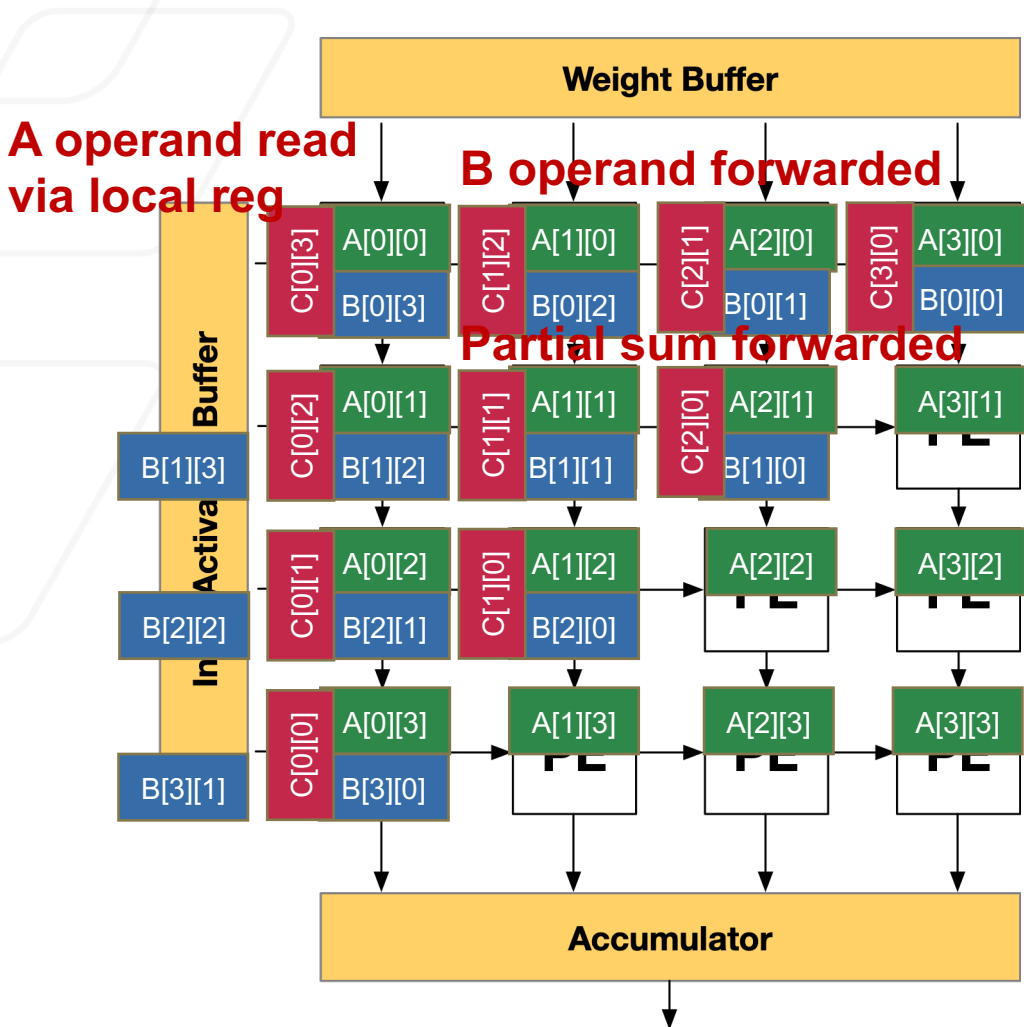
*How to leverage this
in hardware?*

“Dataflow”

fine-grained compute and memory
scheduling over **space** and **time**

Hardware Reuse (e.g., *Weight Stationary Dataflow*)

Data Reuse Analysis: Qualitative



“Weight-stationary” Dataflow

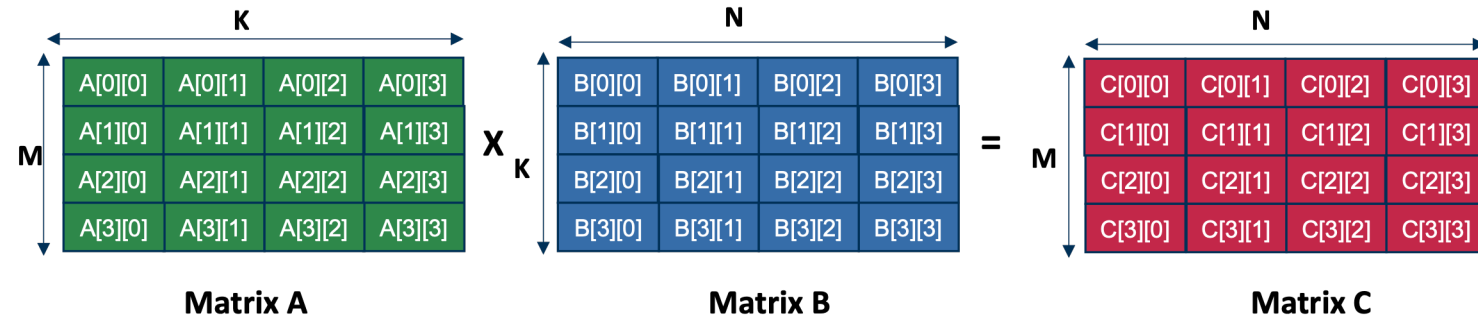
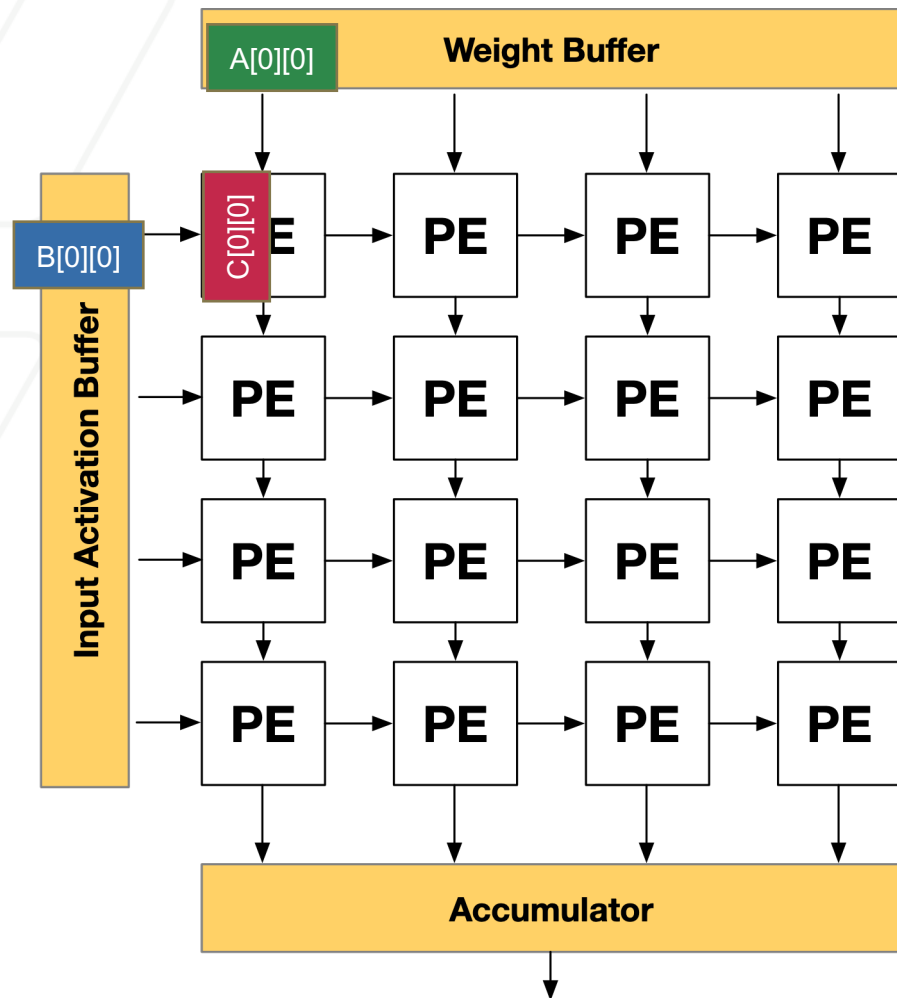
Tensor (A/B/C)	Reuse (Spatial/Temporal/None)
A	Temporal
B	Spatial
C	Spatial

Assume following “mapping”

- Column: Reduced (aka contracted) Dimension (K)
- Row: Spatial Dimension (either M or N)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array



```

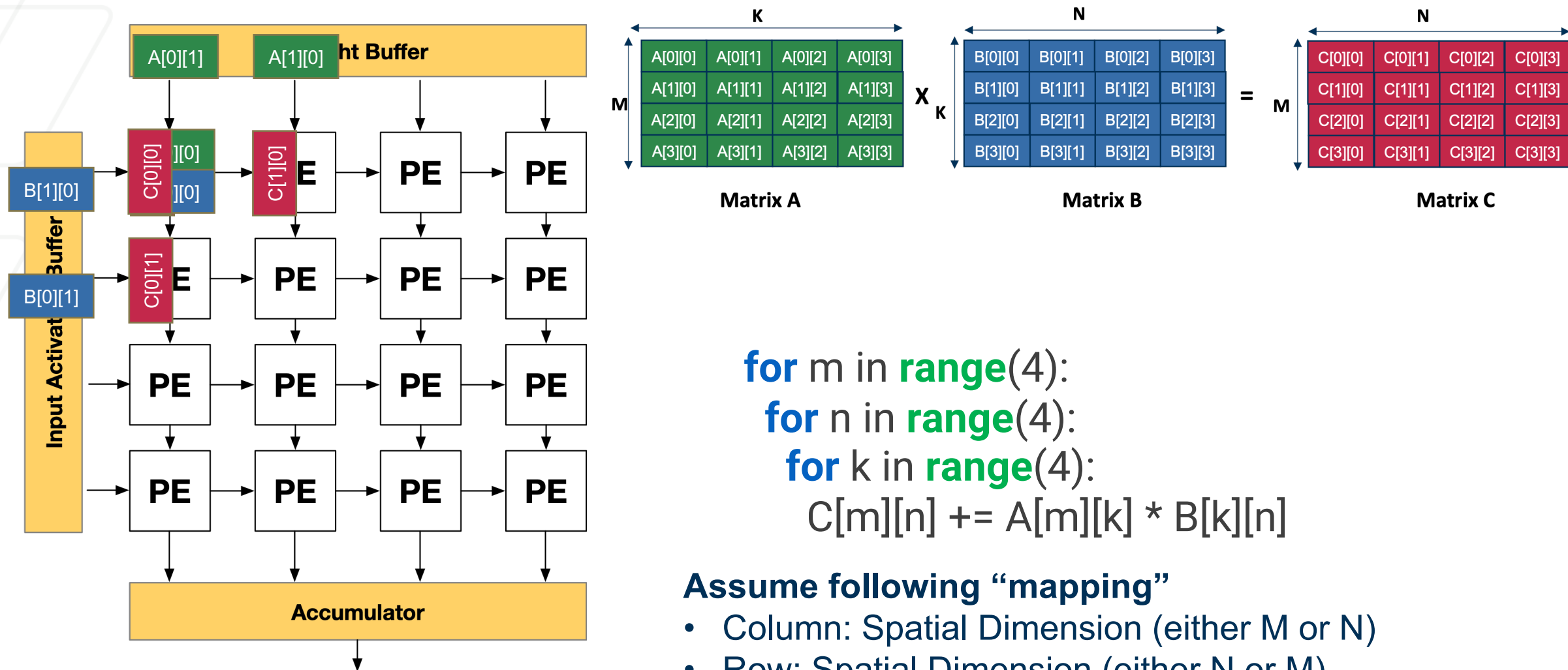
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
  
```

Assume following “mapping”

- Column: Spatial Dimension (either M or N)
- Row: Spatial Dimension (either N or M)
- Within PE: contracted dimension (K)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array



```

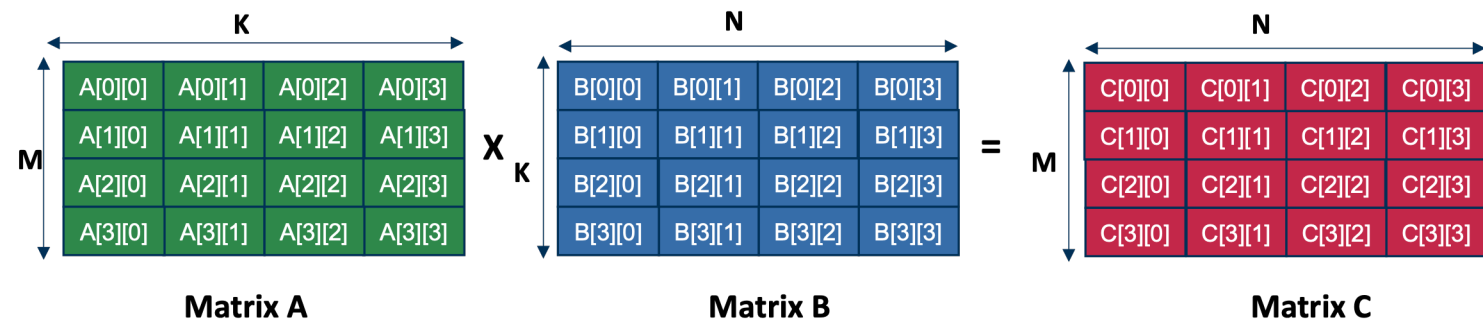
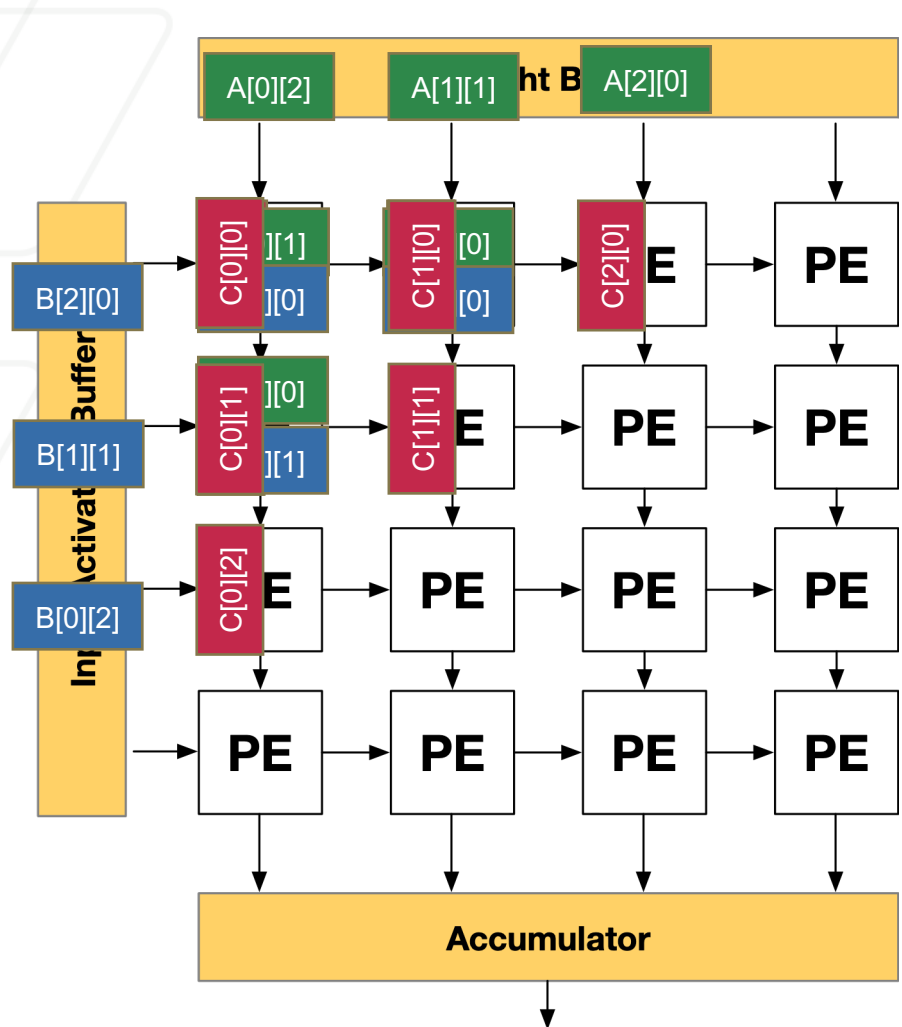
for m in range(4):
    for n in range(4):
        for k in range(4):
            C[m][n] += A[m][k] * B[k][n]
  
```

Assume following “mapping”

- Column: Spatial Dimension (either M or N)
- Row: Spatial Dimension (either N or M)
- Within PE: contracted dimension (K)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array



```

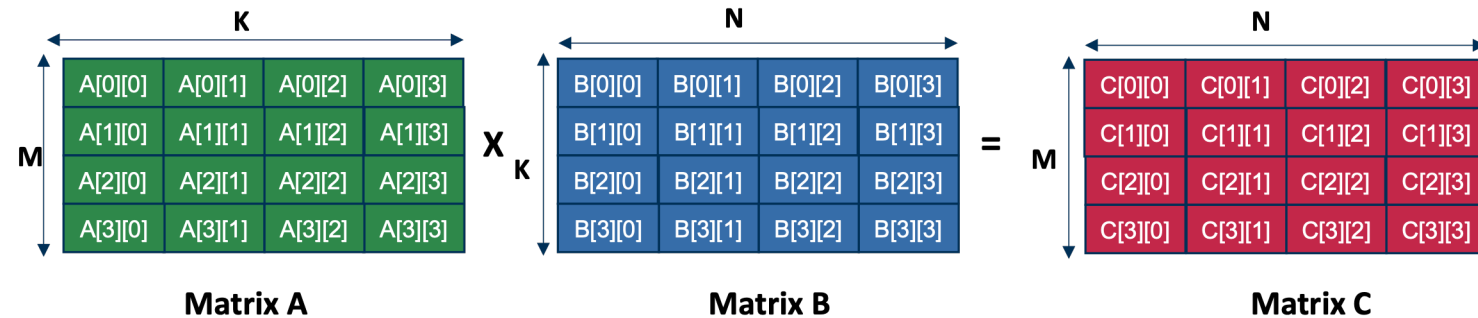
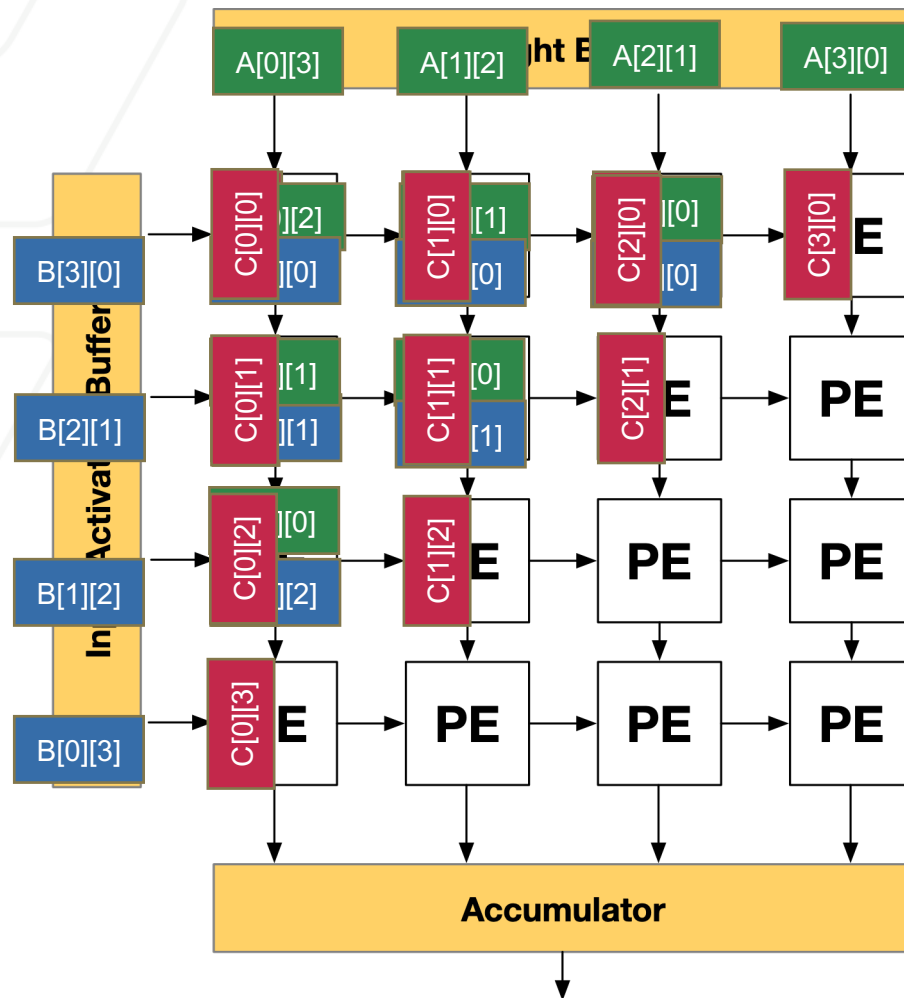
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
  
```

Assume following “mapping”

- Column: Spatial Dimension (either M or N)
- Row: Spatial Dimension (either N or M)
- Within PE: contracted dimension (K)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array



```

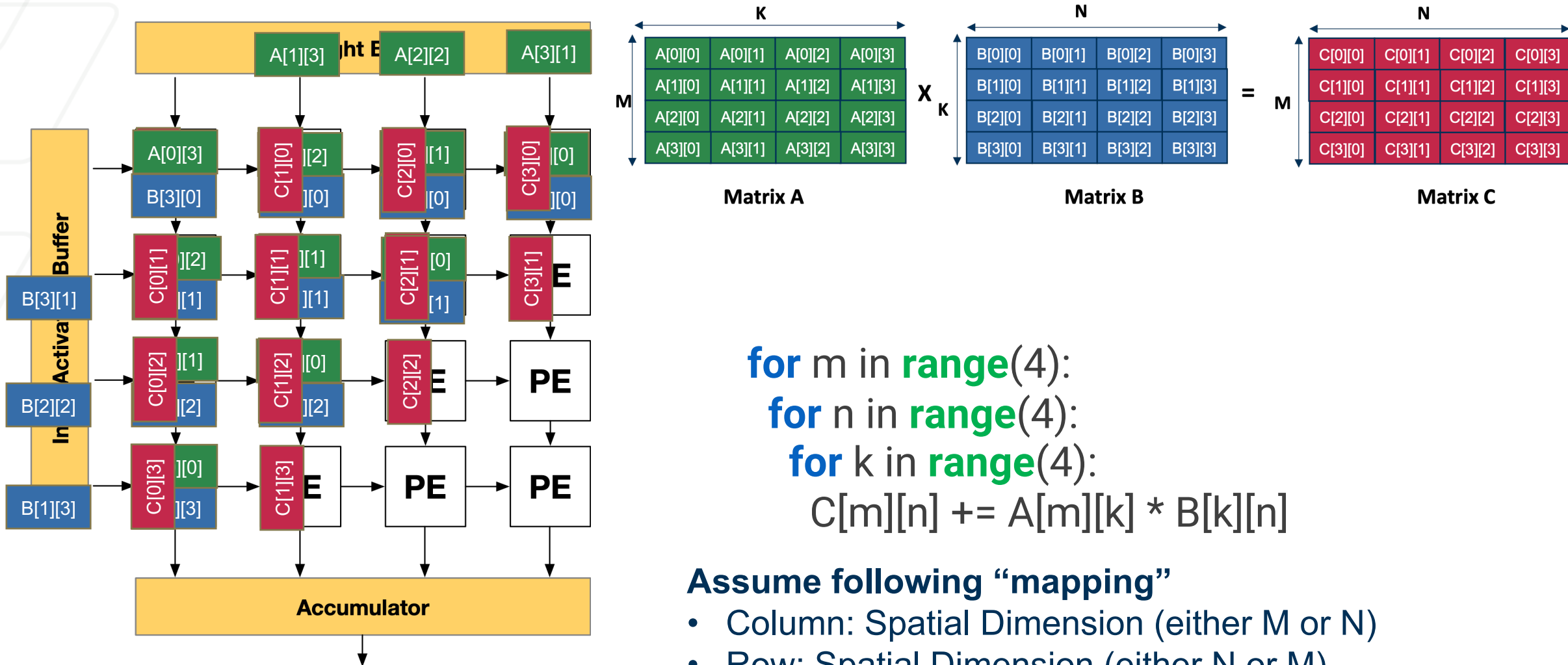
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
  
```

Assume following “mapping”

- Column: Spatial Dimension (either M or N)
- Row: Spatial Dimension (either N or M)
- Within PE: contracted dimension (K)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array



```

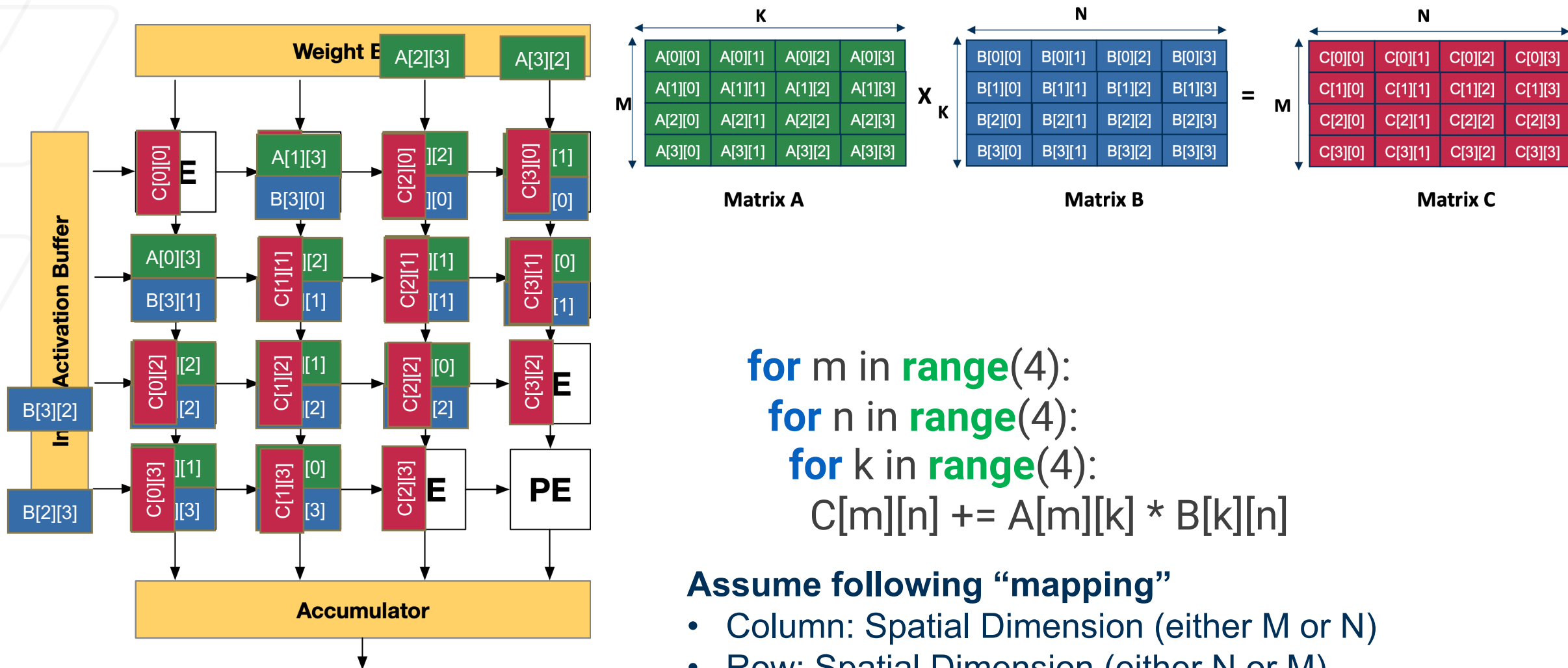
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
  
```

Assume following “mapping”

- Column: Spatial Dimension (either M or N)
- Row: Spatial Dimension (either N or M)
- Within PE: contracted dimension (K)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array



```

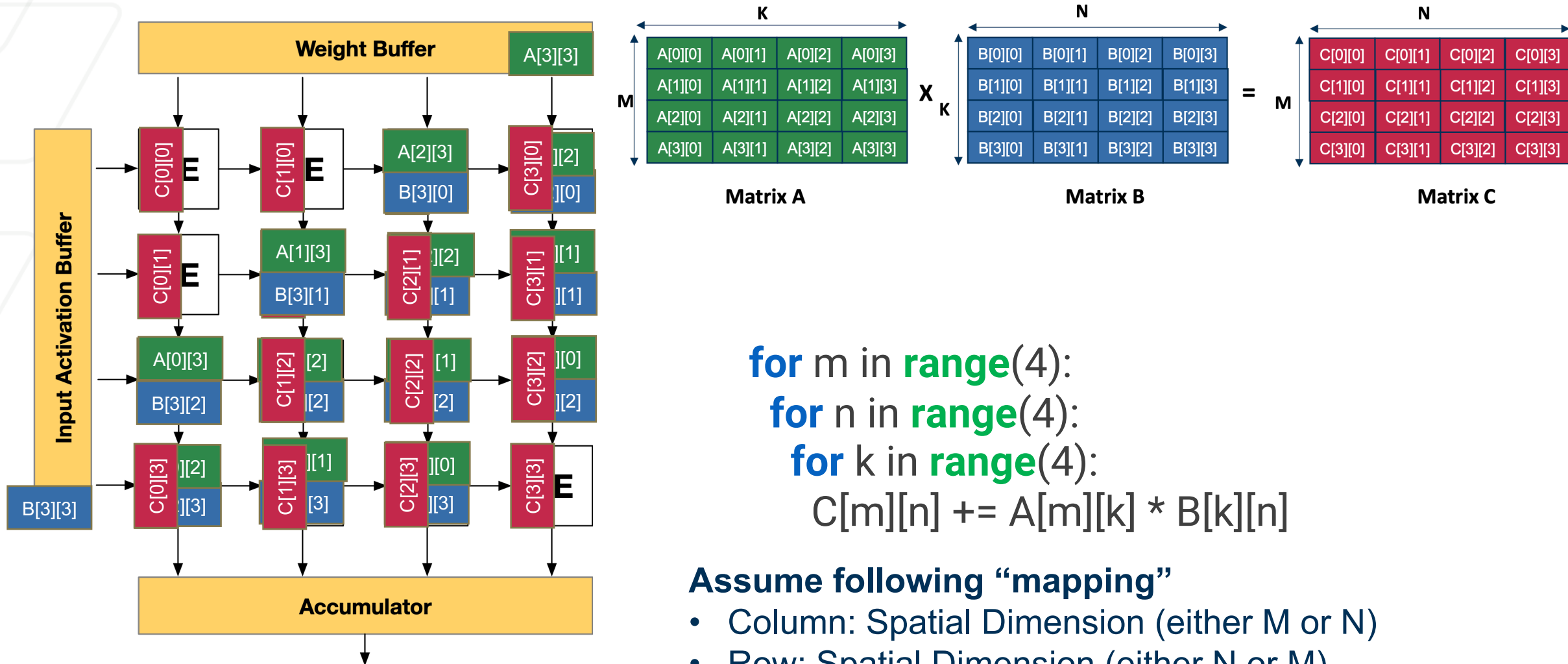
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
  
```

Assume following “mapping”

- Column: Spatial Dimension (either M or N)
- Row: Spatial Dimension (either N or M)
- Within PE: contracted dimension (K)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array



```

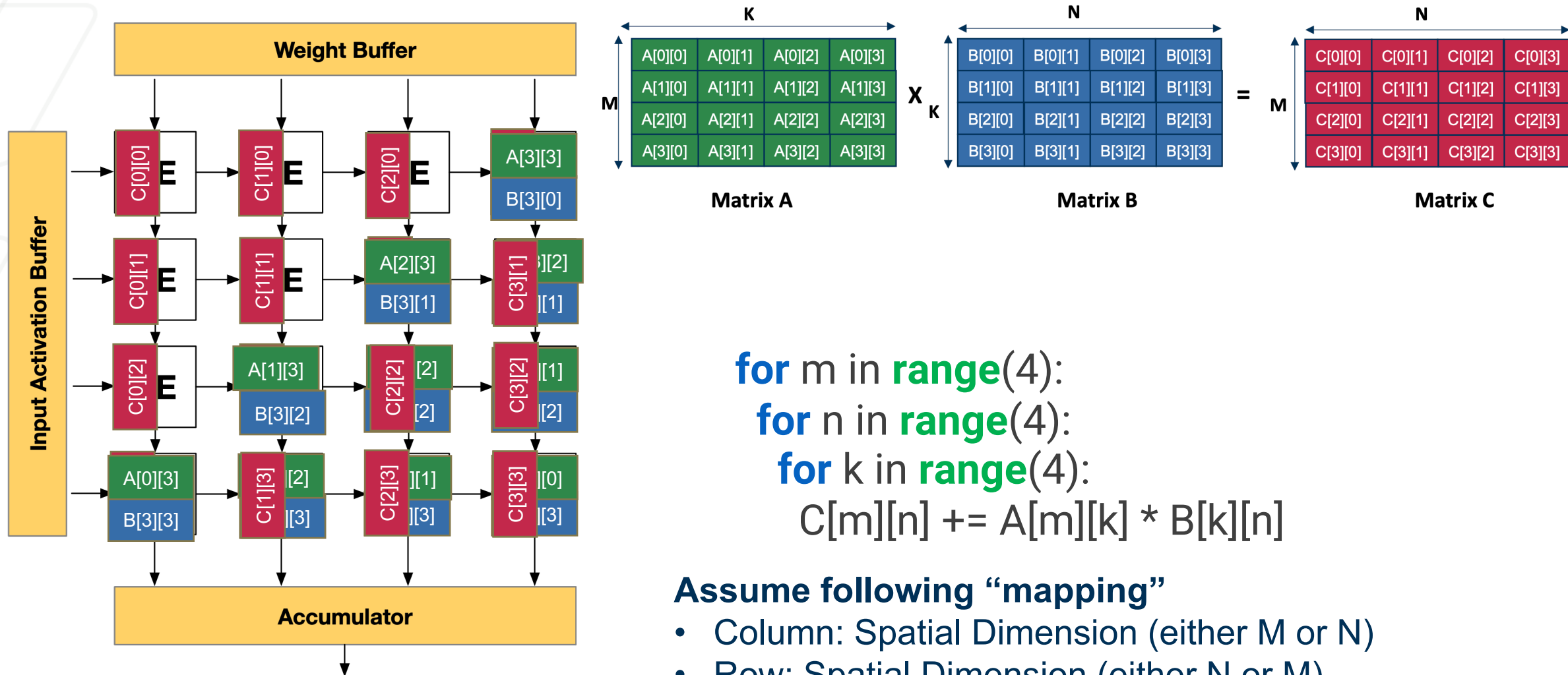
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
  
```

Assume following “mapping”

- Column: Spatial Dimension (either M or N)
- Row: Spatial Dimension (either N or M)
- Within PE: contracted dimension (K)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array



```

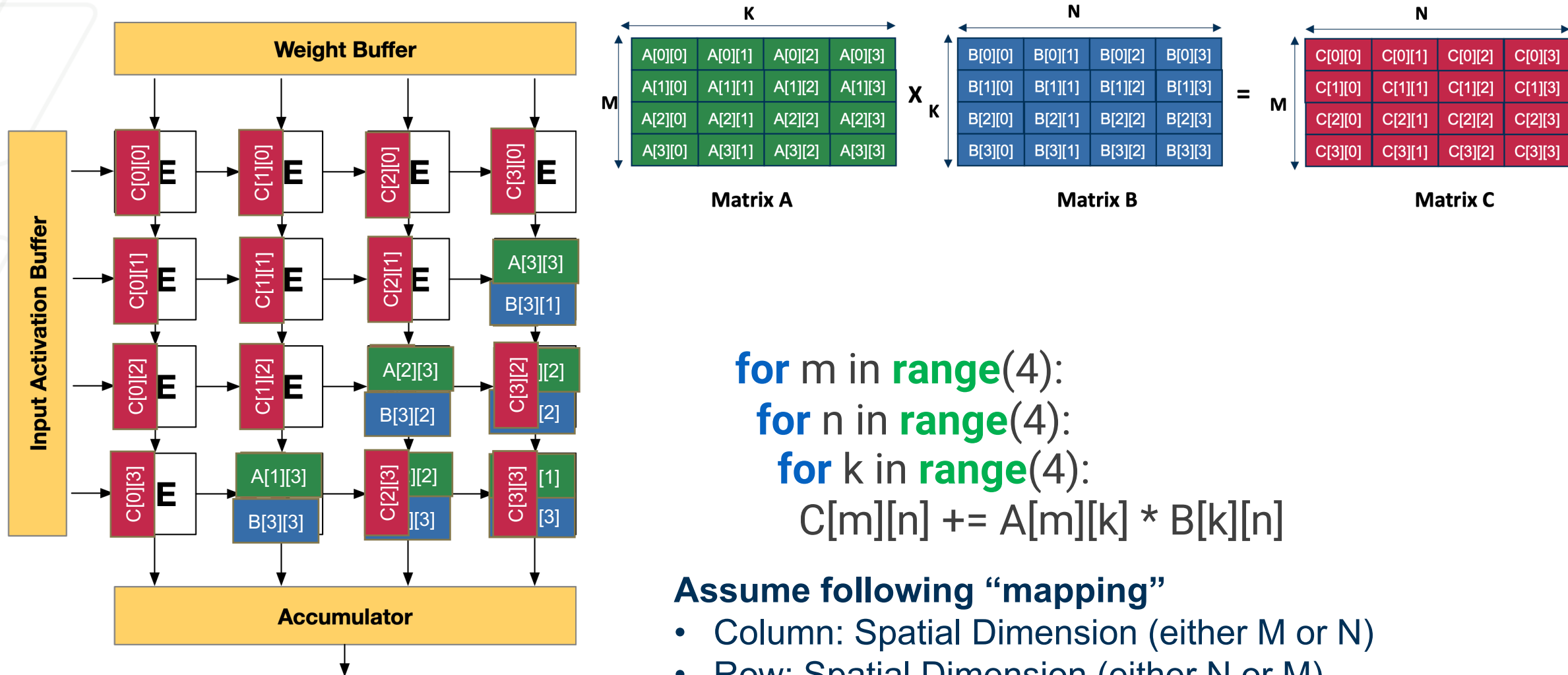
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
  
```

Assume following “mapping”

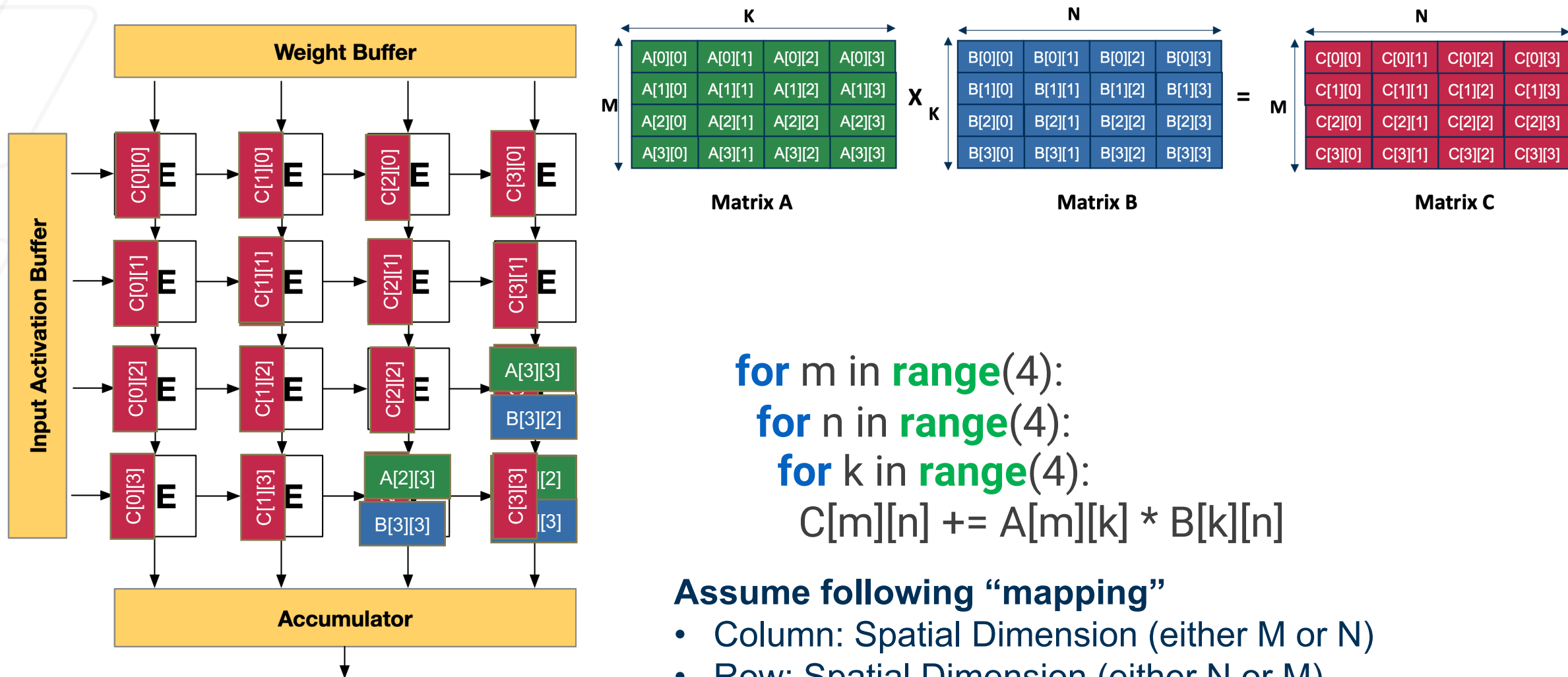
- Column: Spatial Dimension (either M or N)
- Row: Spatial Dimension (either N or M)
- Within PE: contracted dimension (K)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array



Matrix Multiplication on a Systolic Array



```

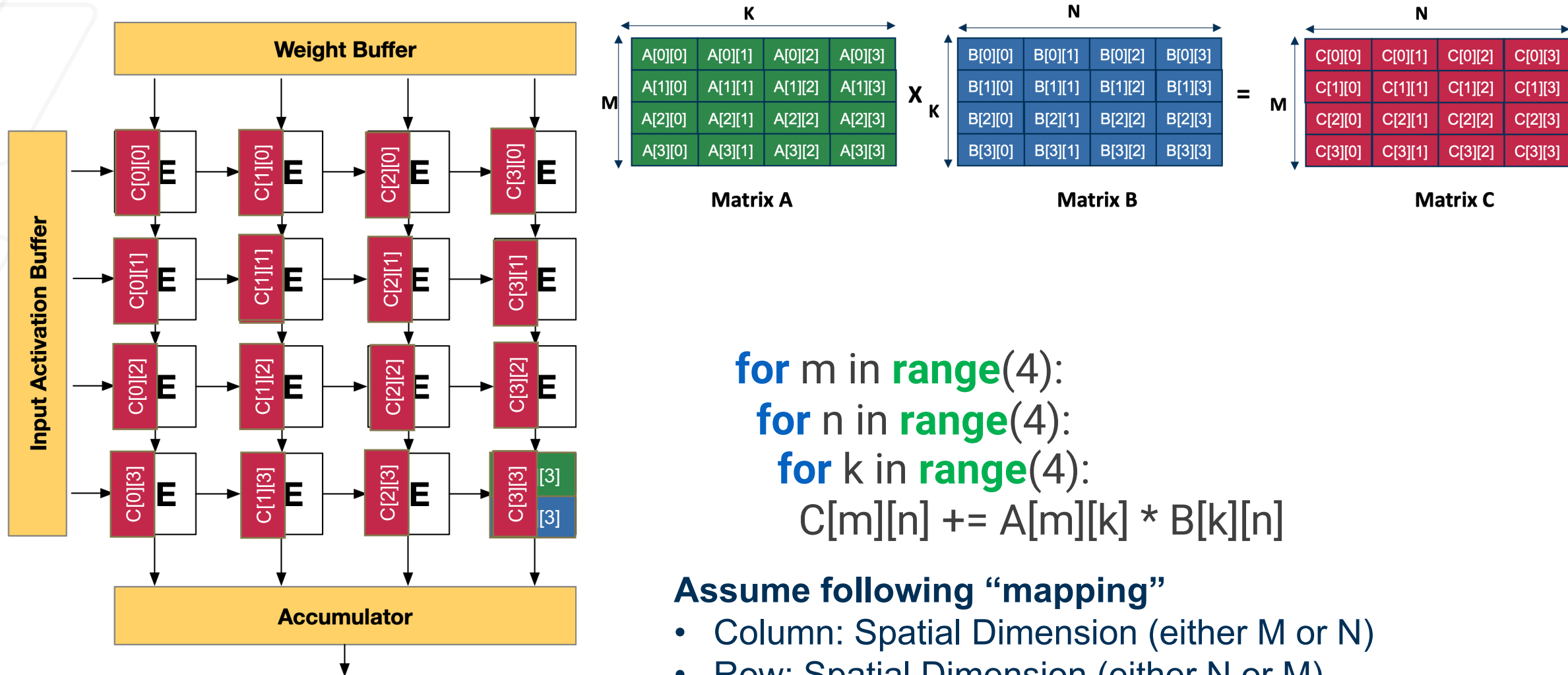
for m in range(4):
    for n in range(4):
        for k in range(4):
            C[m][n] += A[m][k] * B[k][n]
  
```

Assume following “mapping”

- Column: Spatial Dimension (either M or N)
- Row: Spatial Dimension (either N or M)
- Within PE: contracted dimension (K)

Note: alternate mapping styles possible

Matrix Multiplication on a Systolic Array



```

for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
  
```

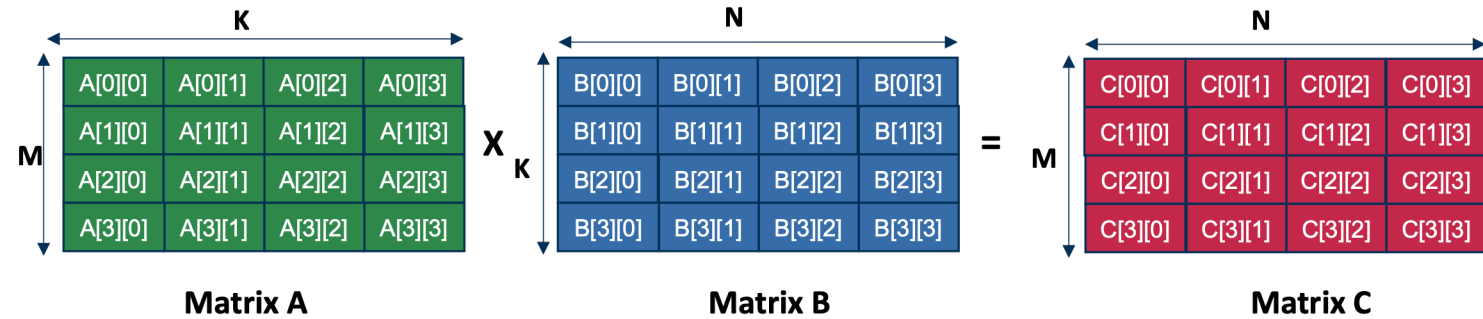
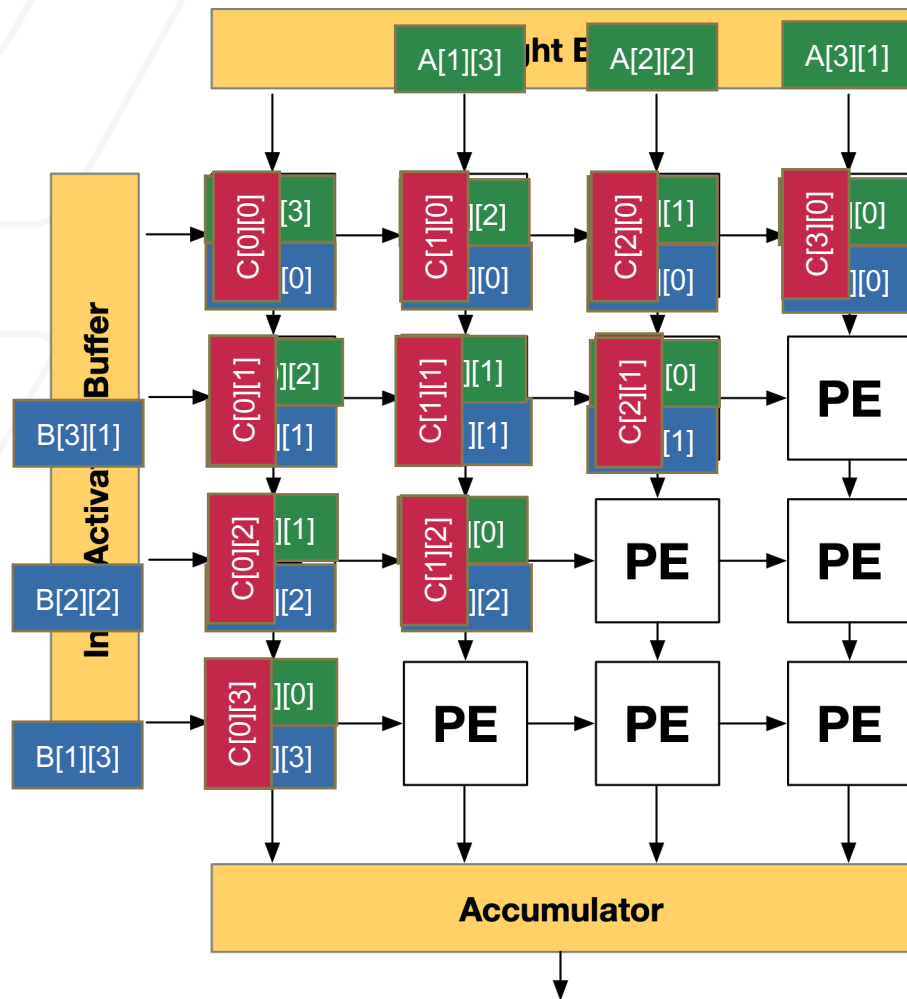
Assume following “mapping”

- Column: Spatial Dimension (either M or N)
- Row: Spatial Dimension (either N or M)
- Within PE: contracted dimension (K)

Note: alternate mapping styles possible

Note: need another phase to read outputs out (not shown)

Data Reuse Analysis: Qualitative



“Output-stationary” Dataflow

Tensor (A/B/C)	Reuse (Spatial/Temporal/None)
A	Spatial
B	Spatial
C	Temporal

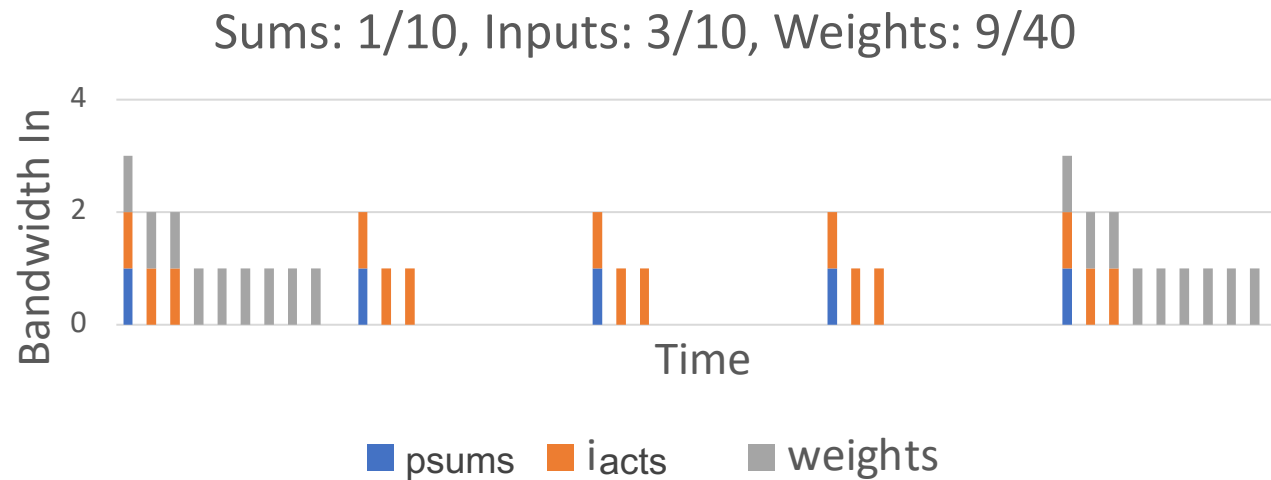
Assume following “mapping”

- Column: Spatial Dimension (either M or N)
- Row: Spatial Dimension (either N or M)
- Within PE: contracted dimension (K)

Note: alternate mapping styles possible

What do we mean by “stationary”?

- The datatype (and dimension) that changes most slowly



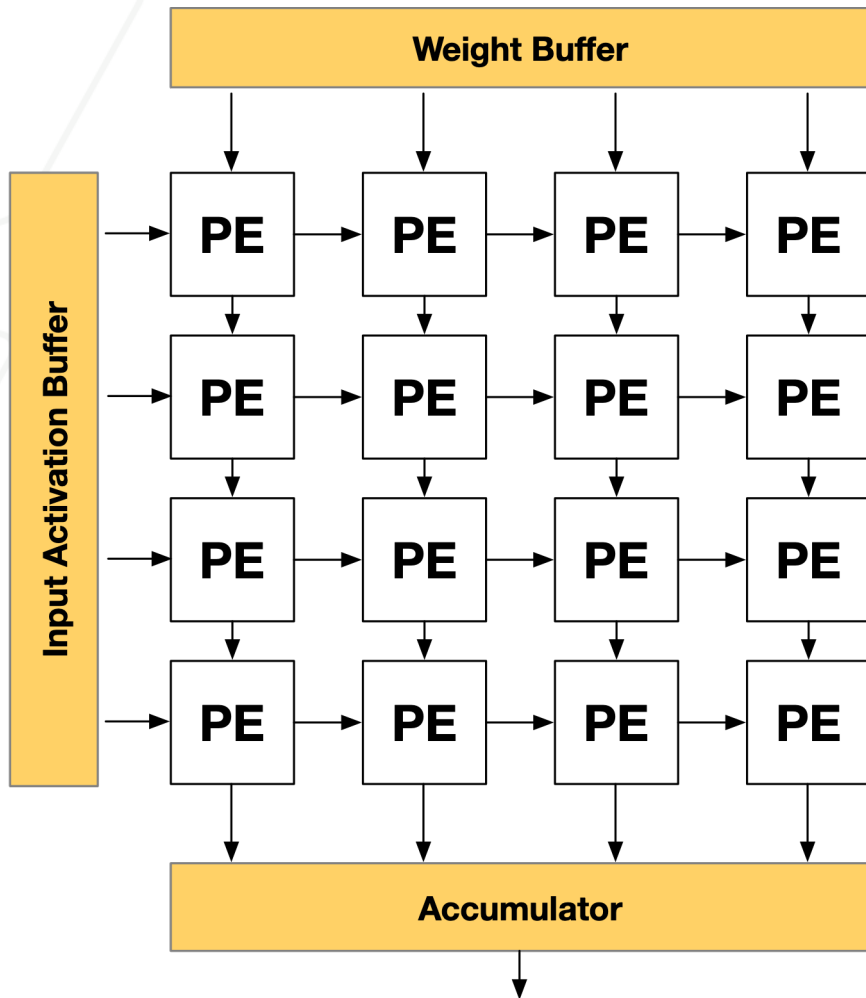
- Often corresponds to datatype that is “done with” earliest without further reloads

Note: the “stationary” name is meant to give intuition, not to be a complete specification of all the behavior of a dataflow

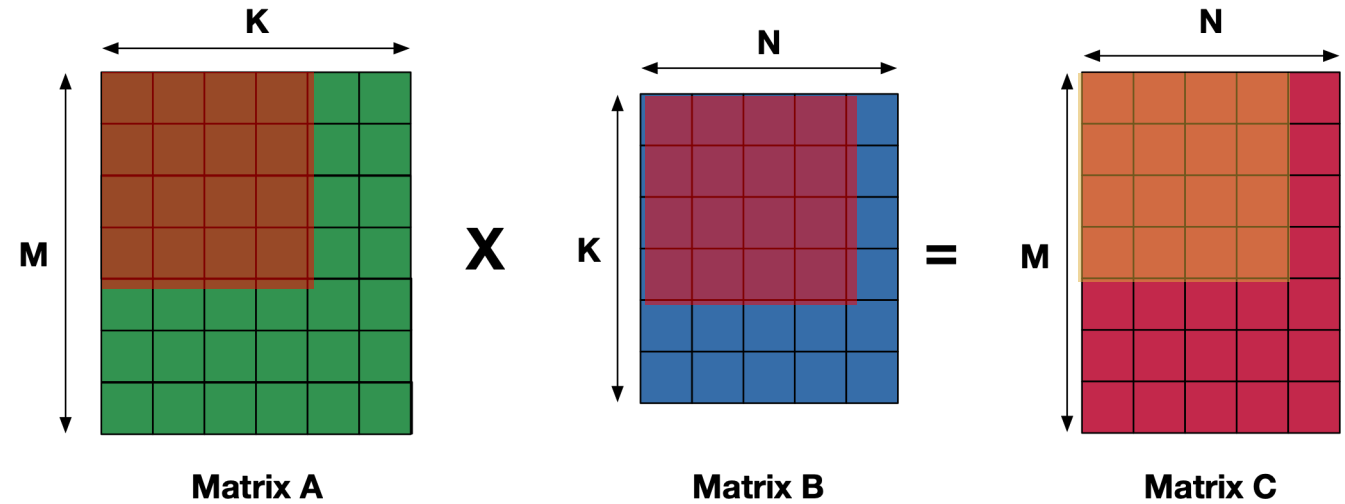
What is “dataflow”

- Fine-grained *schedule* of data movement through the accelerator
- **Components of Dataflow**
 - Loop Order
 - Parallelism
- **What does dataflow determine?**
 - Amount of reuse (**limit:** Arithmetic Intensity)
 - Mechanism of reuse (spatial or temporal)
- **How is dataflow determined?**
 - Depends on the “Mapping”
 - Tile Sizes + Dataflow
 - May be constrained by hardware
 - e.g., a systolic array may only support a specific dataflow

Mapping



Suppose M , K and N are > 4



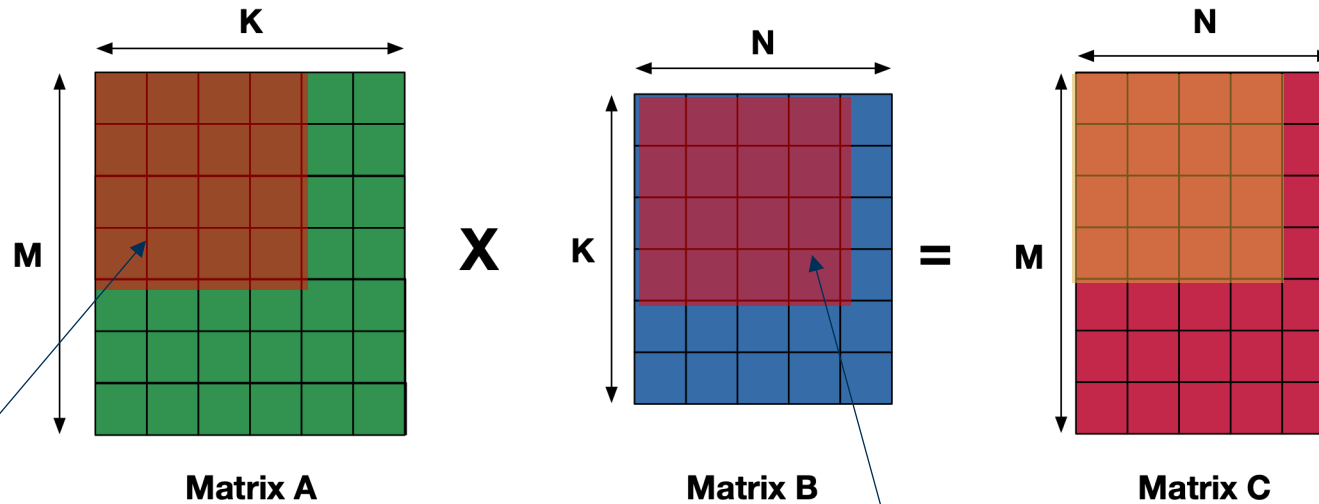
How would you run this over the 4x4 array?

“Tile” the computation

Mapping Choice: Tile Size

Option 1

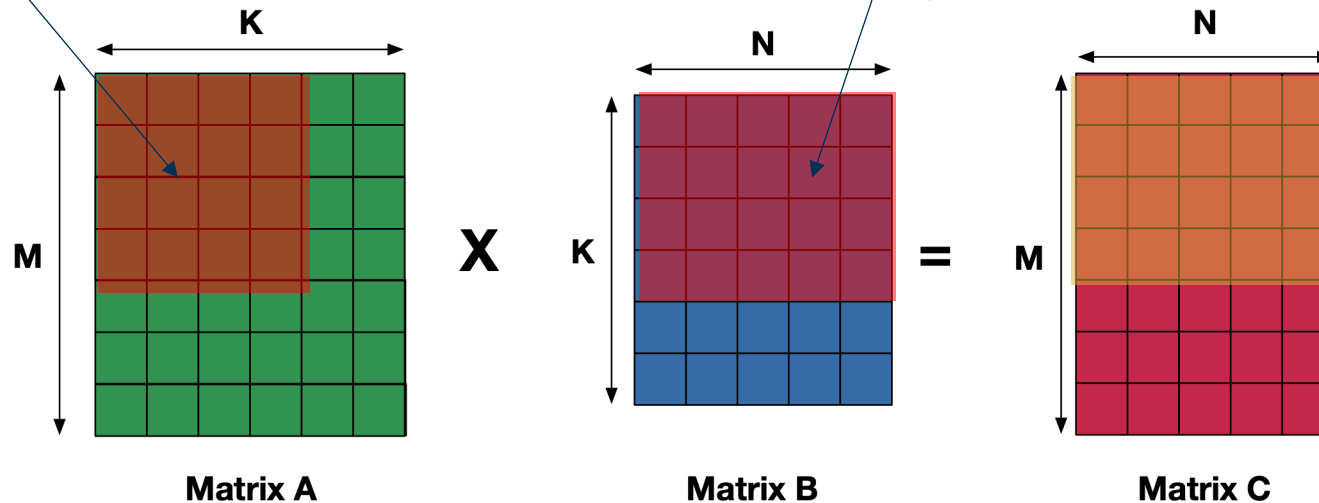
Stationary tile is 4x4
(constrained by
array dimensions)



Streaming tile is 4x4

Streaming tile is 4x5

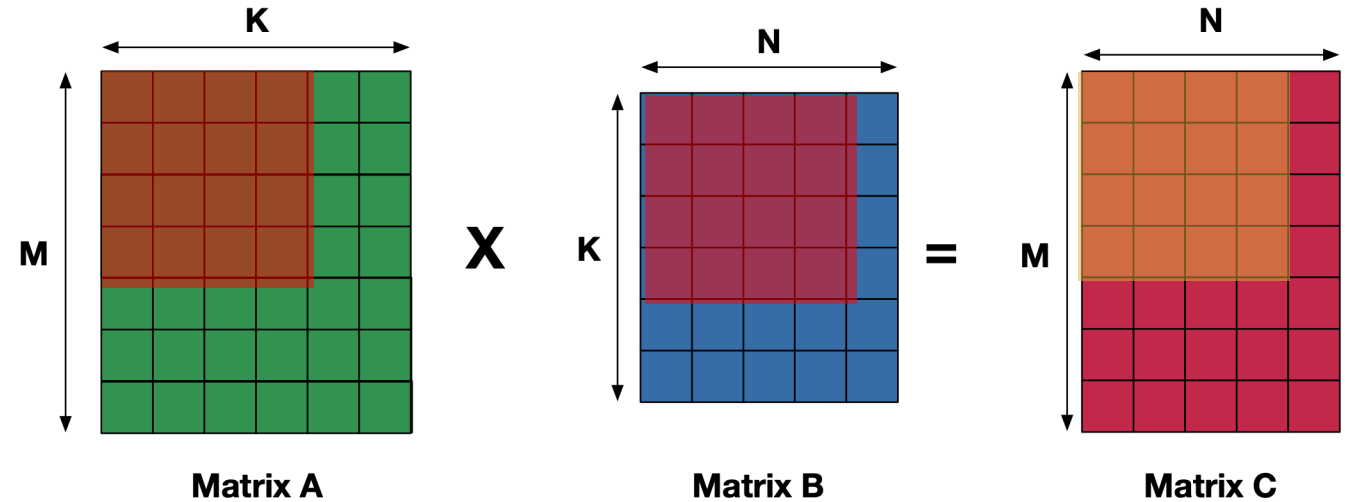
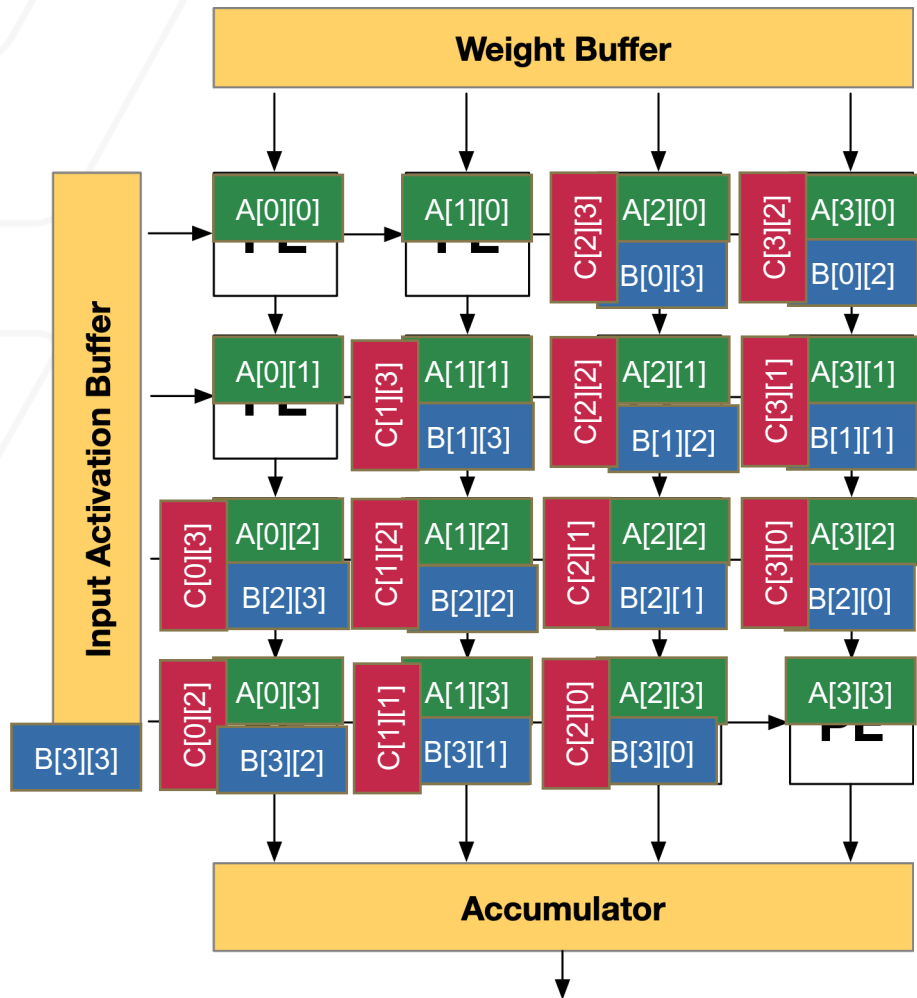
Option 2



Which one is better?

Mapping Choice: Tile Size – Option 1

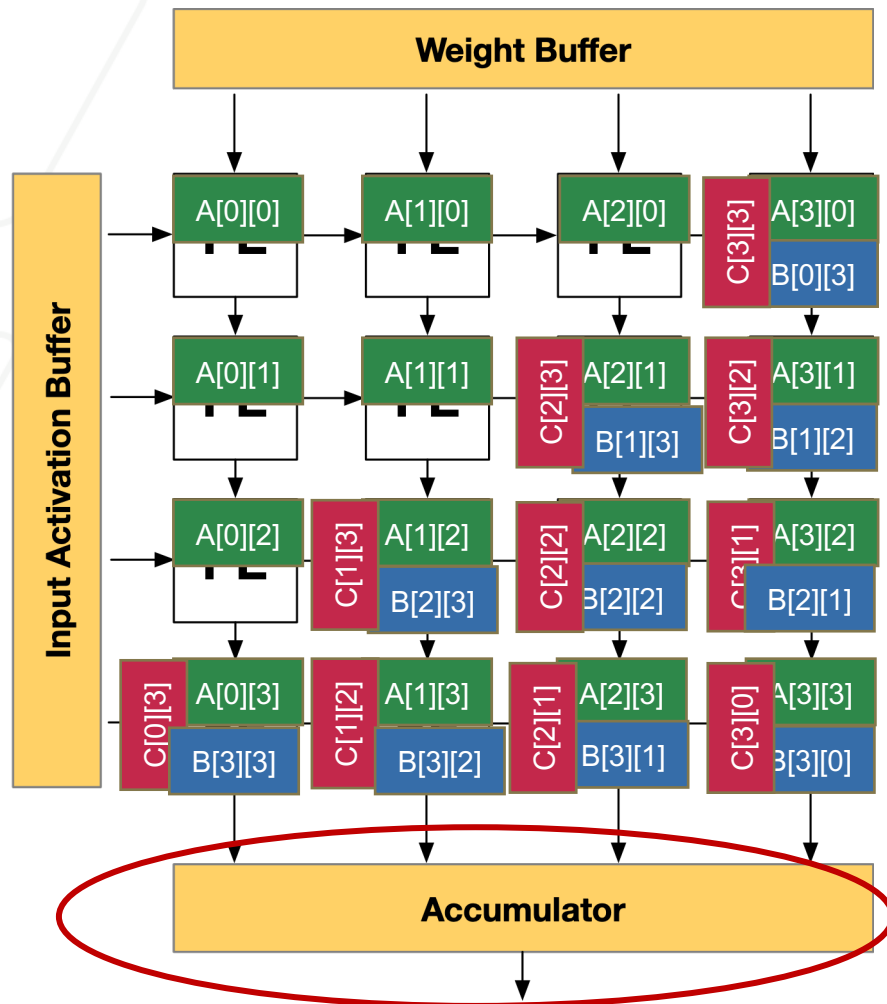
Phase 2:
Stream Activation Tensor



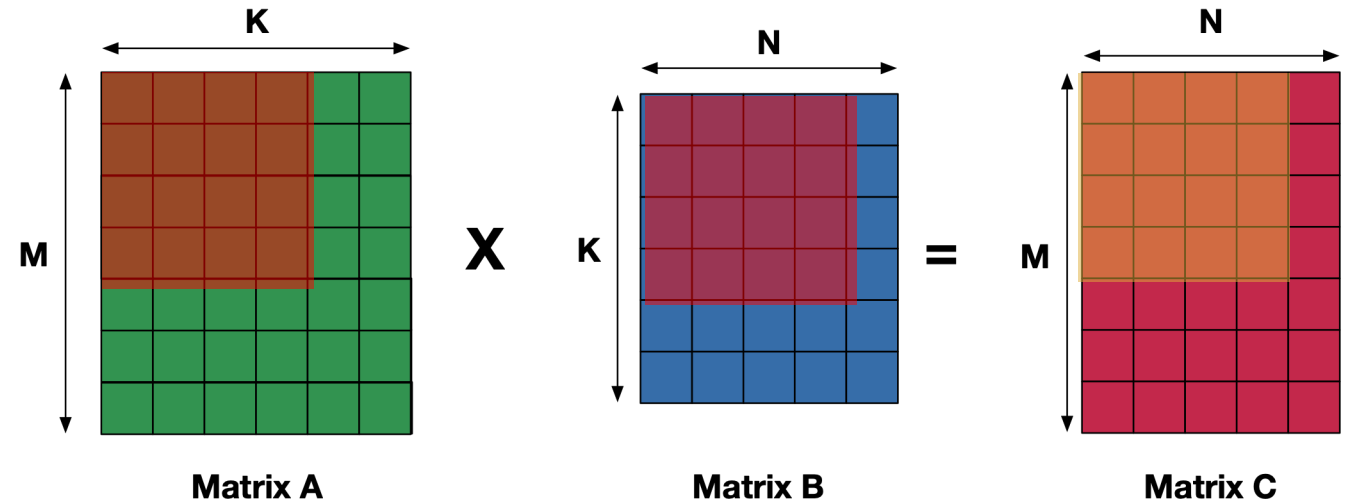
```
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

Mapping Choice: Tile Size – Option 1

Phase 2:
Stream Activation Tensor



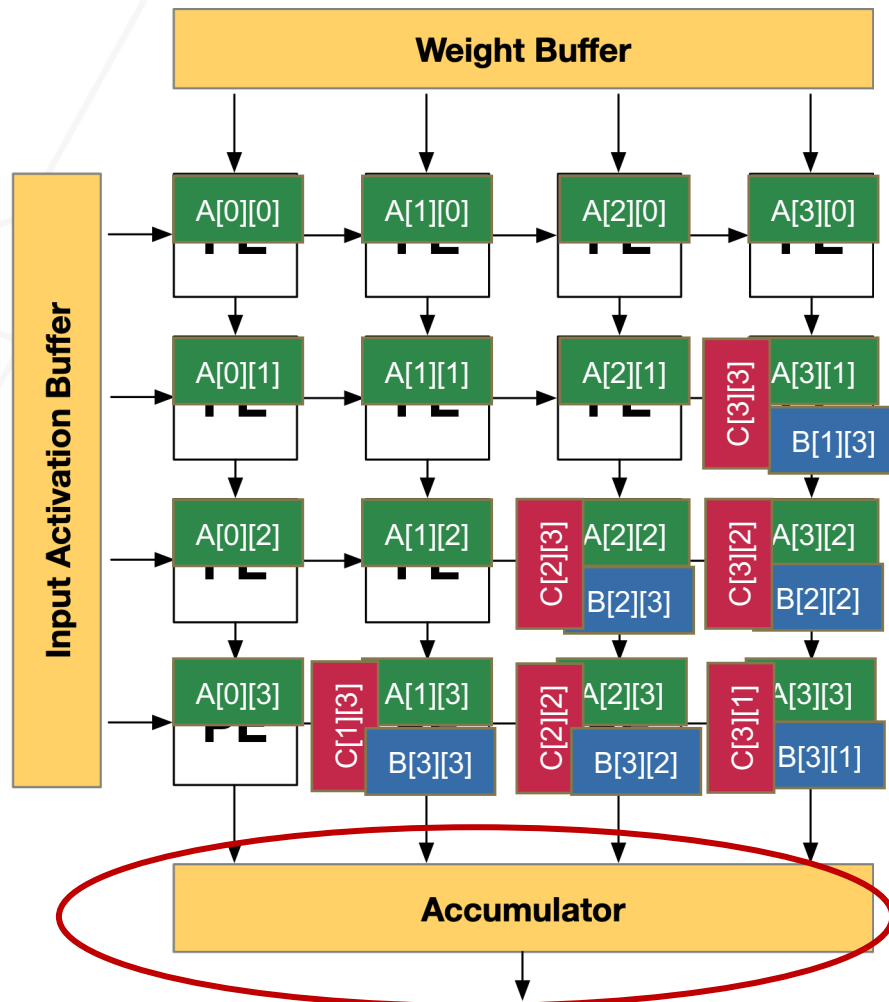
Four outputs



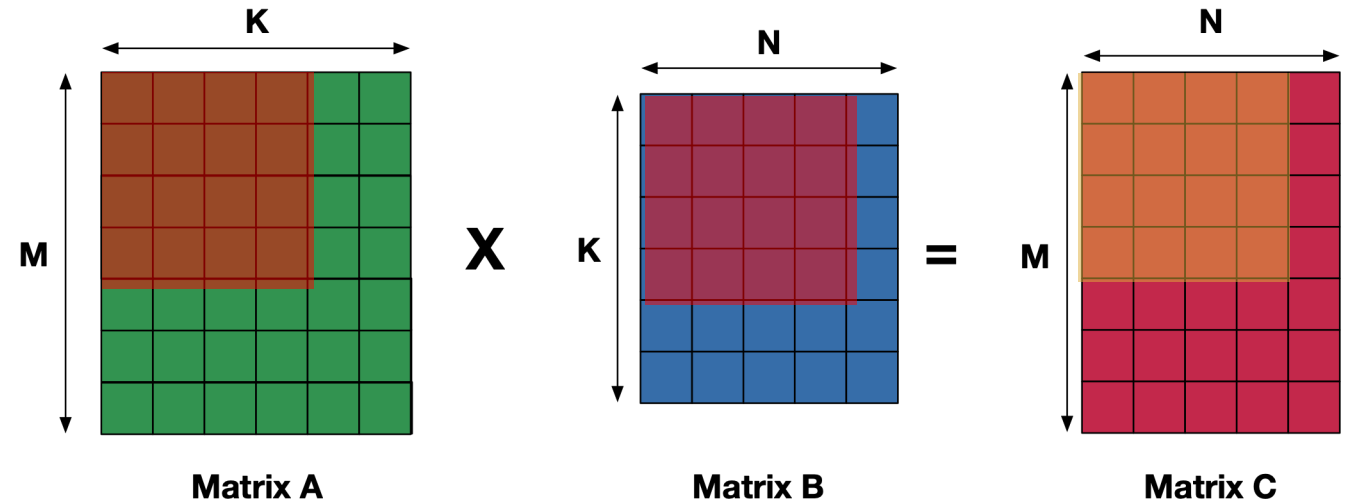
```
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

Mapping Choice: Tile Size – Option 1

Phase 2:
Stream Activation Tensor



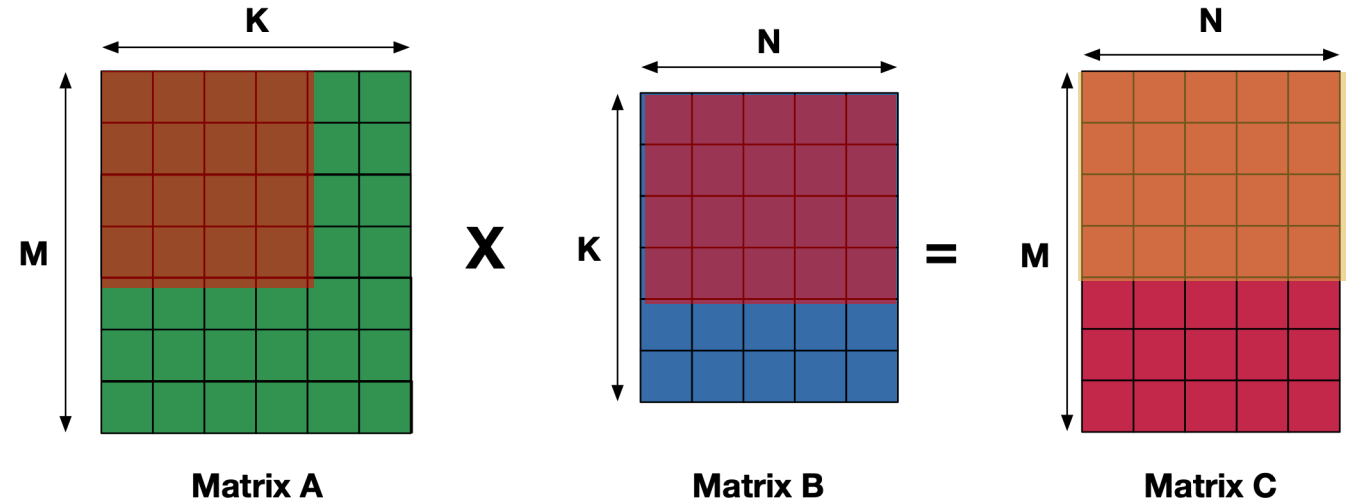
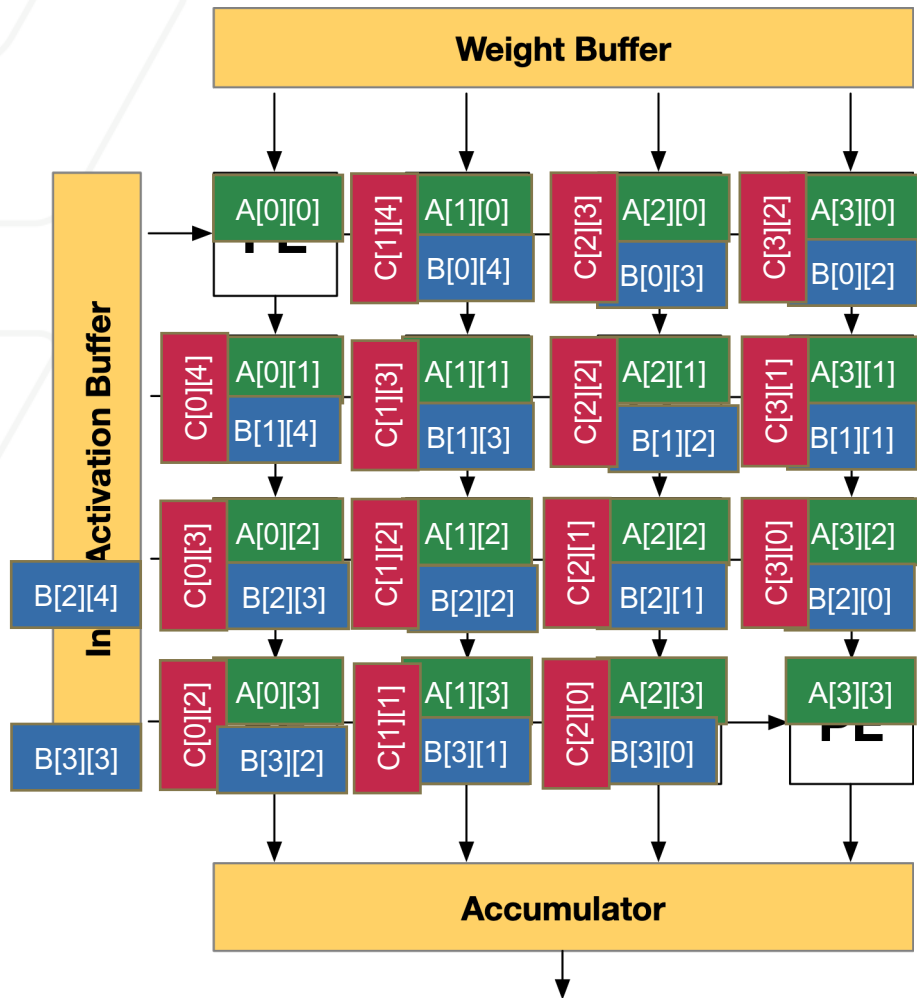
Three outputs



```
for m in range(4):
  for n in range(4):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

Mapping Choice: Tile Size – Option 2

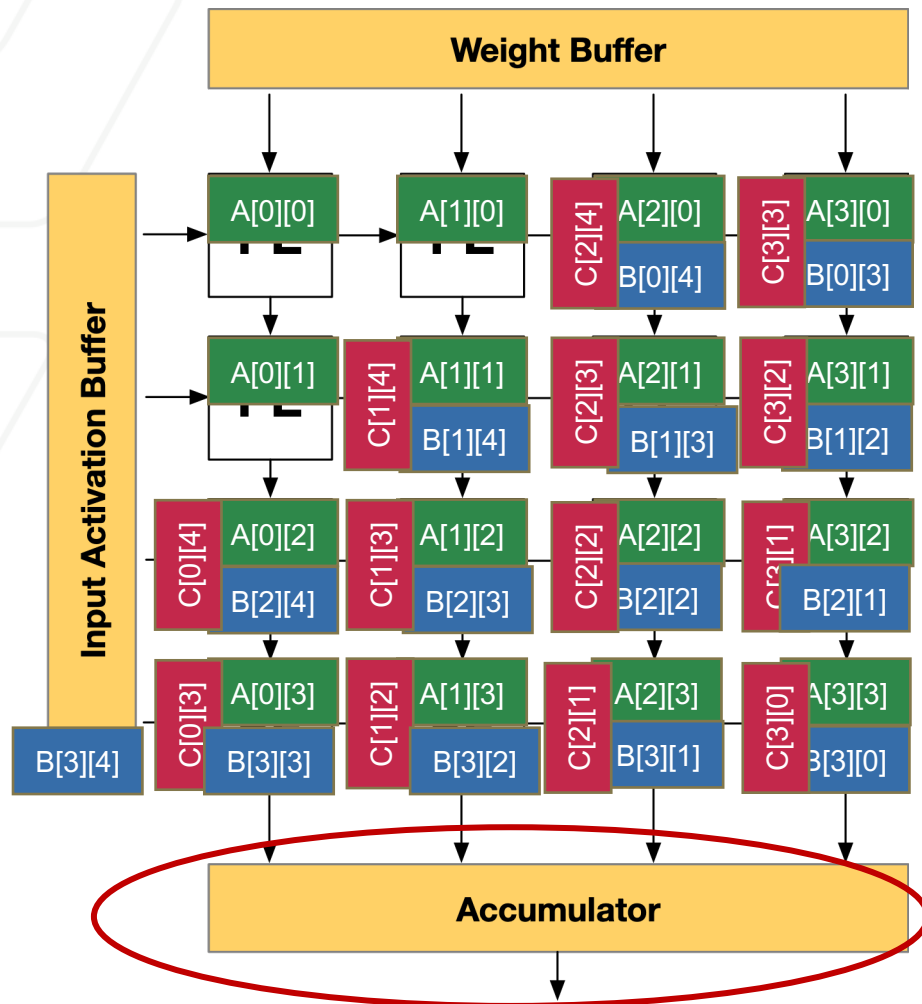
Phase 2:
Stream Activation Tensor



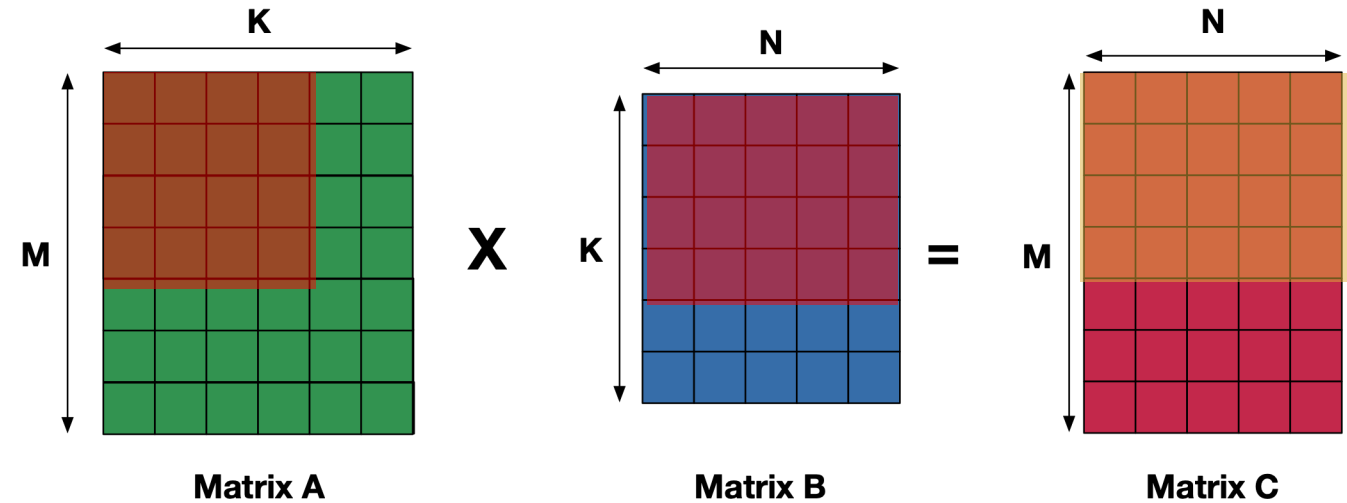
```
for m in range(4):
  for n in range(5):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

Mapping Choice: Tile Size – Option 2

Phase 2:
Stream Activation Tensor



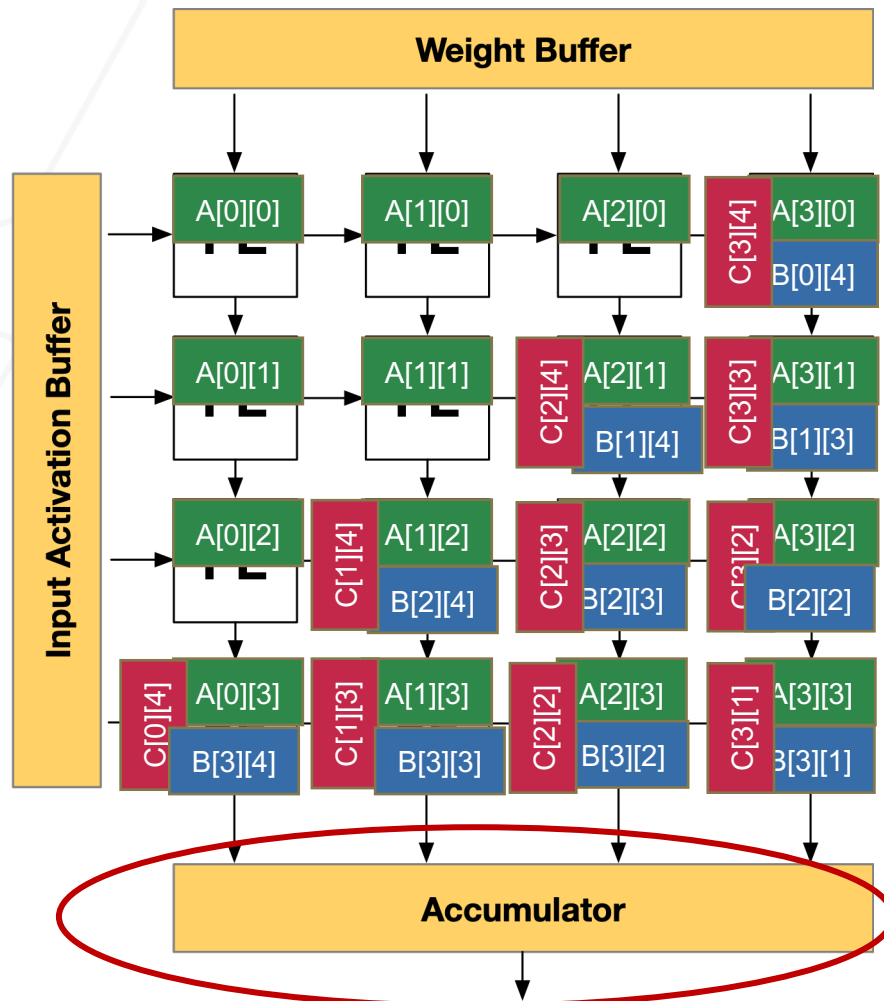
Four outputs



```
for m in range(4):
  for n in range(5):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

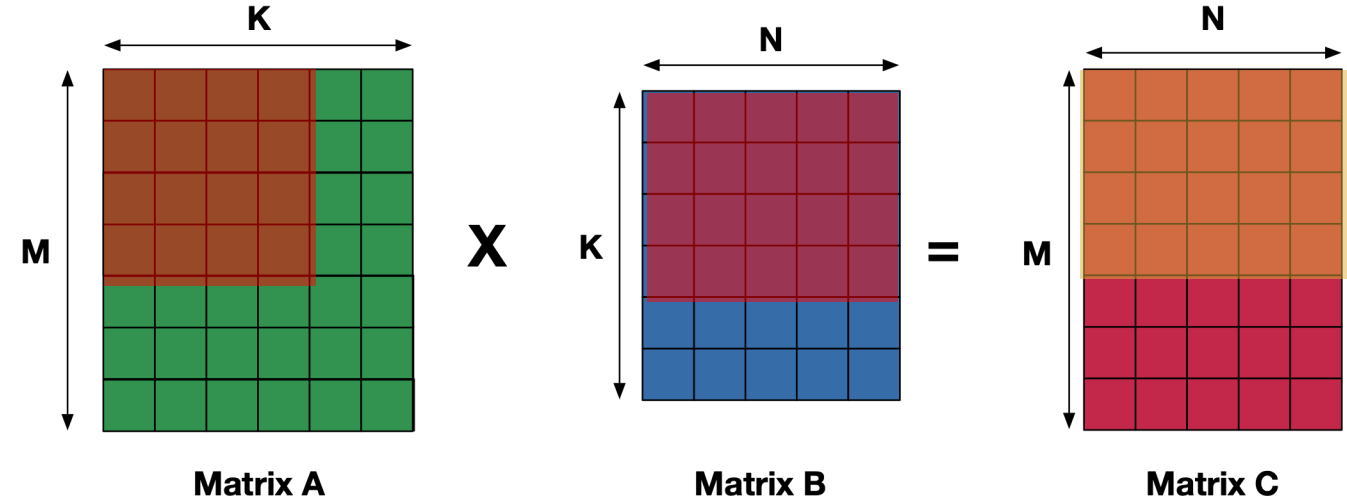
Mapping Choice: Tile Size – Option 2

Phase 2:
Stream Activation Tensor



Four outputs

Longer steady state!



```
for m in range(4):
  for n in range(5):
    for k in range(4):
      C[m][n] += A[m][k] * B[k][n]
```

Could we have chosen a dataflow to get even longer steady state?

B-stationary, A-streaming.

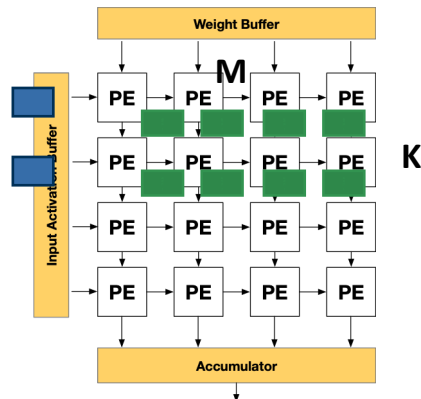
A-tile: 4x6

Mapping Choice: Parallelism Dimension

Suppose:

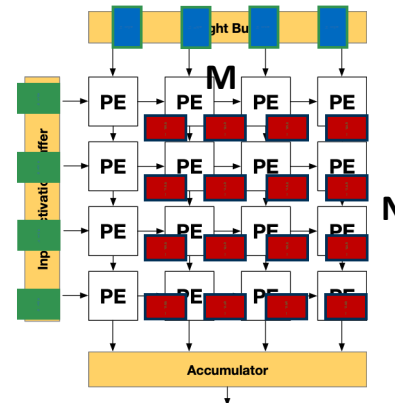
$$\begin{array}{c} M \\ \begin{array}{|c|c|c|c|} \hline \text{Green} \\ \hline \end{array} \end{array} \begin{array}{c} K \\ \begin{array}{|c|c|} \hline \text{Green} \\ \hline \end{array} \end{array} \times \begin{array}{c} K \\ \begin{array}{|c|c|c|c|} \hline \text{Blue} \\ \hline \end{array} \end{array} = \begin{array}{c} M \\ \begin{array}{|c|c|c|c|} \hline \text{Red} \\ \hline \end{array} \end{array} \begin{array}{c} N \\ \begin{array}{|c|c|c|c|} \hline \text{Red} \\ \hline \end{array} \end{array}$$

Which dataflow choice is better?



Parallelism: MK
(Weight Stationary)

X Low PE utilization

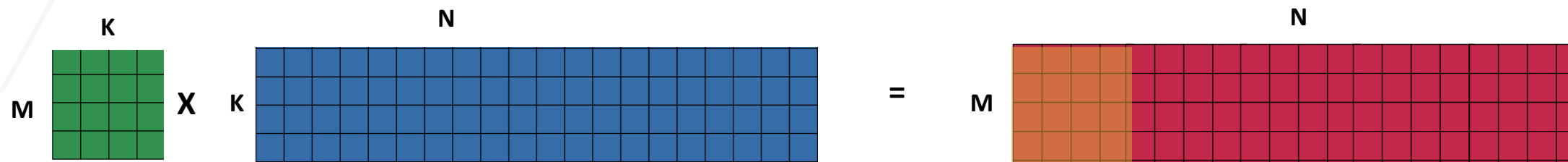


Parallelism: MN
(Output Stationary)

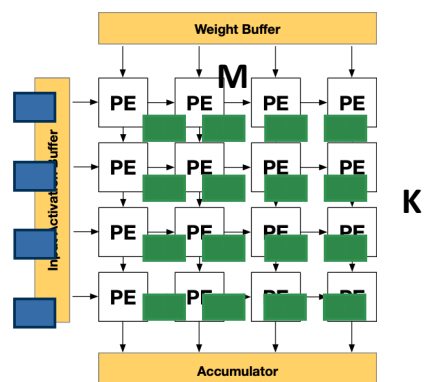
✓ Higher PE utilization

Mapping Choice: Parallelism Dimension

Suppose:

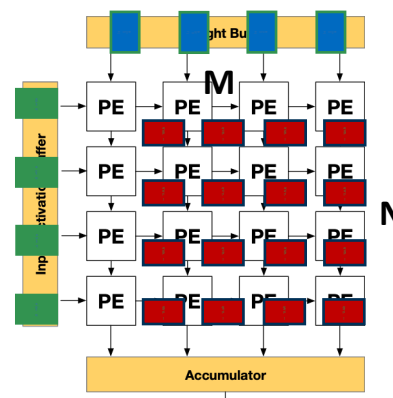


Which dataflow choice is better?



Parallelism: MK
(Weight Stationary)

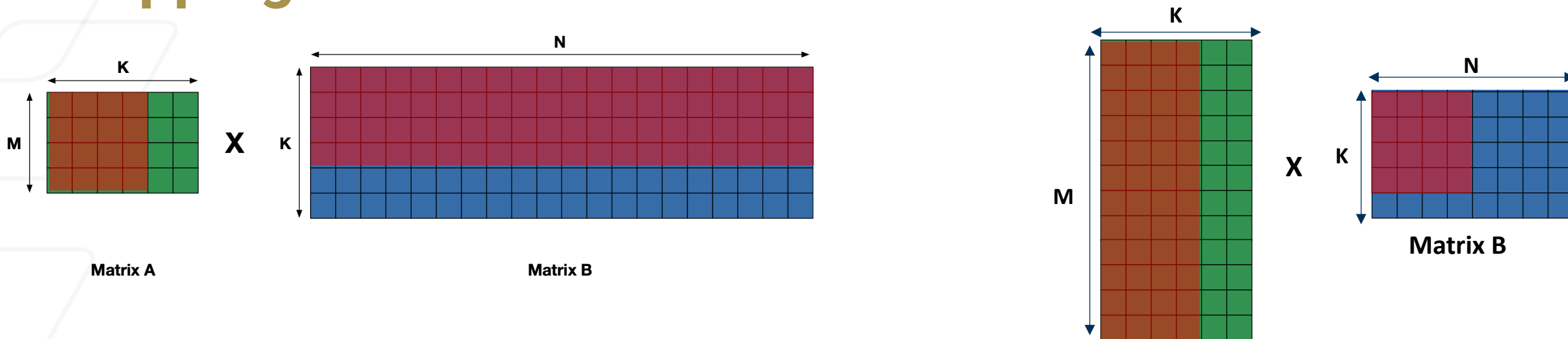
✓ Read A matrix only once!



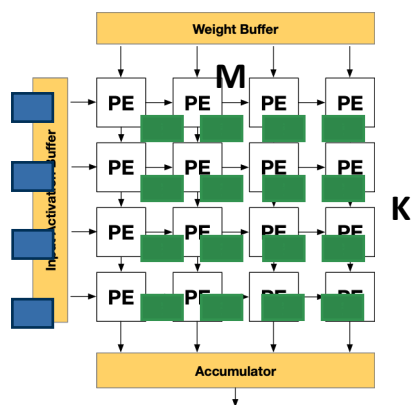
Parallelism: MN
(Output Stationary)

✗ Read Matrix A multiple times
for each tile of Matrix C

Mapping Choice: Parallelism Dimension

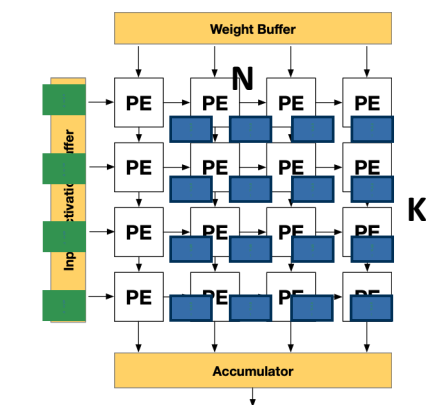


Which one is better?



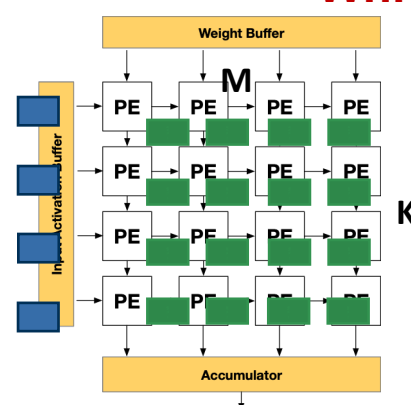
Parallelism Option 1: MK

✓ Longer steady state =>
Higher PE utilization

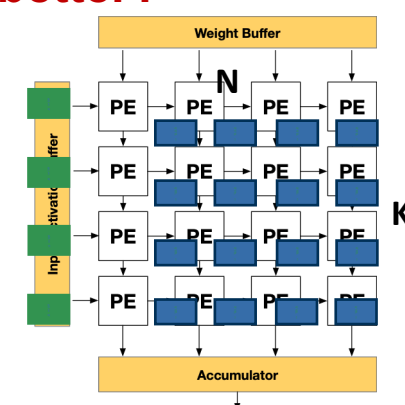


Parallelism Option 2: NK

Matrix A
Which one is better?



Parallelism Option 1: MK



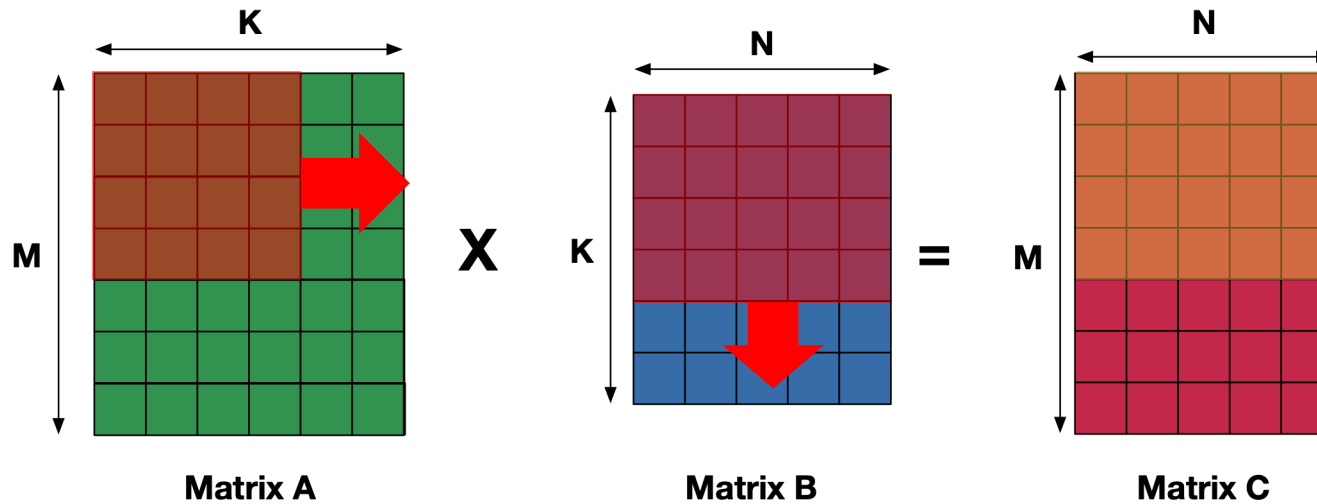
Parallelism Option 2: NK

✓ Longer steady state =>
Higher PE utilization

Mapping Choice: Processing Order

- When M is larger than 4, we have two choices for **next stationary-tile**

Option 1

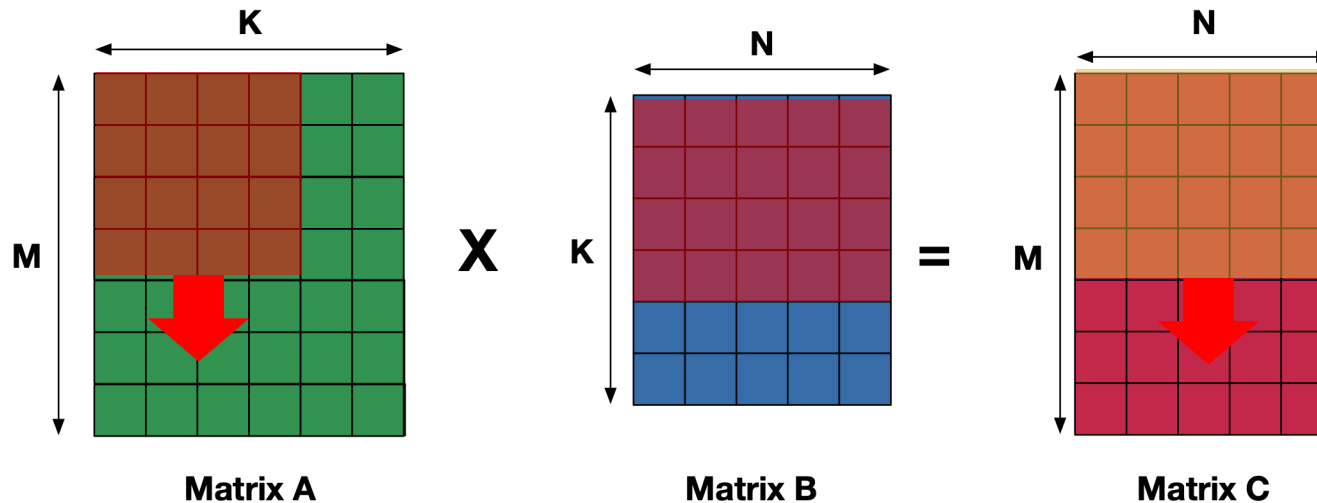


Trade-offs?

Compute full output before moving on to new outputs

This is done via accumulator at bottom of the systolic array

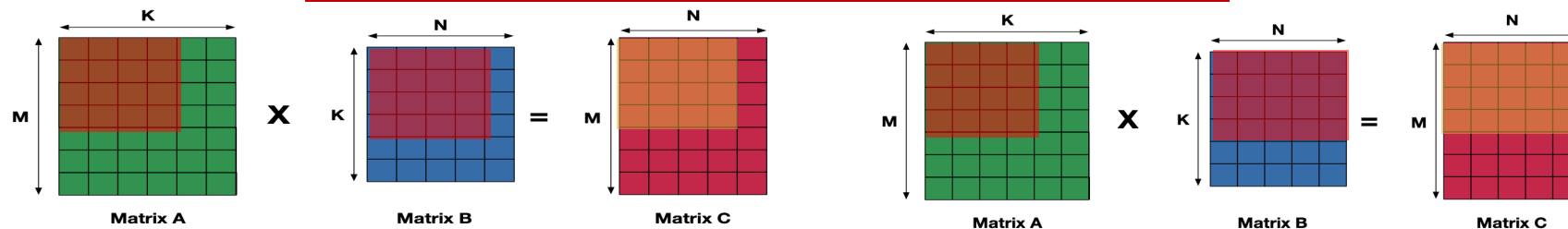
Option 2



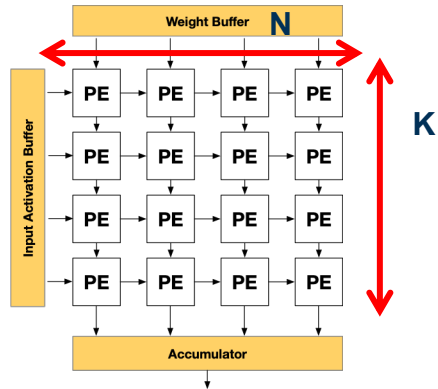
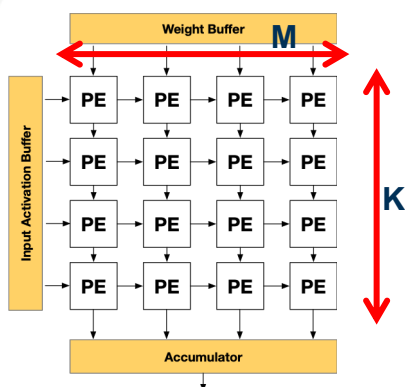
Higher utilization of the next MK tile.
(Three columns utilized in Option 2 vs two columns in Option 1)

Summary of Mapping Choices

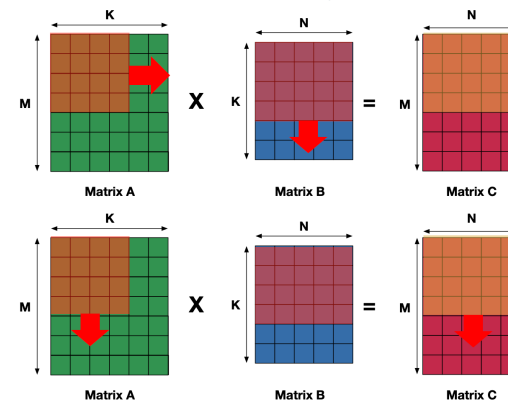
Tiling: How to partition the computation



Parallelism: Which dimension to parallelize?



Order: Where to move next?



Dataflow = processing order + parallel dimension

Mapping = Dataflow + Tile Sizes

Even a simple systolic array running MatMul has different dataflow/mapping choices → Huge trade-off space!

How to search?

Outline

- Systolic Array
- Dataflow
- **Flexible AI Accelerators**
- Takeaways

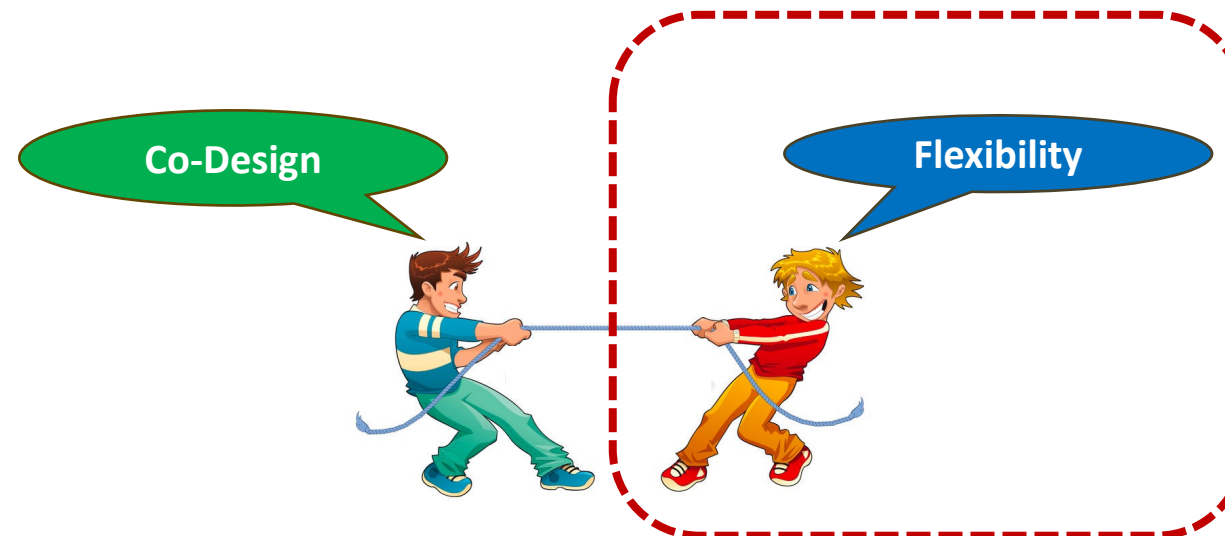
How to design and optimize AI Datacenters?

"AI is not just about algorithms and software; it's about creating the right hardware infrastructure that can enable these algorithms to run efficiently. **Hardware and software must be co-designed** to fully unleash the potential of AI."

— Jensen Huang, CEO of NVIDIA

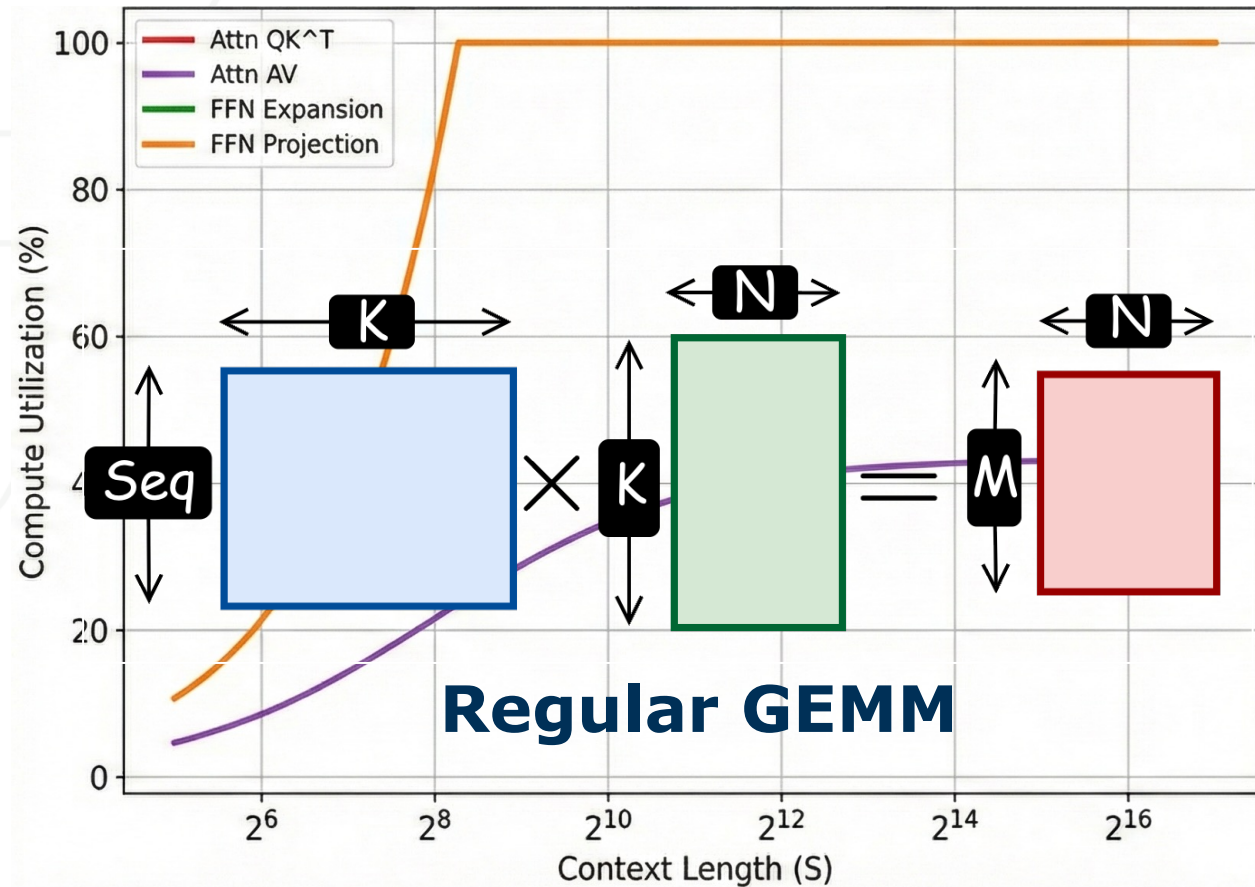
"The future of computing is going to be about **flexibility** — flexibility to create new architectures, flexibility to scale, flexibility to bring AI into every single thing that we do."

— Jensen Huang, CEO of NVIDIA

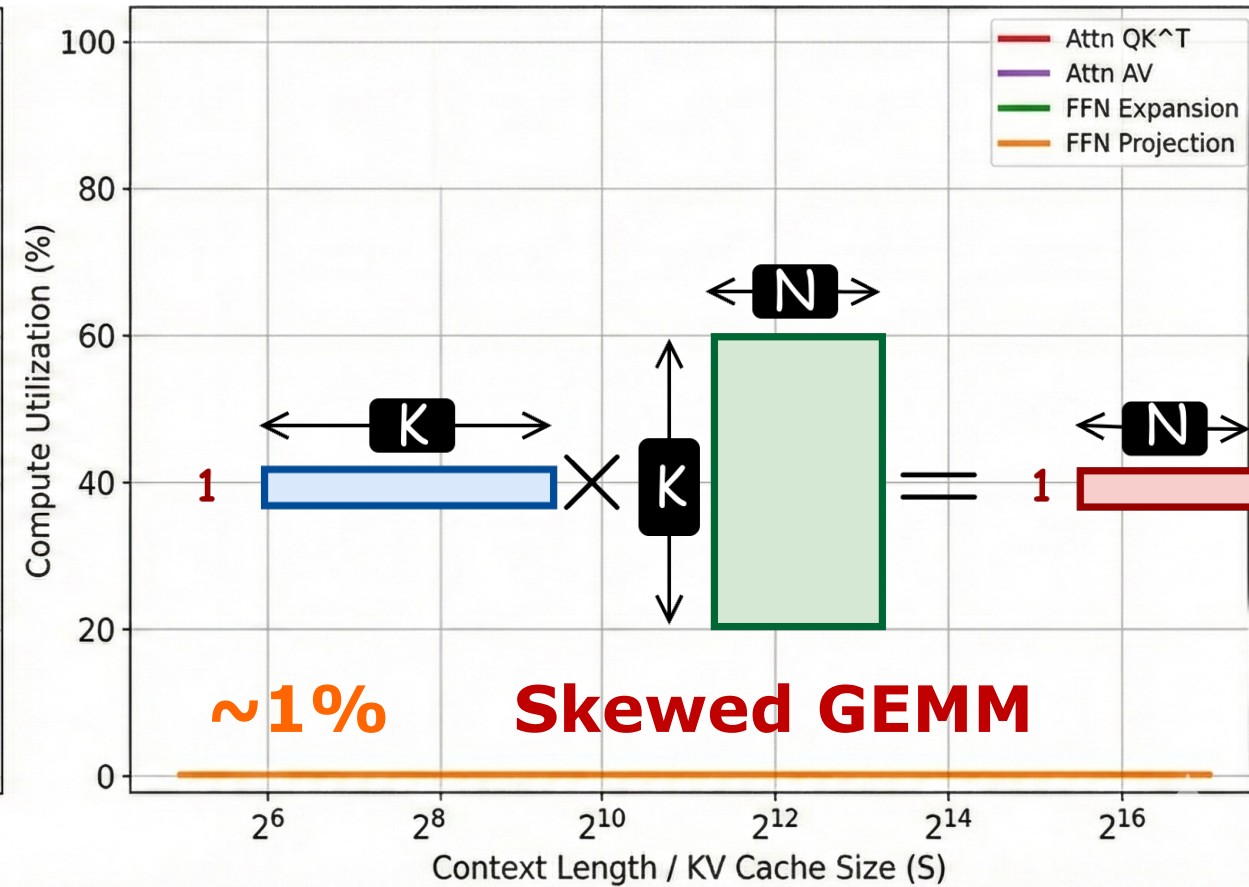


Challenge: Compute Under-Utilization with Diverse Workloads

Prefilling (H100) Large Context

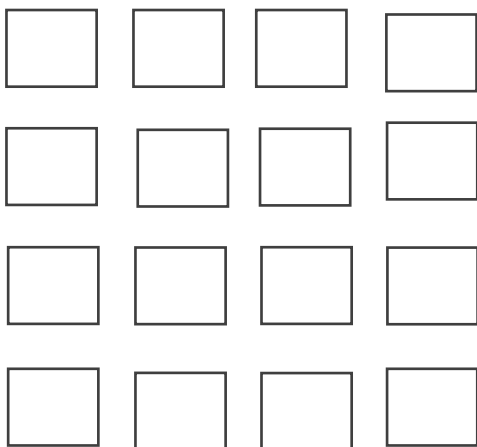
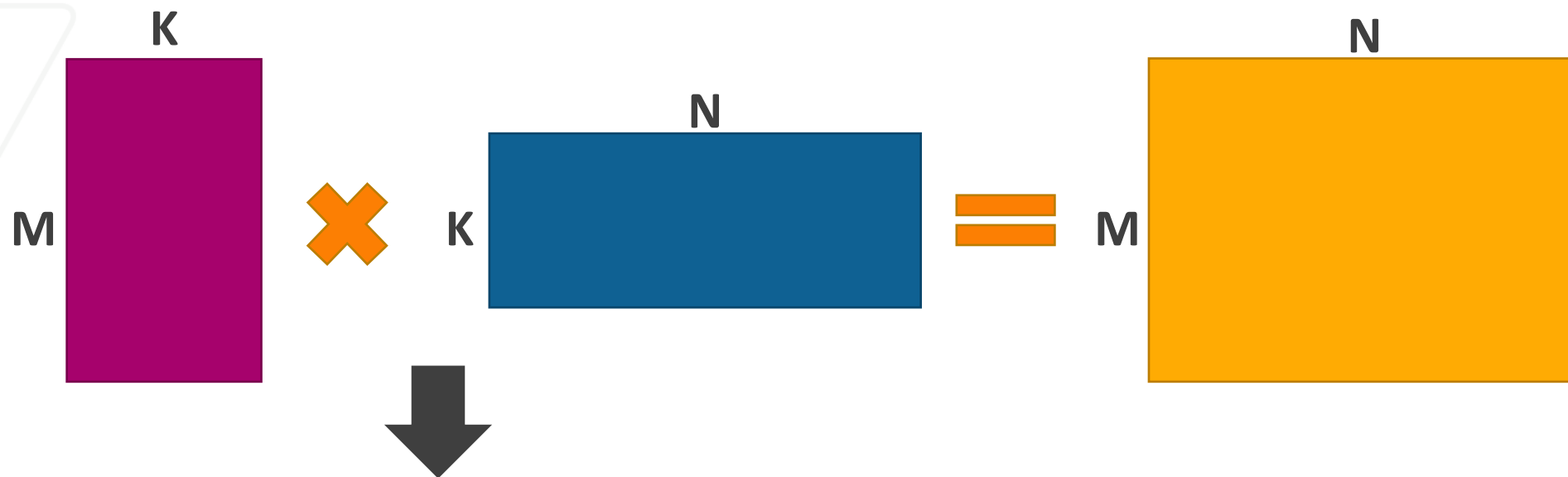


Decoding (H100)



Today's AI Chips are Not Good at **Skewed GEMM**

Example of GEMM Operation



e.g., TPU

Target Metric: Mapping Efficiency

Percentage of PEs with useful computations mapped over them

Utilization = *Mapping Efficiency x Activity*

(Activity < 1) if memory bound operator

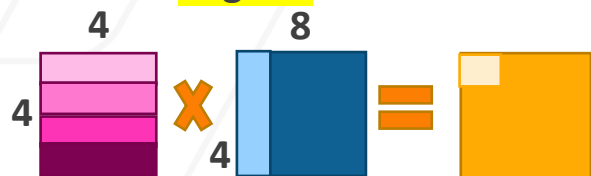
Mapping Examples

Communication

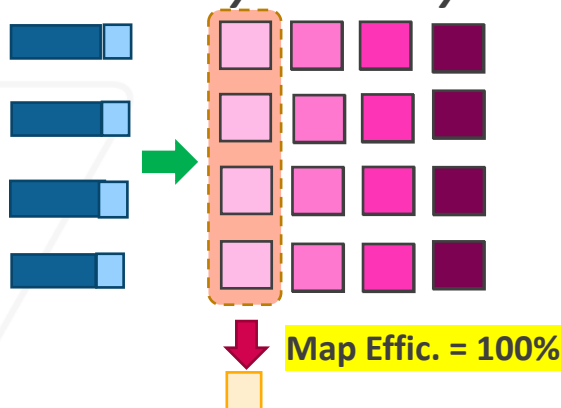
→ Distribute

→ Collect

Regular



Physical Array: 4x4

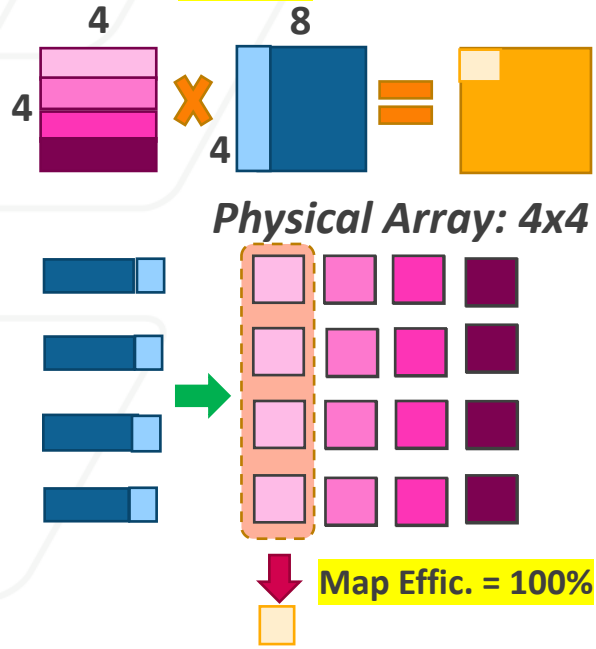


Distribute Row multicast

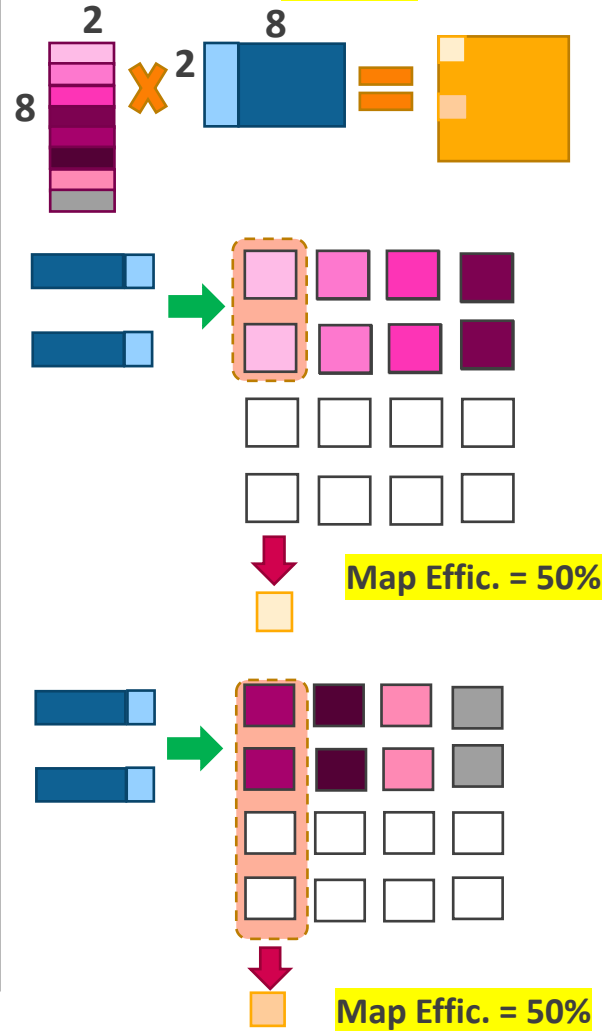
Collect Column Reduce

Mapping Examples

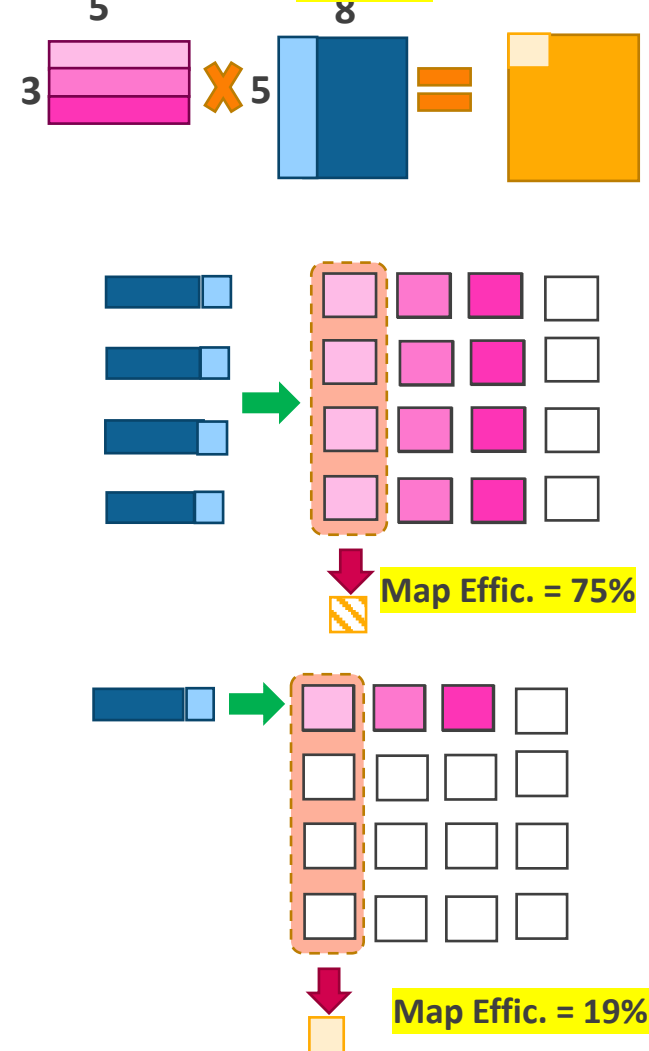
Regular



Skewed



Skewed



Distribute Row multicast
Collect Column Reduce

Challenges with Systolic Arrays

- **Rigid Microarchitecture**
 - Long fill and drain delays for larger arrays
- **Tight coupling of dataflow and microarchitecture**
 - Lacks flexibility to support diverse mappings
 - Inability to support data-dependent optimizations (e.g., sparsity)

Analogy: Opening a Buffet Restaurant*



Challenges

- Diverse Menu
- Need to keep trays full (i.e., need to have high throughput of cooking)

First Attempt: One Specialty Chef



Let's get



and



Naïve Buffet



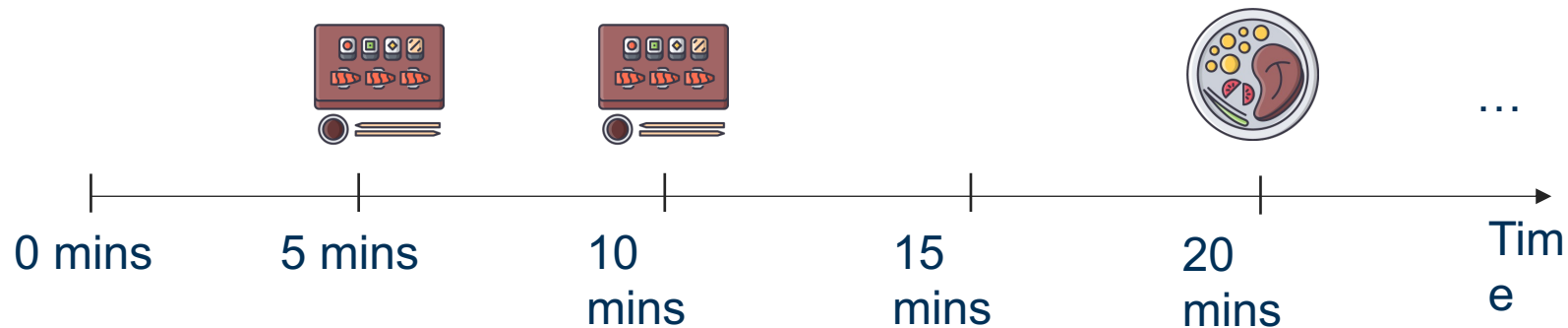
I am an expert in



I am not good at



Long wait time



Running a Buffet Restaurant



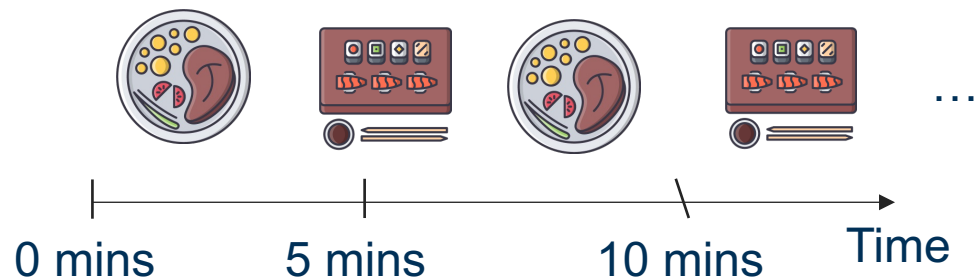
Let's get



and



Top Chef's Buffet



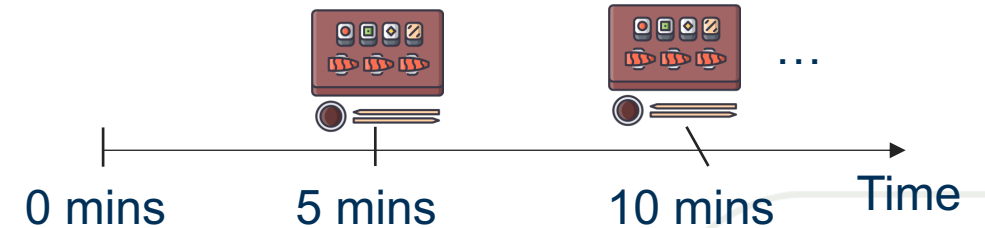
Divide-and-conquer Buffet



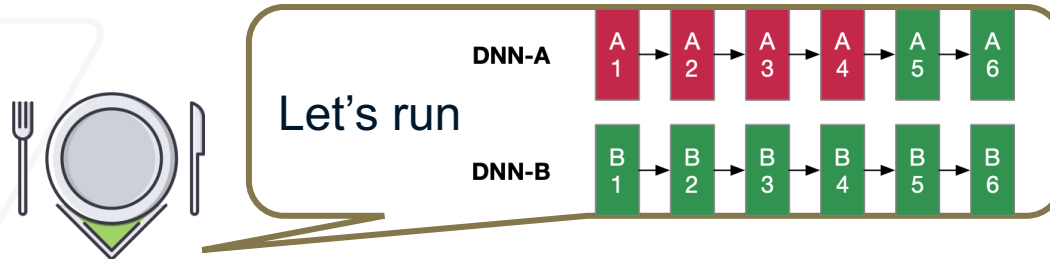
BBQ Chef



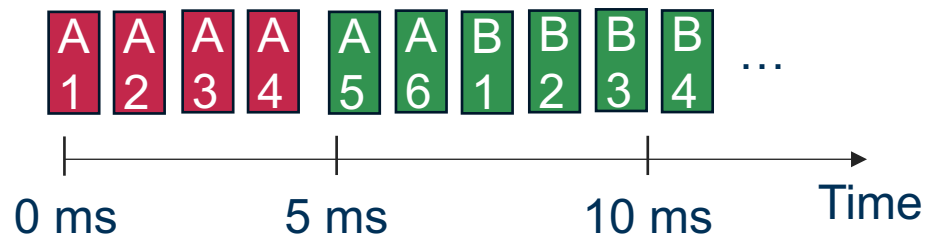
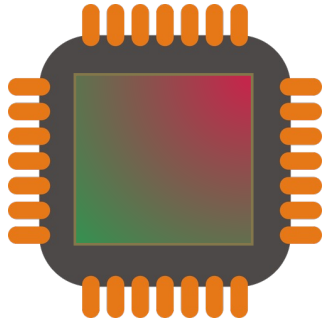
Sushi Chef



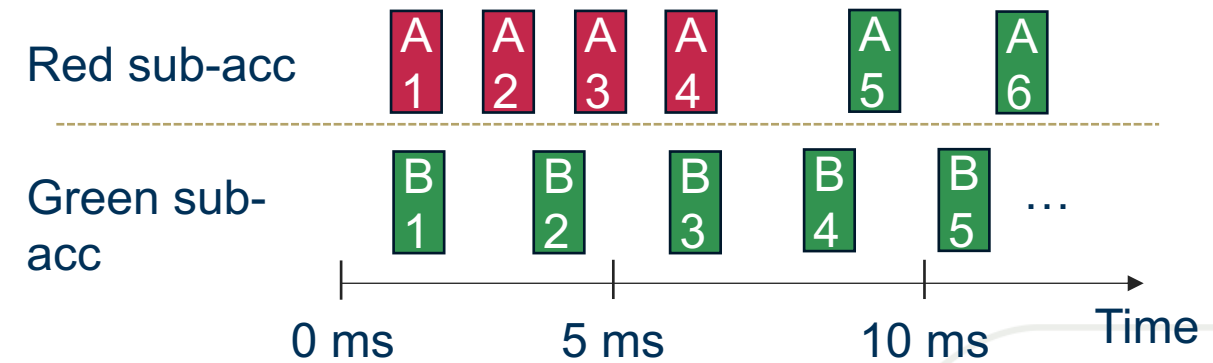
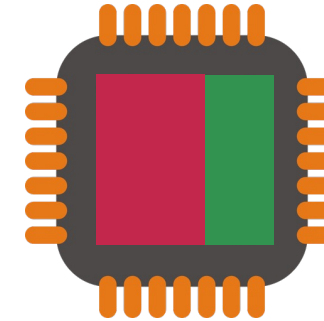
Running a Buffet Restaurant



Reconfigurable Accelerator



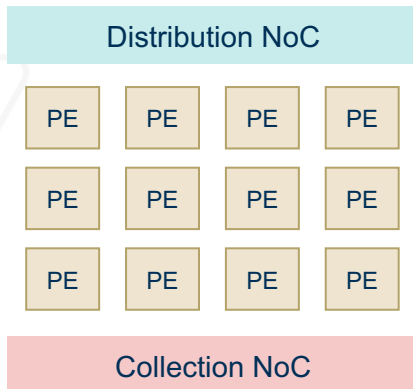
Heterogeneous Accelerator



Which one would be a better choice?

Diverse Workloads: Reconfigure or Go Dark?

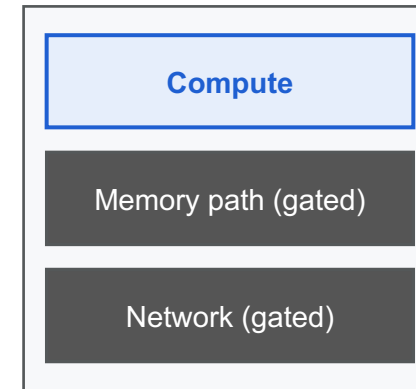
Option A: Reconfigurability



- Approach: One shared pool of PEs, reshaped per GEMM
- **Roots:** Flexible on-chip connectivity
- **Pays for:** area and energy of the flexible NoC, mapping complexity

**Case study: MAERI/SIGMA
(ASPLOS 2018 / HPCA 2019)**

Option B: Specialize + Dark Silicon



- Many specialized blocks on one die; power-gate whatever the current phase does not need
- **Roots:** post-Dennard, you cannot power every transistor at once* anyway
- **Pays for:** silicon area that sits dark much of the time

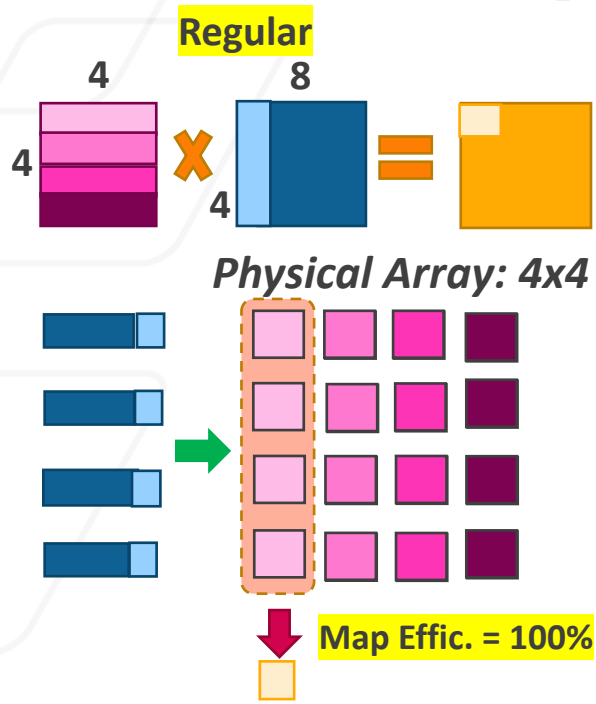
**Case study: OpenAI
Jalapeño (Hot Chips 2026)**

*Dark silicon: H. Esmailzadeh et al., "Dark Silicon and the End of Multicore Scaling," ISCA 2011.

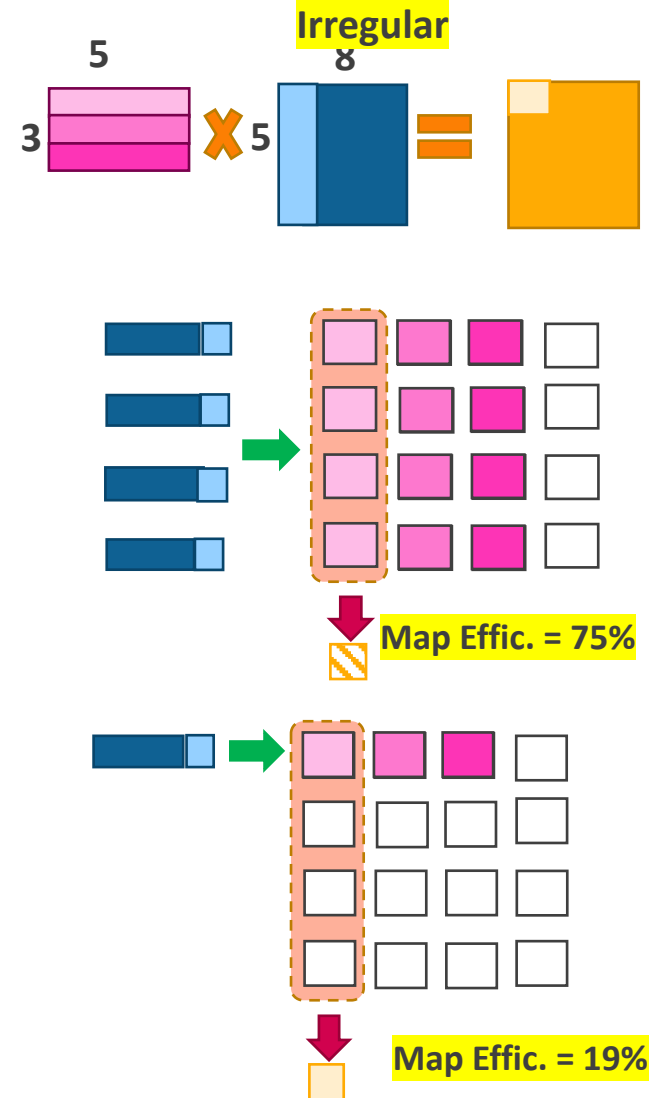
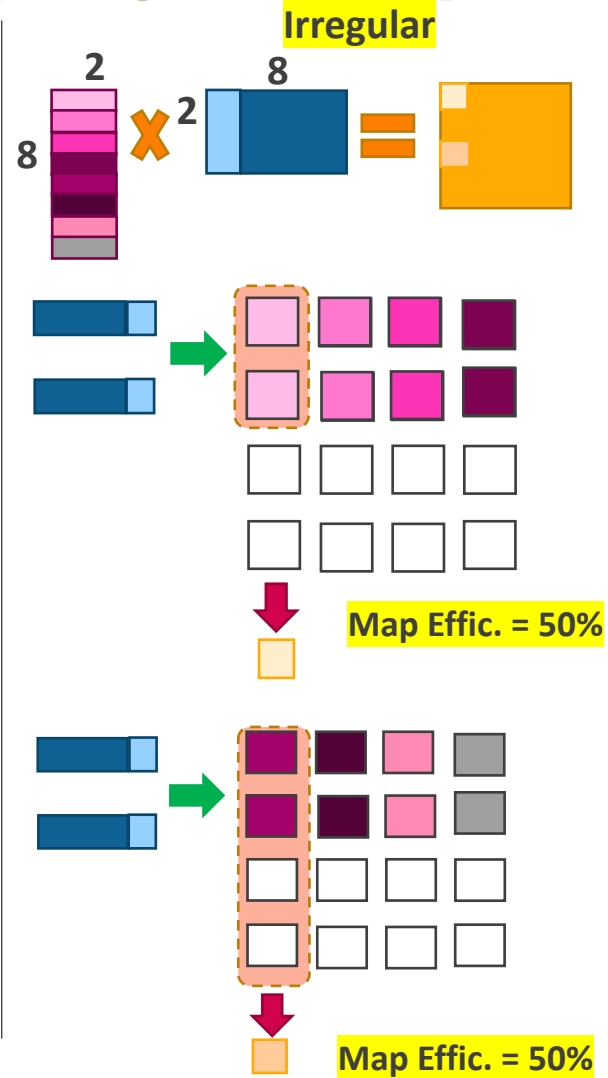
Outline

- Systolic Array
- Dataflow
- **Flexible AI Accelerators**
 - **Reconfigurable Accelerators**
 - Heterogeneous Accelerators
- Takeaways

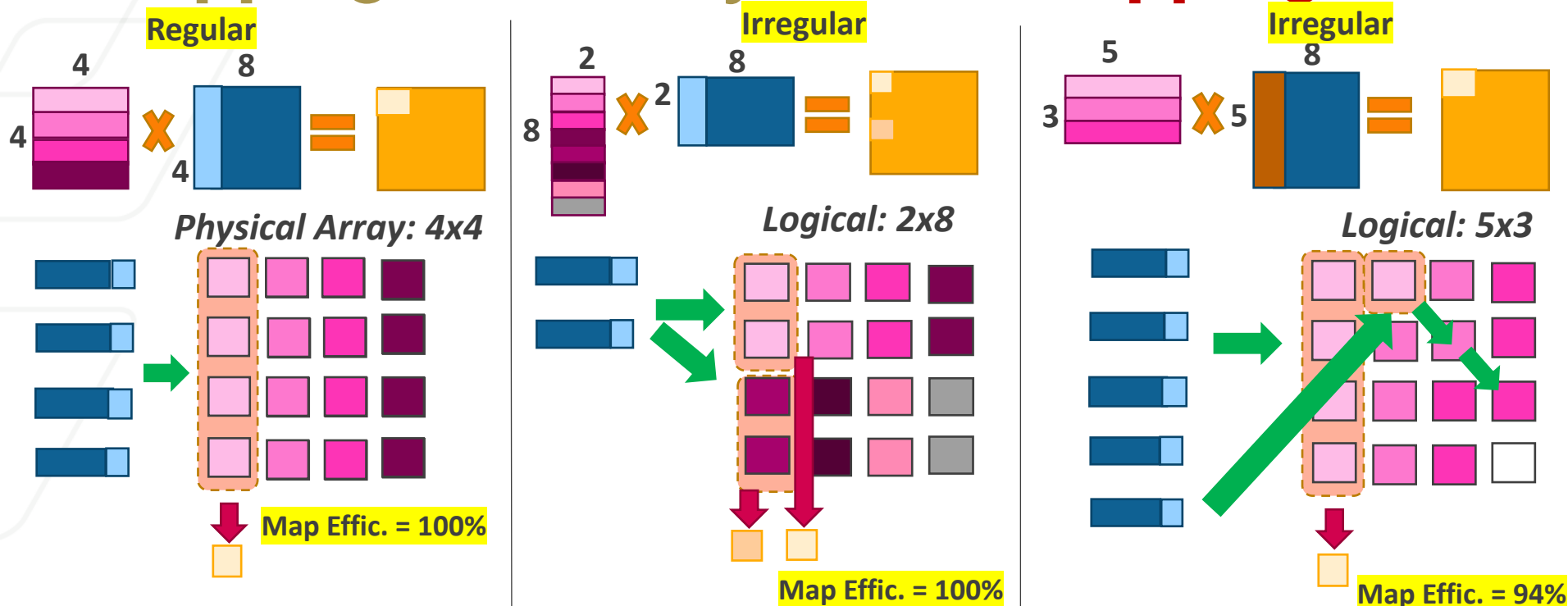
Recall: Mapping Examples



Distribute Row multicast
Collect Column Reduce



Mapping Efficiency needs Mapping Flexibility



How to support Mapping Flexibility?

Distribute	Row multicast	Spatial Multicast	Multicast to non-neighbors
Collect	Column Reduce	Multiple Parallel	Variable Length

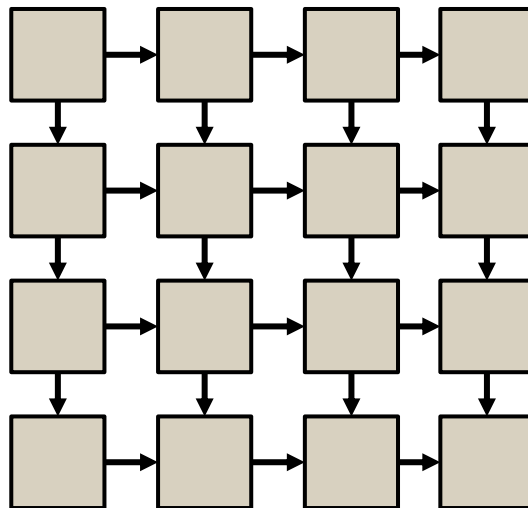
Flexible data distribution and reduction

Summary of Mapping Challenge

- Irregular (and Sparse) GEMMs introduce under-utilization
- Desired Features for 100% Utilization
 - **Distribution**
 - Variable Sized **Multicast**
 - Multiple in Parallel
 - [For Sparsity: Only send non-zero weights/activations] → Not in scope of today's class
 - **Collection**
 - Variable Sized **Reduction**
 - Multiple in Parallel

Enabling Dataflow Flexibility

Inputs and Weights



Outputs

4x4 Rigid Systolic Array

Enabling Dataflow Flexibility

Functionality:

Distribute weight and input operands to any PE
(pattern depends on *Mapping*)

Optimization:

Bandwidth *amplification* via in-network *multicast*

Implementation:

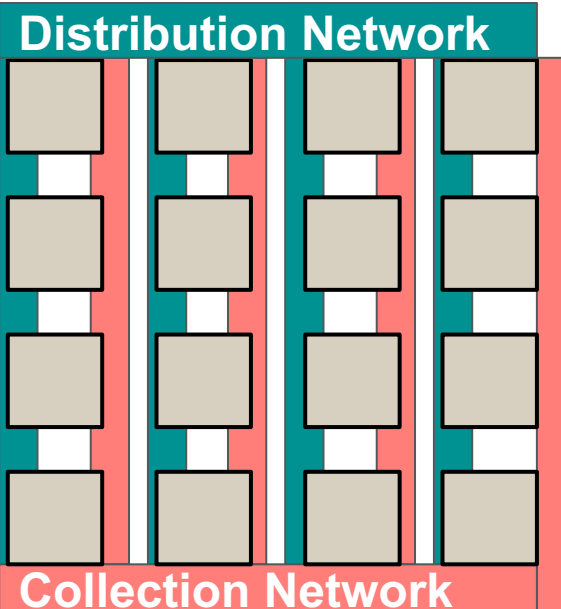
Topology: Bus/Store-and-Fwd/Tree/..

Latency: $O(1)$ to $O(N)$

No-stall Bandwidth: $O(\sqrt{N})$ - $O(N)$

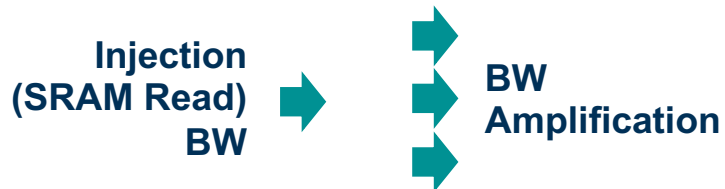
Inputs and Weights

↓ few to many



↓ many to few

Outputs



Functionality:

Collection final (or partial) outputs from the PEs
(pattern depends on *Mapping*)

Optimization:

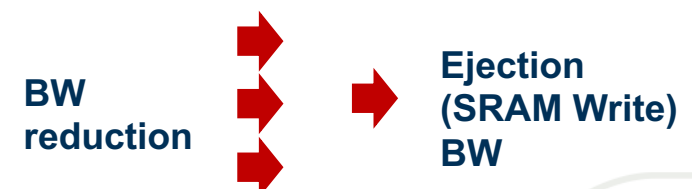
Bandwidth *reduction* via in-network *accumulation/summation*

Implementation:

Topology: Bus/Reduce-and-Fwd/Tree/...

Latency: $O(1)$ / $O(\log(N))$ / $O(\sqrt{N})$

No-stall Bandwidth: $O(\sqrt{N})$ - $O(N)$



4x4 Flexible Spatial Array

© Tushar Krishna, School of ECE, Georgia Institute of Technology

Family of Flexible AI Accelerators

Accelerator	Distribution Network	Collection Network	Level of Flexibility
Eyeriss (ISSCC 2016)	(multiple) Bus $O(1)$, param	Bus $O(1)$, 1	Partially-Flexible Homogeneous Clusters
TPU (ISCA 2017)	Store-and-Fwd $O(\sqrt{N})$, $\sqrt{N}$	Reduce-and-Fwd $O(\sqrt{N})$, $\sqrt{N}$	Fixed Homogeneous Clusters
NVDLA (2017)	Bus $O(1)$, 1	Tree $O(\log N)$, 1	Fixed Homogeneous Clusters
MAERI (ASPLOS 2018)	Chubby-Tree $O(1)$, param	Augmented Reduction Tree (ART) $O(\log N)$, param	Fully-Flexible Homogeneous Clusters
Eyeriss_v2 (JETCAS 2019)	Hierarchical Mesh $O(\sqrt{N})$, param	Hierarchical Mesh $O(\sqrt{N})$, param	Fully-Flexible Homogeneous Clusters <i>(with Sparsity Controller)</i>
SIGMA (HPCA 2020)	Benes $O(1)$, N	Forwarding Adder Network (FAN) $O(\log(N))$, $N/2$	Fully-Flexible Heterogeneous Clusters <i>(with Sparsity Controller)</i>
Flexagon (ASPLOS 2023)	Benes $O(1)$, N	Merge Reduction Network (MRN) $O(\log(N))$, $N/2$	Fully-Flexible Heterogeneous Clusters <i>(with Sparsity Controller)</i>
FEATHER (ISCA 2024)	Crossbar $O(1)$, N	BIRRD $O(\log(N))$, $N/2$	Fully-Flexible Heterogeneous Clusters <i>(with Layout Reordering)</i>

Latency, Injection Bandwidth

Latency, Ejection Bandwidth

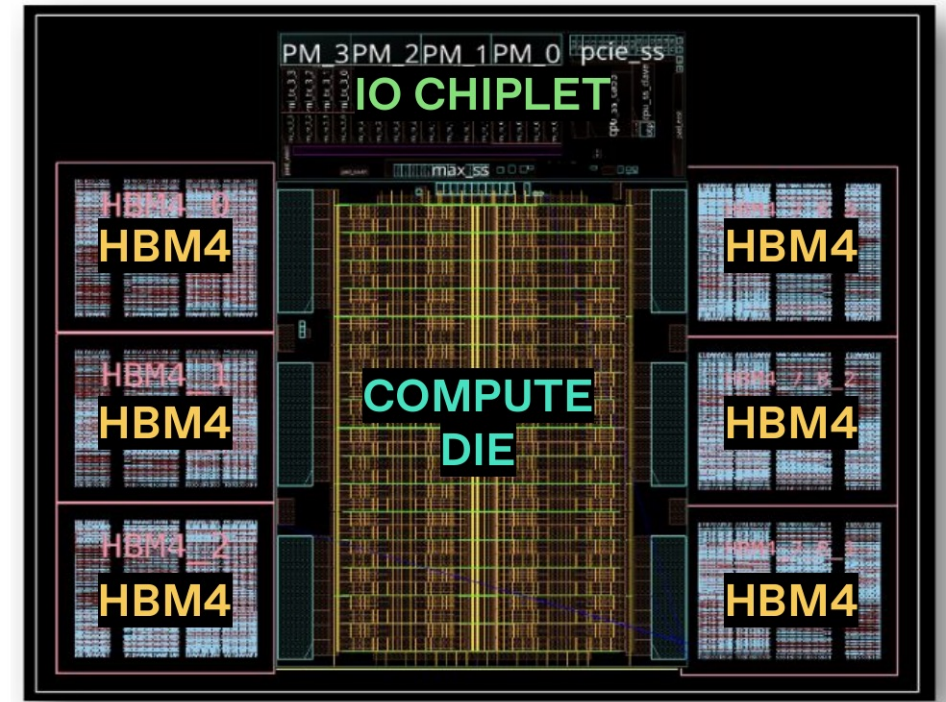
Outline

- Systolic Array
- Dataflow
- **Flexible AI Accelerators**
 - Reconfigurable Accelerators
 - **Heterogeneous Accelerators**
- Takeaways

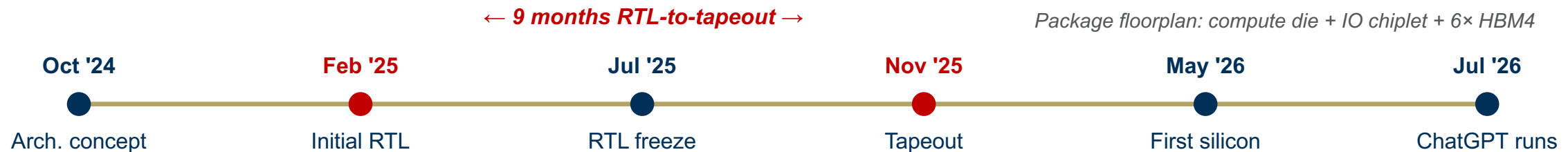
Case Study: OpenAI Jalapeño

- Inference-only ASIC, designed for OpenAI's own serving workloads (ChatGPT, Codex)
- Built with Broadcom (silicon + Tomahawk 6 network) and Celestica (racks)
- Initial RTL → tapeout in **9 months**, with AI-assisted HW design

Matrix compute	13.4 PFLOP/s MXFP4 · 3.4 PFLOP/s MXFP8
Memory	6× HBM4: 216 GiB, 15.4 TB/s
Scale-up network	128 chips @ 600 GB/s (local) 2,048 chips @ 200 GB/s (global)
Power	700 W package TDP



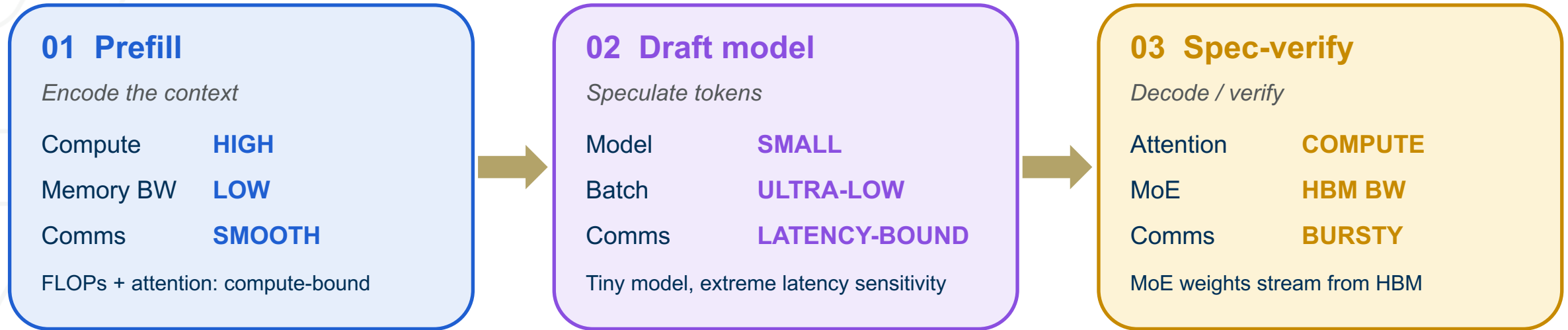
Package floorplan: compute die + IO chiplet + 6× HBM4



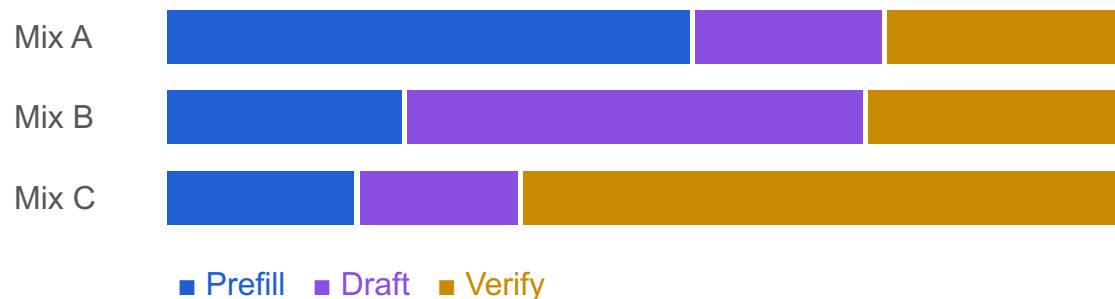
Source: R. Ho, R. Narayanaswami, C. Leary, "You Can Just Build Things ... Chips: OpenAI Jalapeño," Hot Chips 2026.

© Tushar Krishna, School of ECE, Georgia Institute of Technology

One Inference Request, Three Hardware Regimes



The phase mix keeps changing



Ratio depends on:

- Model architecture and context length
- Speculative acceptance rate, attention algorithm
- Latency vs. throughput target (SLA)

Same lesson as Challenge 2: no single fixed compute : memory : network ratio fits all

Jalapeño's Bet: Dark Silicon > Idle Accelerators

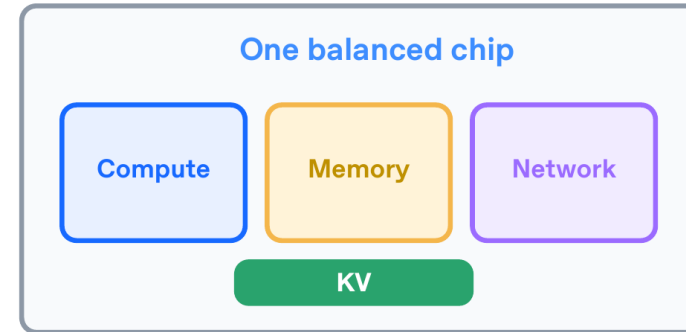
Disaggregated specialization



Network + synchronization at every phase boundary

Specialized hardware, but KV state must move between systems

Unified, internally heterogeneous



Activate the right ratio by phase; unused units go dark

Heterogeneity moves inside the chip—not across a KV-moving network.

Idle accelerators

still burn baseline power: package, HBM, I/O, network, cooling

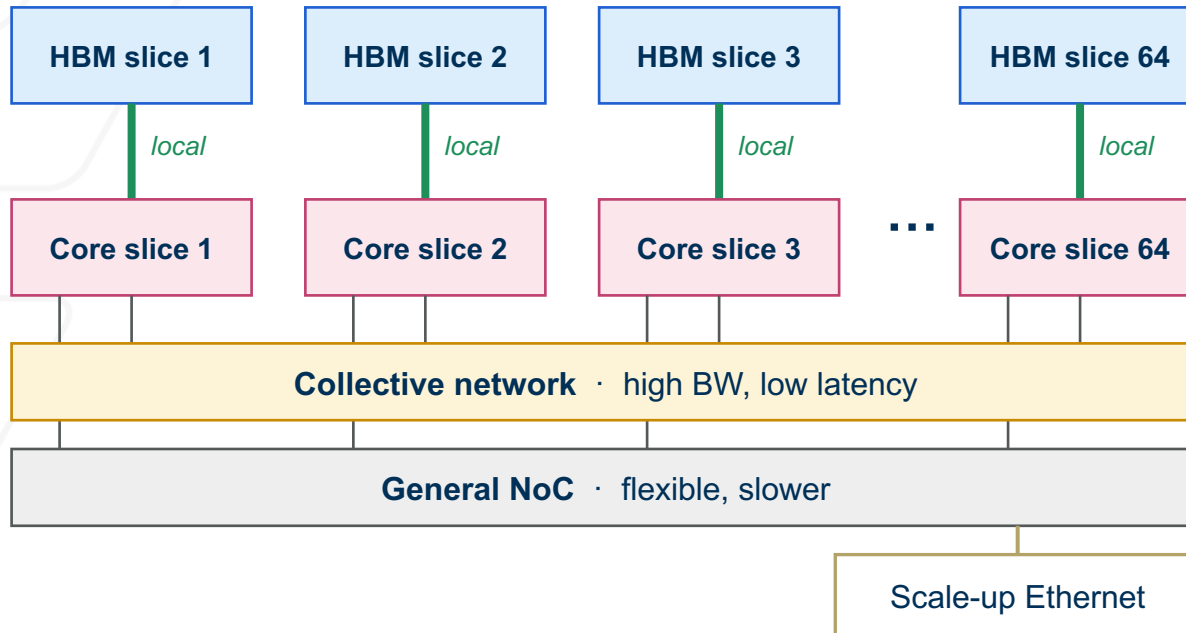
Gated blocks on one chip
cost area, but little power

KV cache stays local

no KV transfer + sync at every phase boundary

Dark silicon is cheaper than idle accelerators: keep KV local

Inside Jalapeño: Locality-First Design



- **Pair each core slice with an HBM slice:** common operands stay on the fast local path
- **Core $\approx$ thread block:** tensor + SIMD + scalar engines share L1
- **Explicit placement + distributed control:** no contended unified memory, no global fences

Raw HBM BW => extraordinary token rates

$$1 \text{ PB/s} \div 0.5 \text{ TB} = 2,000+$$

128-chip aggr HBM4 BW $\div$ 1T params @ 4b = full-model reads/s

=> Theoretically 1000 - 2000 tokens/sec/user

Reality: nowhere close. Compute stalls waiting for operands stuck in network, memory, and global reductions.

Connection to dataflow & mapping

- A spatial architecture: performance = mapping (placement, scheduling, collectives)
- OpenAI uses frontier AI to search mappings: **1.5–1.8 $\times$** over expert-written GPT-OSS attention / MoE kernels

Results

1.5–1.9×

higher perf/W
at peak throughput

1.7–3.6×

lower end-to-end
request latency

2.1–4.1×

higher interactivity
(lower min. time-between-
tokens)

700 W

package TDP
vs. 1.2–1.4 kW (GB200/GB300)

Vendor-reported: SemiAnalysis InferenceX, GPT-OSS-120B / DeepSeek R1 / Kimi K2.5, 8k in / 1k out, FP4 weights, package-TDP normalized. Jalapeño used single-token prediction; some baselines used multi-token prediction.

Outline

- Systolic Array
- Dataflow
- Flexible AI Accelerators
- **Takeaways**

Reconfigurability vs. Dark Silicon

	Reconfigurable fabric <i>MAERI / SIGMA / FEATHER</i>	Heterogeneous + dark silicon <i>OpenAI Jalapeño</i>
What adapts?	Network-on-Chip + mapping of one shared PE array	Which on-chip blocks are powered
Granularity	Per layer / per GEMM	Per phase
Cost of idle resource	Low: same PEs reused	Area; little power when gated
Overhead	Flexible-NoC area/energy; mapper complexity	Silicon area that sits dark
Wins when...		

Not mutually exclusive: flexible fabrics inside blocks, dark silicon across blocks

But .. Jalapeno fixes the compute : memory : network ratio at design time. What if the workload mix drifts outside it?

Summary of AI Accelerators

- AI models need massive compute and memory for efficient training and serving
- **Opportunity:**
 - Predictable Compute and Memory Behavior
 - Opportunity to leverage Data Reuse
- **Compute Substrate: Systolic Arrays**
 - simplest structure to run matrix multiplications
 - common in custom AI accelerators like Google TPU
- **Dataflow**
 - Fine-grained scheduling over space and time
 - Key metric of optimization in AI accelerators
- **Open Problems**
 - AI models (especially LLMs) memory-bound (i.e., low data reuse)
 - Handling sparse models